



StudyLink

Gruppo 31 A.A. 2025/2026

Matteo Gattobigio	0001117104
Lucilla Lucchi	0001116767
Federico Porpora	0001114450

Abstract

Il progetto **StudyLink** è un'applicazione nata per aiutare gli studenti universitari a trovare e formare gruppi di studio dal vivo, rendendo semplicissimo l'incontro tra colleghi.

La piattaforma punta tutto sulla trasparenza: ogni studente ha un Profilo personale chiaro e visibile. L'applicazione gestisce le sessioni di studio in modo flessibile, permettendo agli Utenti di alternarsi in due ruoli chiave:

- **L'Organizzatore**, che prende l'iniziativa e crea l'Evento scegliendo la data, l'orario, il luogo d'incontro e i posti disponibili;
- **Il Partecipante**, che sfoglia la Bacheca e si prenota per i gruppi già avviati occupando i posti liberi.

Inoltre, per facilitare l'organizzazione, tutti gli iscritti a un Evento hanno a disposizione una Chat di gruppo dedicata. È uno spazio comodo e privato per scambiarsi appunti, materiali di studio o mettersi d'accordo sui dettagli prima di vedersi di persona.

Indice

Abstract	1
1 Analisi dei requisiti	4
1.1 Raccolta dei requisiti	4
1.2 Requisiti di sistema	5
1.2.1 Requisiti funzionali	5
1.2.2 Requisiti non funzionali	6
1.3 Analisi del dominio	7
1.3.1 Vocabolario	7
1.4 Analisi dei requisiti	9
1.4.1 Modello dei casi d'uso	9
1.4.2 Scenari	10
1.5 Analisi del Rischio	23
1.5.1 Tabella valutazione dei beni	23
1.5.2 Tabella minacce e controlli	23
1.5.3 Analisi tecnologica della sicurezza	24
1.5.4 Security use case e misuse case	25
1.5.5 Requisiti di sicurezza	28
1.5.6 Aggiornamento requisiti di sistema	28
1.5.7 Aggiornamento vocabolario	29
1.5.8 Casi d'uso aggiornati	29
1.5.9 Diagramma dei casi d'uso aggiornato	30
1.5.10 Integrazione con sistemi esterni	30
2 Analisi del problema	31
2.1 Analisi documento dei requisiti: Analisi delle funzionalità	31
2.1.1 Tabella delle funzionalità	31
2.1.2 Tabelle informazioni/flusso	32
2.2 Analisi documento dei requisiti: Analisi dei vincoli	41
2.2.1 Tabella dei vincoli	41
2.3 Analisi documento dei requisiti: Analisi delle interazioni	43
2.3.1 Tabella delle maschere	43
2.3.2 Tabella dei sistemi esterni	43
2.4 Analisi dei ruoli e responsabilità	45
2.4.1 Tabella dei ruoli	45
2.4.2 Utente: Tabella Ruolo-Informazioni	45
2.4.3 Organizzatore: Tabella Ruolo-Informazioni	46
2.4.4 Partecipante: Tabella Ruolo-Informazioni	46
2.5 Scomposizione del problema	47
2.5.1 Tabella scomposizione delle funzionalità	47
2.5.2 Gestione Profilo: Tabella Sotto-Funzionalità	47
2.5.3 Gestione Evento: Tabella Sotto-Funzionalità	47
2.5.4 Utilizzo Chat: Tabella Sotto-Funzionalità	47
2.6 Modello del dominio	48
2.6.1 Modello del dominio: Utente	48

2.6.2	Modello del dominio: Evento	48
2.6.3	Modello del dominio: Filtro	49
2.6.4	Modello del dominio: Chat	49
2.6.5	Modello del dominio: Log	50
2.7	Architettura logica	52
2.7.1	Struttura	52
2.7.2	Interazioni	56
2.7.3	Comportamento	61
2.8	Piano di lavoro	62
2.8.1	Caratteristiche primo prototipo (Giugno 2026)	63
2.8.2	Sviluppi futuri	63
2.9	Piano del collaudo	64
2.9.1	Collaudo: Evento	64
2.9.2	Collaudo: Profilo	66
3	Progettazione	69
3.1	Progettazione architetturale	69
3.1.1	Requisiti non funzionali	69
3.1.2	Scelta dell'architettura	69
3.1.3	Scelte tecnologiche	69
3.1.4	Diagramma dei package	70
3.1.5	Diagramma dei componenti	71
3.2	Progettazione di dettaglio	72
3.2.1	Struttura	72
3.2.2	Interazioni	82
3.2.3	Interfacce utente	88
3.3	Progettazione della persistenza	91
3.4	Progettazione del collaudo	93
3.4.1	Test di unità: Evento	93
3.4.2	Test di unità: Profilo	96
3.5	Progettazione del deployment	100
3.5.1	Artefatti	100
3.5.2	Target di deploy	100

1. Analisi dei requisiti

1.1. Raccolta dei requisiti

- Il sistema deve permettere all'Utente di registrarsi fornendo le proprie Credenziali (Email e Password). La Registrazione è permessa esclusivamente a Utenti aventi un'Email istituzionale universitaria (es. @studio.unibo.it).
- Ogni Utente può gestire il proprio Profilo personale, contenente nome, cognome, Corso di studi, una breve Biografia e opzionalmente un'immagine.
- Qualsiasi Utente autenticato può creare un nuovo Evento.
- L'Organizzatore, nella creazione di un Evento, deve definire:
 - **Titolo:** il titolo dell'Evento deve essere chiaro e descrittivo, nel caso anche della materia che si intende studiare nell'incontro;
 - **Data e orario:** l'Evento deve avere una data e un orario di inizio e fine ben definiti;
 - **Indirizzo:** indicazione fisica specifica (via, nome della struttura o aula) in cui si terrà l'incontro;
 - **Tipo di Luogo:** scegliere tra Luogo "Pubblico" o "Privato";
 - **Numero di posti:** può essere a posti "Limitati" o "Illimitati".
- Il sistema deve offrire una bacheca globale dove l'Utente può sfogliare gli Eventi di studio creati da altri Utenti. Gli Eventi scompaiono dalla bacheca una volta pieni.
- L'Utente deve poter filtrare gli Eventi in bacheca in base alla Materia (Specifico per Esame o Generale), Tipo di Luogo (Pubblico o Privato), numero di posti (Limitato o Illimitato) e data. Inoltre, l'utente deve poter inserire un indirizzo personalizzato a sua scelta e un raggio massimo espresso in chilometri per limitare la ricerca.
- Gli utenti scelgono l'evento in base al titolo dato dall'organizzatore.
- Un Utente può prenotarsi per un Evento esistente se sono presenti posti disponibili.
- Un Partecipante può annullare la propria Prenotazione fino a un'ora prima, liberando il posto per altri Utenti.
- L'Organizzatore può modificare i dettagli dell'Evento o cancellarlo fino all'ora precedente.
- L'Organizzatore può rimuovere gli Utenti Partecipanti a sua discrezione.
- Alla pubblicazione di un Evento, il sistema deve generare automaticamente una Chat di gruppo privata, accessibile esclusivamente all'Organizzatore e ai Partecipanti di quello specifico Evento.
- Al termine dell'Evento i Partecipanti possono lasciare dei Commenti positivi o negativi.
- L'interfaccia Utente deve essere intuitiva e ottimizzata per l'uso in mobilità (mobile-friendly), facilitando la navigazione rapida tra bacheca e Chat.
- Le informazioni del Profilo devono essere facilmente consultabili prima di unirsi a un gruppo per favorire un clima di fiducia tra colleghi.
- I messaggi scambiati nella Chat devono essere visibili solo ai membri del gruppo di studio corrente. I dati personali devono essere trattati in conformità con le normative vigenti (GDPR).

1.2. Requisiti di sistema

1.2.1. Requisiti funzionali

ID	Requisito	Tipo
R01F	La Registrazione è ammessa solo tramite Email istituzionale	Funzionale
R02F	Una persona può registrarsi, creando un Profilo Utente	Funzionale
R03F	Un Profilo Utente è accessibile tramite Mail istituzionale e Password	Funzionale
R04F	Un Profilo Utente ha le seguenti informazioni: nome, cognome, Corso di studi, Bio, immagine opzionale e Commenti ricevuti	Funzionale
R05F	Le caratteristiche di un Evento sono: titolo, Tipo di Luogo, data, orario, Indirizzo, posti disponibili	Funzionale
R06F	La Materia di un Evento può essere specificata o generica	Funzionale
R07F	Il tipo di accesso all'Evento è aperto a tutti, chiunque può scegliere di aggiungersi, posti permettendo	Funzionale
R08F	Il numero di posti disponibili può essere limitato o illimitato	Funzionale
R09F	Gli Eventi hanno due fasi: programmato e concluso	Funzionale
R10F	Gli Utenti visualizzano una bacheca globale di Eventi programmati con almeno un posto disponibile	Funzionale
R11F	Gli Utenti possono filtrare gli Eventi dalla bacheca per: Tipo di Luogo, data, numero posti, e distanza chilometrica a partire da un indirizzo specifico	Funzionale
R12F	Un Utente può prenotarsi a un Evento, diventandone Partecipante	Funzionale
R13F	Un Utente prima di prenotarsi a un Evento può visualizzare il Profilo Utente dei Partecipanti	Funzionale
R14F	Un Partecipante può annullare la Prenotazione all'Evento fino a un'ora prima dell'inizio	Funzionale
R15F	L'Organizzatore può modificare i dettagli dell'Evento o cancellarlo fino all'ora precedente	Funzionale
R16F	L'Organizzatore può rimuovere i Partecipanti all'Evento a sua discrezione	Funzionale
R17F	Per ogni Evento è prevista una Chat di gruppo accessibile solo ai Partecipanti	Funzionale

ID	Requisito	Tipo
R18F	Al termine dell'Evento, ciascun Partecipante può lasciare un Commento a ognuno degli altri presenti	Funzionale
R19F	Un Utente può richiedere l'eliminazione del proprio Profilo	Funzionale

1.2.2. Requisiti non funzionali

ID	Requisito	Tipo
R01N	La piattaforma deve essere accessibile online	Non funzionale
R02N	Le Credenziali di un Utente sono univoche	Non funzionale
R03N	L'interfaccia della Bacheca deve essere essenziale ed intuitiva	Non funzionale
R04N	Il fuso orario usato dall'applicazione è GMT+1	Non funzionale
R05N	Il formato di data e orario è GG/MM/AAAA e HH:MM 24h	Non funzionale
R06N	L'applicazione deve rispettare le norme definite nel <u>GDPR</u>	Non funzionale
R07N	La Bacheca globale deve essere costantemente aggiornata	Non funzionale
R08N	La Password scelta dall'Utente deve essere di almeno 8 caratteri e contenere almeno un numero e un carattere speciale (#@?!_-)	Non funzionale
R09N	La ricezione ed invio dei Messaggi deve essere istantanea	Non funzionale
R10N	L'applicazione deve poter operare in totale autonomia senza l'intervento attivo di amministratori umani, garantendo così una maggiore tutela della privacy degli utenti	Non funzionale

1.3. Analisi del dominio

1.3.1. Vocabolario

Voce	Definizione	Sinonimi
Profilo	Insieme delle informazioni personali e accademiche che rappresentano l'identità di un Utente all'interno della piattaforma	Account
Registrazione	Creazione di un Profilo da parte di uno studente	
Utente	Studente che ha eseguito la Registrazione	
Autenticazione	Processo tramite il quale uno studente si identifica e si associa al proprio Profilo	Accesso, Login
Informazioni del Profilo	Dati inseriti dall'Utente, quali nome, cognome, Email, Corso di studi, Biografia ed eventualmente un'immagine	
Password	Sequenza di caratteri segreta	
Credenziali	Coppia di Email e Password	
Evento	Sessione di studio programmata da un Utente, caratterizzata da un titolo, un Indirizzo, un Tipo di Luogo, una data e un orario	Sessione di studio
Titolo	Nome descrittivo dell'Evento	Nome
Luogo Pubblico	Luogo fisico accessibile a tutti in cui si svolge un Evento (es. biblioteca, aula studio)	
Luogo Privato	Luogo fisico privato in cui si svolge un Evento	
Indirizzo	Indicazione fisica specifica (via, nome della struttura o aula) in cui si terrà l'incontro	Posizione, luogo
Evento a posti Limitati	Tipologia di Evento che prevede un numero massimo di partecipanti	
Evento a posti Illimitati	Tipologia di Evento che non prevede un numero massimo di partecipanti	
Bacheca	Sezione in cui è possibile visualizzare l'elenco degli Eventi disponibili	
Organizzatore	Utente che crea, gestisce, coordina ed è Partecipante a un Evento	
Partecipante	Utente che sceglie di unirsi a un Evento	Iscritto
Prenotazione	Iscrizione a un Evento	Partecipazione, iscrizione

Voce	Definizione	Sinonimi
Filtro	Funzionalità di ricerca che permette di restringere la visualizzazione degli Eventi in Bacheca secondo criteri specifici	
Commento	Valutazione espressa dai Partecipanti su un altro Partecipante	Recensione
Chat	Canale di comunicazione testuale privato, creato automaticamente per ogni Evento e riservato ai suoi Partecipanti	Chat condivisa, Chat di gruppo
Messaggio	Singola comunicazione testuale inviata da un Partecipante all'interno della Chat di gruppo	

1.4. Analisi dei requisiti

1.4.1. Modello dei casi d'uso

L'analisi dei requisiti e del dominio hanno portato ai seguenti casi d'uso che interagiscono fra di loro come descritto nel seguente schema.

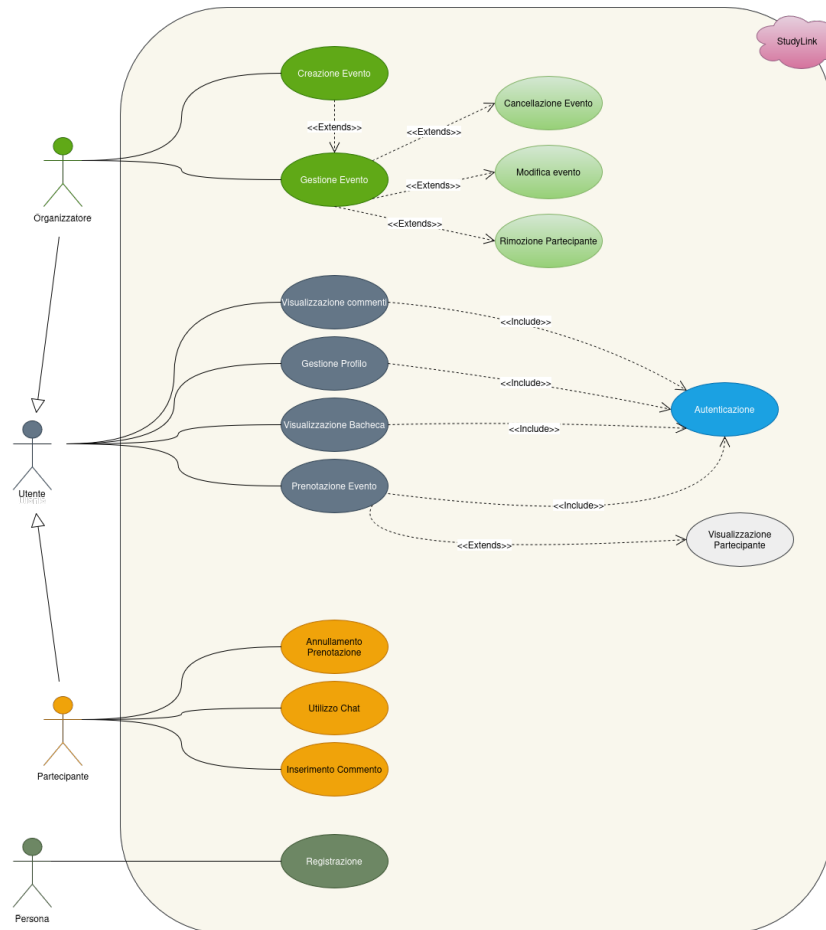


Figura 1.1: Modello dei casi d'uso

1.4.2. Scenari

Di seguito sono presentati tutti gli scenari nel loro dettaglio.

1.4.2.1. Registrazione

Titolo	Registrazione
Descrizione	Procedura di creazione di un account Utente
Attori	Studente
Relazioni	
Precondizioni	
Postcondizioni	Il sistema memorizza il nuovo Utente
Scenario principale	1. Il sistema richiede i dati per la registrazione • Email • Password 2. L'attore inserisce i dati richiesti 3. Il sistema verifica la validità delle credenziali 4. Il sistema crea il nuovo Utente
Scenari alternativi	• L'Email fornita è già in uso 4. Il sistema informa l'Utente che l'Email è già registrata 5. Viene mostrata la schermata di autenticazione • La Password non è sicura 4. Viene mostrato un messaggio di errore 5. Lo studente può inserire una nuova Password • L'Email non è istituzionale 4. Viene mostrato un messaggio di errore 5. Lo studente può inserire una nuova Email
Requisiti non funzionali	R02N, R06N, R08N
Punti aperti	

1.4.2.2. Autenticazione

Titolo	Autenticazione
Descrizione	L'Utente inserisce le proprie Credenziali e si autentica sulla piattaforma
Attori	Utente
Relazioni	• Visualizzazione Commenti • Gestione Profilo • Visualizzazione Bacheca • Prenotazione Evento • Creazione Evento • Gestione Evento • Rimozione Partecipante • Annullamento Prenotazione • Utilizzo Chat • Inserimento Commento
Precondizioni	L'Utente è registrato nel sistema
Postcondizioni	L'Utente è autenticato nel sistema
Scenario principale	1. Il sistema richiede l'inserimento delle Credenziali di accesso 2. L'Utente inserisce le proprie credenziali 3. Il sistema verifica che le credenziali appartengano a un Utente 4. L'Utente è autenticato nel sistema
Scenari alternativi	• Le Credenziali sono errate 4. Viene mostrato un messaggio di errore 5. L'Utente può reinserire le Credenziali
Requisiti non funzionali	R02N
Punti aperti	

1.4.2.3. Gestione profilo

Titolo	Gestione profilo
Descrizione	Inserimento, modifica o eliminazione di dati specifici dell'Utente
Attori	Utente
Relazioni	• Autenticazione
Precondizioni	
Postcondizioni	Le modifiche del profilo sono memorizzate nel sistema
Scenario principale	1. Autenticazione 2. Il sistema mostra le funzionalità disponibili: (a) Inserire e modificare i propri dati come: • Nome • Cognome • Password • Corso di studi • Biografia • Immagine (b) Eliminare l'account dal sistema 3. Il sistema apporta le modifiche effettuate
Scenari alternativi	• La Password non è sicura 3. Viene mostrato un messaggio di errore 4. L'Utente può inserire una nuova Password • La Biografia è troppo lunga 3. Viene mostrato un messaggio di errore 4. L'Utente può inserire una nuova Biografia • L'Immagine è troppo grande 3. Viene mostrato un messaggio di errore 4. L'Utente può inserire una nuova Immagine
Requisiti non funzionali	R06N, R08N
Punti aperti	

1.4.2.4. Creazione Evento

Titolo	Creazione Evento
Descrizione	L'Organizzatore inserisce i dettagli necessari per programmare una nuova sessione di studio
Attori	Utente
Relazioni	Autenticazione
Precondizioni	
Postcondizioni	Il sistema memorizza le modifiche effettuate e l'Utente diventa Organizzatore e Partecipante all'Evento
Scenario principale	- Autenticazione - Il sistema mostra la possibilità di creare un Evento - Il sistema mostra i dati relativi all'Evento: - Titolo dell'Evento - Numero di partecipanti (posti Limitati o Illimitati) - Tipo di Luogo - Indirizzo - Data - Orario - Il sistema verifica la presenza e validità dei dati - Il sistema apporta le modifiche effettuate
Scenari alternativi	- Dei campi sono stati omessi - Viene mostrato un messaggio che indica i campi mancanti - L'Utente può inserire nuovamente i dati mancanti - La data scelta è antecedente alla data corrente - Viene mostrato un messaggio di errore - L'Utente può inserire una nuova data - Il numero di posti inseriti non è valido - Viene mostrato un messaggio di errore - L'Utente può inserire un numero di partecipanti valido - L'Indirizzo o il Tipo di Luogo non sono validi o non esistenti - Viene mostrato un messaggio di errore - L'Utente può inserire un nuovo Indirizzo
Requisiti non funzionali	R04N, R05N, R07N
Punti aperti	

1.4.2.5. Gestione Evento

Titolo	Gestione Evento
Descrizione	L'Organizzatore modifica i dettagli dell'Evento
Attori	Organizzatore
Relazioni	• Autenticazione
Precondizioni	L'Evento deve esistere
Postcondizioni	Le modifiche dell'Evento sono memorizzate nel sistema
Scenario principale	1. Autenticazione 2. Il sistema mostra le funzionalità disponibili: (a) Modificare i dettagli relativi all'Evento creato: • Titolo dell'Evento • Numero di Partecipanti (posti Limitati o Illimitati) • Tipo di Luogo • Indirizzo • Data • Orario (b) Eliminare l'Evento 3. Il sistema apporta le modifiche effettuate
Scenari alternativi	• Il Numero di Partecipanti inserito è minore del numero di Partecipanti attuali 3. Viene mostrato un messaggio di errore 4. L'Organizzatore può inserire un numero di partecipanti valido • Dei campi sono stati omessi 3. Viene mostrato un messaggio che indica i campi mancanti 4. L'Utente può inserire nuovamente i dati mancanti • La data scelta è antecedente alla data corrente 3. Viene mostrato un messaggio di errore 4. L'Utente può inserire una nuova data • Il numero di posti inseriti non è valido 3. Viene mostrato un messaggio di errore 4. L'Utente può inserire un numero di partecipanti valido • L'Indirizzo o il Tipo di Luogo non sono validi o non esistenti 3. Viene mostrato un messaggio di errore 4. L'Utente può inserire un nuovo Indirizzo
Requisiti non funzionali	R04N, R05N, R07N
Punti aperti	

1.4.2.6. Rimozione Partecipante

Titolo	Rimozione Partecipante
Descrizione	L'Organizzatore rimuove un Partecipante dall'Evento
Attori	Organizzatore
Relazioni	• Autenticazione • Gestione Evento
Precondizioni	L'Evento deve esistere
Postcondizioni	Le modifiche dell'Evento sono memorizzate nel sistema
Scenario principale	1. Autenticazione 2. Il sistema mostra all'Organizzatore la possibilità di rimuovere un Partecipante 3. L'Organizzatore seleziona il Partecipante da rimuovere 4. Il sistema aggiorna l'Evento rimuovendo il Partecipante 5. Il sistema apporta le modifiche effettuate
Scenari alternativi	
Requisiti non funzionali	R03N, R07N
Punti aperti	

1.4.2.7. Visualizzazione Bacheca

Titolo	Visualizzazione Bacheca
Descrizione	L'Utente visualizza tutti gli Eventi disponibili con la possibilità di applicare Filtri
Attori	Utente
Relazioni	• Autenticazione
Precondizioni	
Postcondizioni	
Scenario principale	1. Autenticazione 2. Il sistema mostra all'Utente la Bacheca degli Eventi 3. Il sistema mostra la possibilità di applicare i seguenti Filtri: • Data • Indirizzo • Tipo di Luogo • Numero di Partecipanti 4. L'Utente inserisce le condizioni del Filtro 5. Il sistema mostra gli Eventi che rispettano le condizioni applicate
Scenari alternativi	
Requisiti non funzionali	R03N, R07N
Punti aperti	

1.4.2.8. Prenotazione Evento

Titolo	Prenotazione Evento
Descrizione	Permette all'Utente di prenotare un Evento dalla Bacheca
Attori	Utente
Relazioni	• Autenticazione • Visualizzazione Bacheca
Precondizioni	
Postcondizioni	L'Utente diventa Partecipante dell'Evento
Scenario principale	1. Autenticazione 2. L'Utente seleziona un Evento dalla Bacheca 3. Il sistema mostra i dettagli dell'Evento e i suoi Partecipanti 4. L'Utente conferma la sua prenotazione all'Evento 5. Il sistema apporta le modifiche effettuate
Scenari alternativi	
Requisiti non funzionali	R07N
Punti aperti	

1.4.2.9. Visualizzazione Partecipante

Titolo	Visualizzazione Partecipante
Descrizione	Permette agli Utenti di visualizzare i dettagli e i commenti dei Partecipanti ad un Evento
Attori	Utente
Relazioni	• Autenticazione • Visualizzazione Bacheca
Precondizioni	
Postcondizioni	
Scenario principale	1. Autenticazione 2. L'Utente seleziona un Evento dalla Bacheca 3. Il sistema mostra i dettagli dell'Evento e i suoi Partecipanti 4. Il sistema mostra all'Utente la possibilità di visualizzare i dettagli ed i commenti dei Partecipanti
Scenari alternativi	
Requisiti non funzionali	R03N, R06N
Punti aperti	

1.4.2.10. Visualizzazione Commenti

Titolo	Visualizzazione Commenti
Descrizione	Permette all'Utente di visualizzare i commenti presenti sul suo Profilo
Attori	Utente
Relazioni	• Autenticazione
Precondizioni	
Postcondizioni	
Scenario principale	1. Autenticazione 2. Il sistema mostra all'Utente la possibilità di visualizzare i commenti che gli altri Utenti hanno lasciato su di lui
Scenari alternativi	
Requisiti non funzionali	R03N, R06N
Punti aperti	

1.4.2.11. Annullamento Prenotazione

Titolo	Annullamento Prenotazione
Descrizione	L'Utente annulla una prenotazione effettuata in precedenza
Attori	Utente
Relazioni	• Autenticazione
Precondizioni	L'Utente deve essere iscritto ad un Evento
Postcondizioni	La Prenotazione all'Evento viene annullata
Scenario principale	1. Autenticazione 2. Il sistema mostra all'Utente la lista delle Prenotazioni attive 3. L'Utente seleziona la Prenotazione da annullare 4. Il sistema apporta le modifiche effettuate
Scenari alternativi	
Requisiti non funzionali	R07N
Punti aperti	

1.4.2.12. Utilizzo Chat

Titolo	Utilizzo Chat
Descrizione	Permette al Partecipante di comunicare con gli altri Partecipanti ad un Evento tramite messaggi
Attori	Partecipante
Relazioni	• Autenticazione • Prenotazione Evento
Precondizioni	Il Partecipante deve essere iscritto all'Evento
Postcondizioni	La comunicazione avviene con successo tra tutti i Partecipanti
Scenario principale	1. Autenticazione 2. Il Partecipante seleziona l'Evento 3. Il sistema mostra la chat dell'Evento 4. Avviene uno scambio di messaggi tra i Partecipanti
Scenari alternativi	
Requisiti non funzionali	R09N
Punti aperti	

1.4.2.13. Inserimento Commento

Titolo	Inserimento Commento
Descrizione	Alla chiusura di un Evento, un Partecipante inserisce un commento nel profilo di un altro Partecipante
Attori	Partecipante
Relazioni	• Autenticazione • Prenotazione Evento
Precondizioni	Il Partecipante deve essere iscritto all'Evento
Postcondizioni	Il commento viene salvato nel profilo del Partecipante destinatario
Scenario principale	1. Autenticazione 2. Il Partecipante seleziona l'Evento passato 3. Il sistema mostra la lista dei Partecipanti all'Evento 4. Il Partecipante seleziona il Partecipante destinatario 5. Il Partecipante aggiunge un commento per il Partecipante selezionato
Scenari alternativi	
Requisiti non funzionali	R06N
Punti aperti	

1.5. Analisi del Rischio

1.5.1. Tabella valutazione dei beni

Bene	Valore	Esposizione
Sistema Informativo	Alto. L'infrastruttura è fondamentale per garantire l'accesso alla Bacheca, la gestione delle Prenotazioni e le Chat in tempo	Alta. L'indisponibilità di un sistema causerebbe l'interruzione del servizio e l'impossibilità per gli studenti di incontrarsi
Dati Profilo Utente e Credenziali	Alto. Contiene dati strettamente personali tutelati dal (GDPR): Credenziali e informazioni fornite dall'Utente	Alta. In caso di furto c'è il rischio di gravi violazioni della privacy
Dati Eventi e Prenotazioni	Molto Alto. Questi dati rivelano l'indirizzo fisico esatto, la data e l'orario in cui un gruppo specifico di studenti si troverà	Molto Alta. L'esposizione non autorizzata di questi dati comporta rischi per la sicurezza degli studenti
Contenuto Chat di Gruppo	Medio. Le chat sono canali privati accessibili esclusivamente dai Partecipanti all'Evento	Media. L'intercettazione dei messaggi comprometterebbe la riservatezza dell'incontro

1.5.2. Tabella minacce e controlli

Minaccia	Probabilità	Controllo	Rischio
Furto Credenziali Utente	Media. La Mail istituzionale è nota, le password devono essere indovinate	Imposizione di requisiti di sicurezza sulle Password e limitazioni di tentativi di accesso	Costo Medio-basso. L'Utente è obbligato a memorizzare una password più difficile
Intercettazione delle comunicazioni	Alta. L'app viene usata in mobilità spesso su reti wi-fi universitarie o pubbliche non sempre sicure	Cifratura delle comunicazioni	Costo basso. Le tecnologie per la cifratura del traffico web sono gratuite e facilmente integrabili

Minaccia	Probabilità	Controllo	Rischio
Denial of Service	Alta. Essendo un servizio esposto su rete pubblica, chiunque può tentare un attacco DoS	Progettazione infrastruttura adeguata	Costo basso. Prevenzione effettuabile attraverso la limitazione di richieste per uno stesso Utente
Accesso non autorizzato alle Chat private e Dati Evento	Media. Un Utente potrebbe tentare di leggere le Chat di Gruppi a cui non è iscritto manipolando le chiamate al Server	Severo controllo degli accessi, verifica che l'Utente richiedente della Chat partecipi effettivamente all'evento	Costo basso. Richiede attenzione nella scrittura del codice, verificare sempre il ruolo dell'Utente
Manomissione del Database	Bassa. Difficile a meno di gravi falle nel codice	Mantenimento di un file di Log inalterabile e separato per le operazioni sensibili	Costo medio. Richiede un'infrastruttura ben progettata e la creazione del componente di Log

1.5.3. Analisi tecnologica della sicurezza

Tecnologia	Vulnerabilità
Autenticazione tramite Credenziali	• Password banali; • Password cracking; • Social Engineering; • Negligenza
Registrazione vincolata	• Creazione di account fake (gli indirizzi universitari sono facilmente reperibili);
Sistema di Chat integrato	• Cross-Site Scripting (XSS); • Iniezione di link malevoli/Phishing
Architettura Client/Server	• Attacco DoS; • Attacco Man in the Middle; • Sniffing della comunicazione

Tecnologia	Vulnerabilità
Cifratura delle Comunicazioni	La sicurezza dipende dal tipo di cifratura: 1. Cifratura simmetrica; - tempo di vita della chiave; - lunghezza della chiave; - memorizzazione della chiave; - Più informazioni cifrate con la stessa chiave, più materiale per l'analisi del testo di un attaccante; 2. Cifratura asimmetrica; - lunghezza della chiave; - memorizzazione della chiave privata;

1.5.4. Security use case e misuse case

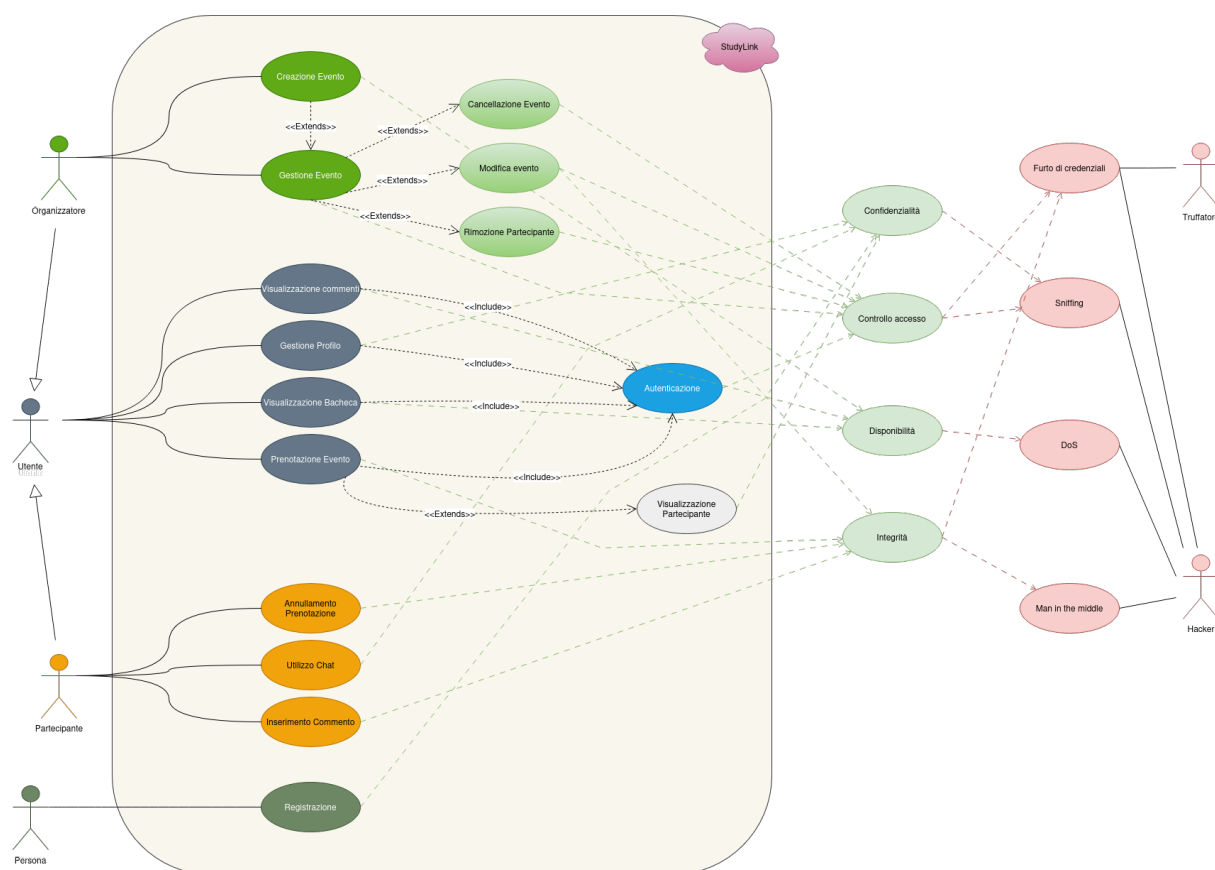


Figura 1.2: Diagramma dei Security use case e misuse case

Titolo	Controllo accesso
Descrizione	Gli accessi al sistema
Misuse case	Furto credenziali, sniffing
Relazioni	Autenticazione

Titolo	Controllo accesso	
Precondizione	L'attaccante conosce l'Email Istituzionale di un Utente	
Postcondizione	L'accesso viene temporaneamente bloccato all'Attaccante	
Scenario Principale	Sistema	Attaccante
		Dopo aver ottenuto una Email Istituzionale tenta di accedere al sistema inserendo delle password errate
	Il sistema controlla le credenziali immesse e blocca temporaneamente l'accesso per quell'Account	
Scenario di attacco avvenuto con successo	Sistema	Attaccante
		Accede al sistema inserendo le credenziali corrette
	Controlla le credenziali immesse e consente l'accesso	
		Naviga nel sistema come fosse l'Utente stesso
	registra i Log delle operazioni dell'Utente	

Titolo	Confidenzialità e Protezione Dati
Descrizione	Riservatezza dei Dati durante il trasferimento tra Client e Server
Misuse case	Sniffing dati, Man in the Middle
Relazioni	Utilizzo Chat, Visualizzazione Bacheca e Partecipante
Precondizione	L'attaccante ha i mezzi per intercettare il traffico di rete
Postcondizione	L'attaccante non ottiene nessun informazione sensibile

Titolo	Confidenzialità e Protezione Dati	
Scenario Principale	Sistema	Attaccante
	Invia messaggi crittografati sulla rete per fornire i suoi servizi	
		Intercetta i pacchetti di rete in transito
		Non riesce a decifrare il contenuto dei pacchetti di rete grazie alla cifratura
	Se il pacchetto è stato modificato viene scartato	
Scenario di attacco avvenuto con successo	Sistema	Attaccante
	Invia messaggi crittografati sulla rete per fornire i suoi servizi	
		Intercetta i pacchetti di rete in transito
		Riesce a decifrare i dati trasmessi e ottiene la possibilità di leggere e alterare le informazioni
	Riconosce i dati intercettati come validi e li registra nel Log	

Titolo	Integrità dei Dati
Descrizione	I dati persistenti devono rimanere validi per tutta la vita dell'Applicazione
Misuse case	Furto Credenziali, Man in the Middle
Relazioni	Gestione Evento, Rimozione Partecipante, Annullamento Prenotazione
Precondizione	L'attaccante: • ha i mezzi per intercettare e modificare i dati scambiati; • predispone delle credenziali di accesso di un Utente

Titolo	Integrità dei Dati	
Postcondizione	Il sistema rileva la non validità dei dati e preserva la sicurezza dell'applicazione	
Scenario Principale	Sistema	Attaccante
		Esegue operazioni come Utente che comportano modifiche indesiderate
	Memorizza le modifiche apportate in un sistema di logging	
	Legge i Log ed identifica le operazioni illecite annullandole	
Scenario di attacco avvenuto con successo	Sistema	Attaccante
		Esegue operazioni come Utente che comportano modifiche indesiderate
	Memorizza le modifiche apportate in un sistema di logging	
	Non si accorge delle modifiche che quindi rimangono nel sistema	

Non viene presentato lo **Scenario di attacco avvenuto con successo** dato che la versione distribuita del DoS è quasi impossibile da distinguere rispetto al normale traffico, perciò non si può contrastare.

1.5.5. Requisiti di sicurezza

Dall'analisi del rischio si possono evincere i seguenti ulteriori requisiti:

1. Creazione di un log inalterabile per tracciare azioni sensibili come:
 - Accessi effettuati e tentativi di accesso falliti di un account;
 - Modifica o cancellazione di Eventi da parte dell'Organizzatore;
2. Protezione da accessi indesiderati:
 - Dopo 5 tentativi di accesso con credenziali errate, il sistema blocca temporaneamente l'accesso all'account;

1.5.6. Aggiornamento requisiti di sistema

ID	Requisito	Tipo
R20F	Il sistema registra le operazioni sensibili e i tentativi di accesso in un file di log.	Funzionale
R21F	I log di sistema possono essere inviati a un sistema esterno per l'analisi e conservazione.	Funzionale
R11N	La comunicazione tra client e server deve essere obbligatoriamente cifrata tramite protocollo HTTPS/TLS.	Non funzionale
R12N	Dopo 5 tentativi di Autenticazione falliti consecutivi, il sistema blocca l'accesso a quell'account per 15 minuti.	Non funzionale

1.5.7. Aggiornamento vocabolario

Voce	Definizione	Sinonimi
Log di sistema	Registro inalterabile in cui vengono tracciate le operazioni sensibili (es. login, modifiche agli Eventi)	Registro, File di Log
Voce di log	Singola riga registrata nel log di sistema, contenente data, ora, utente e azione eseguita	Entry

1.5.8. Casi d'uso aggiornati

A seguito dell'introduzione dei controlli di sicurezza, il caso d'uso **Autenticazione** subisce le seguenti variazioni:

Titolo	Autenticazione
Descrizione	L'Utente inserisce le proprie Credenziali e si autentica sulla piattaforma
Attori	Utente
Relazioni	<i>Invariate rispetto all'analisi precedente</i>
Precondizioni	L'Utente è registrato nel sistema
Postcondizioni	L'Utente è autenticato nel sistema
Scenario principale	1. Il sistema richiede l'inserimento delle Credenziali di accesso 2. L'Utente inserisce le proprie credenziali 3. Il sistema verifica che le credenziali appartengano a un Utente 4. L'Utente è autenticato nel sistema

Titolo	Autenticazione
Scenari alternativi	- Le Credenziali sono errate 4. Viene mostrato un messaggio di errore e l'Utente può reinserire le Credenziali 5. Se il numero di tentativi errati supera i 5 consecutivi, il sistema blocca temporaneamente l'accesso per 15 minuti e registra l'evento nel Log.
Requisiti non funzionali	R02N, R11N, R12N
Punti aperti	

1.5.9. Diagramma dei casi d'uso aggiornato

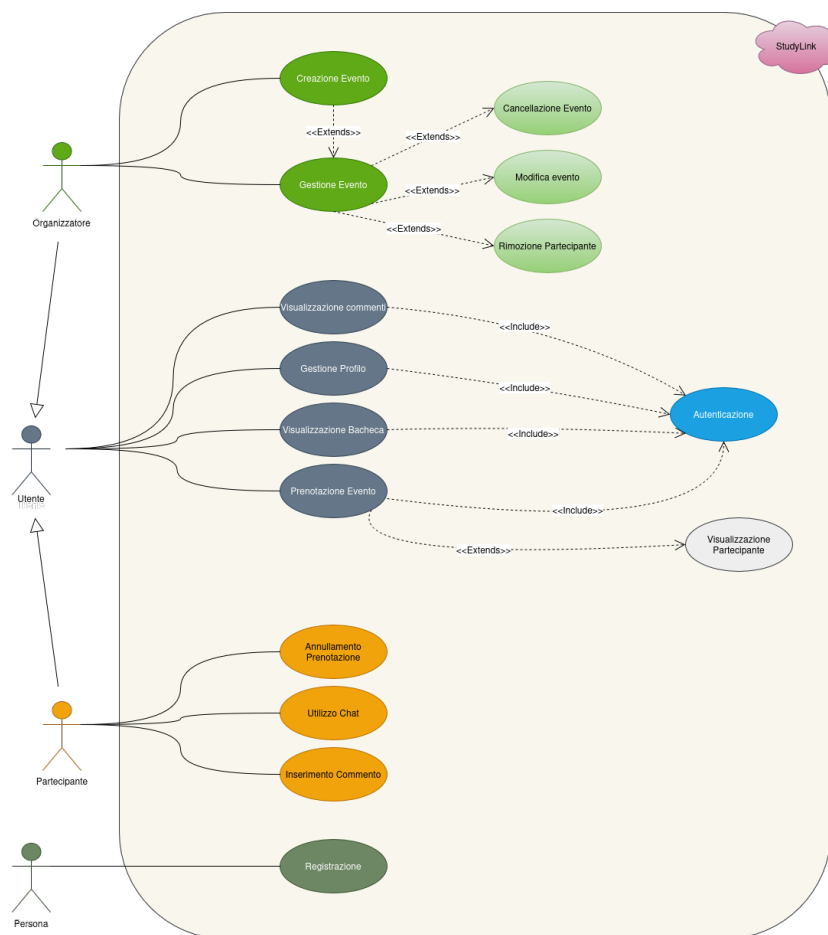


Figura 1.3: Modello dei casi d'uso aggiornato

1.5.10. Integrazione con sistemi esterni

Data la complessità e la presenza sul mercato di software specializzati nell'analisi dei log, si concorda con il cliente di utilizzare un sistema esterno per l'analisi dei log, il cui servizio potrà essere fornito anche da soluzioni di terzi. Il file di log sarà generato da un componente interno al sistema chiamato **GestoreDeiLog** che si occuperà di collezionare i log prodotti e di serializzarli su file.

2. Analisi del problema

2.1. Analisi documento dei requisiti: Analisi delle funzionalità

2.1.1. Tabella delle funzionalità

Funzionalità	Tipo	Grado Complessità	Requisiti Collegati
Registrazione	• Memorizzazione dati; • Interazione esterno;	Semplice	R01F, R02F
Autenticazione	• Gestione dati; • Interazione esterno;	Semplice	R03F, R12N
Creazione evento	• Memorizzazione dati; • Interazione esterno;	Semplice	R05F, R06F, R07F, R08F
Gestione evento	• Gestione e memorizzazione dati; • Interazione esterno;	Complesso	R05F, R06F, R07F, R08F, R09F, R15F, R16F, R17F
Cancellazione evento	• Gestione dati; • Interazione esterno;	Semplice	R15F
Modifica evento	• Gestione dati; • Interazione esterno;	Semplice	R05F, R06F, R08F, R15F
Rimozione partecipante	• Gestione dati; • Interazione esterno;	Semplice	R16F, R17F
Gestione profilo	• Gestione e memorizzazione dati; • Interazione esterno;	Complesso	R04F, R19F
Visualizzazione commenti	• Interazione esterno;	Semplice	R04F, R18F
Visualizzazione bacheca	• Interazione esterno;	Semplice	R10F, R11F
Visualizzazione partecipante	• Interazione esterno;	Semplice	R04F, R13F
Prenotazione evento	• Gestione dati; • Interazione esterno;	Semplice	R07F, R12F
Annullamento prenotazione	• Gestione dati; • Interazione esterno;	Semplice	R14F

Funzionalità	Tipo	Grado Complessità	Requisiti Collegati
Utilizzo chat	- Gestione e memorizzazione dati; - Interazione esterno;	Complesso	R17F
Inserimento commento	- Memorizzazione dati; - Interazione esterno;	Semplice	R04F, R18F
Registrazione Log	Memorizzazione dati;	Semplice	R20F
Esportazione Log	Interazione esterno;	Semplice	R21F

2.1.2. Tabelle informazioni/flusso

Di seguito, saranno riportate le scomposizioni delle informazioni composte per rendere più leggibili le diverse tabelle informazioni/flusso.

2.1.2.1. Informazione: Utente

L'informazione Utente è composta da:

Informazione	Tipo	Livello protezione Privacy	Vincoli
Credenziali	composto	Protezione alta	
Profilo	composto	Protezione alta	

2.1.2.2. Informazione: Credenziali

L'informazione Credenziali è composta da:

Informazione	Tipo	Livello protezione Privacy	Vincoli
Email	semplice	Protezione alta	Email istituzionale universitaria valida (es. @studio.unibo.it)
Password	semplice	Protezione alta	Almeno 8 caratteri, contenente un numero e un carattere speciale

2.1.2.3. Informazione: Profilo

L'informazione Profilo è composta da:

Informazione	Tipo	Livello protezione Privacy	Vincoli
Nome	semplice	Protezione media	Non più di 50 caratteri
Cognome	semplice	Protezione media	Non più di 50 caratteri
Corso di studi	semplice	Protezione media	Non più di 50 caratteri
Biografia	semplice	Protezione media	Non più di 500 caratteri
Immagine	semplice	Protezione media	Percorso file valido, non più di 5 Mb
Commento	composto	Protezione media	

2.1.2.4. Informazione: Evento

L'informazione Evento è composta da:

Informazione	Tipo	Livello protezione Privacy	Vincoli
Titolo	semplice	Protezione media	Non più di 50 caratteri
Materia	semplice	Protezione media	Non più di 50 caratteri
Data	semplice	Protezione media	Nel formato GG/MM/AAAA
Orario	semplice	Protezione media	Nel formato HH:MM
Indirizzo	composto	Protezione alta	
Tipo di Luogo	semplice	Protezione media	Valore scelto tra Pubblico o Privato
Numero posti	semplice	Protezione media	Illimitati o non più di 50 persone

2.1.2.5. Informazione: Indirizzo

L'informazione Indirizzo è composta da:

Informazione	Tipo	Livello protezione Privacy	Vincoli
Via	semplice	Protezione alta	Non più di 50 caratteri
Nome Luogo	semplice	Protezione alta	Non più di 50 caratteri

2.1.2.6. Informazione: Bacheca

L'informazione Bacheca è composta da:

Informazione	Tipo	Livello protezione Privacy /	Vincoli
Evento	composto	Protezione media	

2.1.2.7. Informazione: Prenotazione

L'informazione Prenotazione è composta da:

Informazione	Tipo	Livello protezione Privacy /	Vincoli
Utente	composto	Protezione alta	
Evento	composto	Protezione alta	
Data	semplice	Protezione media	Nel formato GG/MM/AAAA
Orario	semplice	Protezione media	Nel formato HH:MM

2.1.2.8. Informazione: Filtro

L'informazione Filtro è composta da:

Informazione	Tipo	Livello protezione Privacy /	Vincoli
Indirizzo	composto	Protezione media	
Distanza	semplice	Protezione media	Valore in km maggiore di 0
Tipo di Luogo	semplice	Protezione media	Valore scelto tra Pubblico o Privato
Numero posti	semplice	Protezione media	Illimitati o non più di 50 persone
Data di inizio	semplice	Protezione media	Nel formato GG/MM/AAAA
Data di fine	semplice	Protezione media	Nel formato GG/MM/AAAA
Orario di inizio	semplice	Protezione media	Nel formato HH:MM
Orario di fine	semplice	Protezione media	Nel formato HH:MM

2.1.2.9. Informazione: Commento

L'informazione Commento è composta da:

Informazione	Tipo	Livello protezione Privacy /	Vincoli
Utente	composto	Protezione media	
Testo	semplice	Protezione media	Non più di 500 caratteri

Informazione	Tipo	Livello protezione Privacy	Vincoli
Valutazione	semplice	Protezione media	Valore numerico da 1 a 5
Data	semplice	Protezione media	Nel formato GG/MM/AAAA
Orario	semplice	Protezione media	Nel formato HH:MM

2.1.2.10. Informazione: Chat

L'informazione Chat è composta da:

Informazione	Tipo	Livello protezione Privacy	Vincoli
Messaggio	composto	Protezione alta	
Evento	composto	Protezione alta	

2.1.2.11. Informazione: Messaggio

L'informazione Messaggio è composta da:

Informazione	Tipo	Livello protezione Privacy	Vincoli
Utente	composto	Protezione alta	
Testo	semplice	Protezione alta	Non più di 500 caratteri
Data	semplice	Protezione media	Nel formato GG/MM/AAAA
Orario	semplice	Protezione media	Nel formato HH:MM

2.1.2.12. Informazione: Log

L'informazione Log è composta da:

Informazione	Tipo	Livello protezione Privacy	Vincoli
Operazione	semplice	Protezione alta	
Data	semplice	Protezione media	Nel formato GG/MM/AAAA
Orario	semplice	Protezione media	Nel formato HH:MM
Utente	composto	Protezione media	Opzionale

2.1.2.13. Tabella Informazioni / Flusso: Registrazione

Informazione	Tipo	Livello protezione / Privacy	Input / Output	Vincoli
Credenziali	composto	Protezione alta	Input	
Profilo	composto	Protezione media	Output	

2.1.2.14. Tabella Informazioni / Flusso: Autenticazione

Informazione	Tipo	Livello protezione / Privacy	Input / Output	Vincoli
Credenziali	composto	Protezione alta	Input	
Profilo	composto	Protezione media	Output	

2.1.2.15. Tabella Informazioni / Flusso: Creazione evento

Informazione	Tipo	Livello protezione / Privacy	Input / Output	Vincoli
Titolo	semplice	Protezione media	Input	Non più di 50 caratteri
Materia	semplice	Protezione media	Input	Non più di 100 caratteri
Data	semplice	Protezione media	Input	Nel formato GG/M-M/AAAA
Orario	semplice	Protezione media	Input	Nel formato HH:MM
Indirizzo	composto	Protezione alta	Input	
Tipo di Luogo	semplice	Protezione media	Input	Valore scelto tra Pubblico o Privato
Numero posti	semplice	Protezione media	Input	Valore intero maggiore o uguale a 0
Evento	composto	Protezione media	Output	

2.1.2.16. Tabella Informazioni / Flusso: Gestione evento

Informazione	Tipo	Livello protezione / Privacy	Input / Output	Vincoli
Evento	composto	Protezione media	Input	
Evento	composto	Protezione media	Output	

2.1.2.17. Tabella Informazioni / Flusso: Cancellazione evento

Informazione	Tipo	Livello protezione / Privacy	Input / Output	Vincoli
Evento	composto	Protezione media	Input	

2.1.2.18. Tabella Informazioni / Flusso: Modifica evento

Informazione	Tipo	Livello protezione / Privacy	Input / Output	Vincoli
Evento	composto	Protezione media	Input	
Evento	composto	Protezione media	Output	

2.1.2.19. Tabella Informazioni / Flusso: Rimozione partecipante

Informazione	Tipo	Livello protezione / Privacy	Input / Output	Vincoli
Profilo	composto	Protezione alta	Input	
Evento	composto	Protezione media	Input/Output	

2.1.2.20. Tabella Informazioni / Flusso: Gestione profilo

Informazione	Tipo	Livello protezione / Privacy	Input / Output	Vincoli
Profilo	composto	Protezione media	Input	

Informazione	Tipo	Livello protezione / Privacy	Input / Output	Vincoli
Profilo	composto	Protezione media	Output	

2.1.2.21. Tabella Informazioni / Flusso: Visualizzazione commenti

Informazione	Tipo	Livello protezione / Privacy	Input / Output	Vincoli
Profilo	composto	Protezione media	Input	
Commento	composto	Protezione media	Output	

2.1.2.22. Tabella Informazioni / Flusso: Visualizzazione bacheca

Informazione	Tipo	Livello protezione / Privacy	Input / Output	Vincoli
Filtro	composto	Protezione media	Input	
Bacheca	composto	Protezione media	Output	

2.1.2.23. Tabella Informazioni / Flusso: Visualizzazione partecipante

Informazione	Tipo	Livello protezione / Privacy	Input / Output	Vincoli
Profilo	composto	Protezione media	Input	
Profilo	composto	Protezione media	Output	

2.1.2.24. Tabella Informazioni / Flusso: Prenotazione evento

Informazione	Tipo	Livello protezione / Privacy	Input / Output	Vincoli
Profilo	composto	Protezione alta	Input	
Evento	composto	Protezione media	Input	

Informazione	Tipo	Livello protezione / Privacy	Input / Output	Vincoli
Prenotazione	composto	Protezione alta	Output	

2.1.2.25. Tabella Informazioni / Flusso: Annullamento prenotazione

Informazione	Tipo	Livello protezione / Privacy	Input / Output	Vincoli
Prenotazione	composto	Protezione alta	Input	
Evento	composto	Protezione media	Output	

2.1.2.26. Tabella Informazioni / Flusso: Utilizzo chat

Informazione	Tipo	Livello protezione / Privacy	Input / Output	Vincoli
Messaggio	composto	Protezione alta	Input	
Chat	composto	Protezione alta	Output	

2.1.2.27. Tabella Informazioni / Flusso: Inserimento commento

Informazione	Tipo	Livello protezione / Privacy	Input / Output	Vincoli
Profilo (Mittente)	composto	Protezione media	Input	
Profilo (Destinatario)	composto	Protezione media	Input	
Testo	semplice	Protezione media	Input	Massimo 500 caratteri
Valutazione	semplice	Protezione media	Input	Valore numerico da 1 a 5
Commento	composto	Protezione media	Output	

2.1.2.28. Tabella Informazioni / Flusso: Registrazione Log

Informazione	Tipo	Livello protezione / Privacy	Input / Output	Vincoli
Log	composto	Protezione alta	Input/Output	

2.1.2.29. Tabella Informazioni / Flusso: Esportazione Log

Informazione	Tipo	Livello protezione / Privacy	Input / Output	Vincoli
Log	composto	Protezione alta	Input	

2.2. Analisi documento dei requisiti: Analisi dei vincoli

2.2.1. Tabella dei vincoli

Vincolo	Categorie	Impatto	Funzionalità
Accessibilità e facilità di utilizzo della piattaforma [R01N, R03N]	Disponibilità, Usabilità	Garantisce continuità del servizio e un'interfaccia intuitiva per l'utente	Registrazione, Autenticazione, Creazione evento, Gestione evento, Cancellazione evento, Modifica evento, Rimozione partecipante, Gestione profilo, Visualizzazione commenti, Visualizzazione bacheca, Visualizzazione partecipante, Prenotazione evento, Annullamento prenotazione, Utilizzo chat, Inserimento commento, Registrazione Log, Esportazione Log
Creazione e sicurezza dell'account [R02N, R08N, R12N]	Sicurezza	Migliora l'integrità dei dati e previene accessi non autorizzati obbligando all'uso di credenziali sicure e univoche	Registrazione, Autenticazione, Gestione profilo
Standardizzazione dei formati temporali e di visualizzazione [R04N, R05N]	Usabilità	Uniformità e chiarezza nella visualizzazione delle informazioni temporali per tutti gli utenti	Registrazione, Autenticazione, Creazione evento, Gestione evento, Cancellazione evento, Modifica evento, Rimozione partecipante, Gestione profilo, Visualizzazione commenti, Visualizzazione bacheca, Visualizzazione partecipante, Prenotazione evento, Annullamento prenotazione, Utilizzo chat, Inserimento commento, Registrazione Log, Esportazione Log

Vincolo	Categorie	Impatto	Funzionalità
Privacy e sicurezza dei dati del sistema [R06N, R10N, R11N]	Sicurezza, Legale	Protegge la riservatezza delle comunicazioni e garantisce il rispetto delle normative vigenti sui dati personali (GDPR)	Registrazione, Autenticazione, Creazione evento, Gestione evento, Cancellazione evento, Modifica evento, Rimozione partecipante, Gestione profilo, Visualizzazione commenti, Visualizzazione bacheca, Visualizzazione partecipante, Prenotazione evento, Annullamento prenotazione, Utilizzo chat, Inserimento commento, Registrazione Log, Esportazione Log
Prestazioni e responsività del sistema [R07N, R09N]	Tempo di risposta	Migliora l'esperienza utente rendendo l'interazione dinamica ed aggiornata in tempo reale	Visualizzazione bacheca, Utilizzo chat

2.3. Analisi documento dei requisiti: Analisi delle interazioni

2.3.1. Tabella delle maschere

Maschera	Informazioni	Funzionalità
View Autenticazione	Email, Password	Autenticazione
View Registrazione	Nome, Cognome, Corso di studi, Email, Password	Registrazione
Home Bacheca	Filtro (Data, Indirizzo, Tipo di Luogo, Numero posti), Elenco degli Eventi	Visualizzazione bacheca
View Dettaglio Evento	Titolo, Materia, Data, Orario, Indirizzo, Tipo di Luogo, Numero posti, Elenco Partecipanti	Visualizzazione bacheca, Prenotazione evento, Annullamento prenotazione
View Creazione Evento	Titolo, Materia, Data, Orario, Indirizzo, Tipo di Luogo, Numero posti	Creazione evento
View Gestione Evento	Dati Evento precompilati, Elenco Partecipanti (con opzione di rimozione)	Gestione evento, Rimozione partecipante, Cancellazione evento, Modifica evento
View Profilo Personale	Nome, Cognome, Corso di studi, Biografia, Immagine, Commenti ricevuti	Gestione profilo, Visualizzazione commenti
View Modifica Profilo	Nome, Cognome, Corso di studi, Biografia, Immagine, Password	Gestione profilo
View Profilo Partecipante	Nome, Cognome, Corso di studi, Biografia, Immagine, Commenti	Visualizzazione partecipante, Visualizzazione commenti
View Chat Evento	Cronologia Messaggi, Input nuovo messaggio	Utilizzo chat
View Inserimento Commento	Profilo del destinatario, Testo del commento, Valutazione	Inserimento commento

2.3.2. Tabella dei sistemi esterni

Il programma non fa utilizzo di nessun sistema esterno nel suo funzionamento interno, l'unico sistema esterno presente è il visualizzatore di log che interagisce attraverso un file generato dall'applicazione per fornire servizi di sicurezza.

Nome sistema	Visualizzatore log
--------------	--------------------

Descrizione	Sistema che si occupa della gestione dei log generati dall'applicazione per la visualizzazione e individuazione di comportamenti indesiderati
Protocollo di interazione	L'applicazione genera tramite il Gestore-Log righe di messaggi su un file che poi il visualizzatore utilizzerà come sorgente per la visualizzazione dei log
Livello di protezione	Alto

2.4. Analisi dei ruoli e responsabilità

2.4.1. Tabella dei ruoli

Ruolo	Responsabilità	Maschere	Riservatezza	Numerosità
Studente	Navigazione iniziale e inserimento dati al fine di creare un Profilo e accedere alla piattaforma	View Registrazione	Basso grado di riservatezza	Elevata (tutti i potenziali interessati)
Utente	Registrazione, login, gestione del proprio profilo e navigazione della Bacheca	View Autenticazione, View Registrazione, Home Bacheca, View Profilo Personale, View Modifica Profilo	Grado medio di riservatezza	Elevata (potenzialmente tutti gli studenti universitari)
Organizzatore	Creazione di nuovi Eventi di studio e gestione dei dettagli (modifica, cancellazione, rimozione partecipanti)	View Creazione Evento, View Gestione Evento	Grado medio di riservatezza	Variabile in base agli Eventi attivi in contemporanea
Partecipante	Prenotazione agli Eventi, partecipazione alle Chat di gruppo e inserimento Commenti	View Dettaglio Evento, View Profilo Partecipante, View Chat Evento, View Inserimento Commento	Alto grado di riservatezza per accedere ai dati dell'Evento e Chat private	Limitata alla capienza dei singoli Eventi di studio

2.4.2. Utente: Tabella Ruolo-Informazioni

Informazione	Tipo di Accesso
Email	Lettura/scrittura
Password	Scrittura
Nome	Lettura/scrittura
Cognome	Lettura/scrittura
Corso di studi	Lettura/scrittura
Biografia	Lettura/scrittura
Immagine	Lettura/scrittura
Commento (ricevuto)	Lettura
Filtro	Scrittura
Bacheca (Eventi)	Lettura

2.4.3. Organizzatore: Tabella Ruolo-Informazioni

Informazione	Tipo di Accesso
Titolo	Lettura/scrittura
Materia	Lettura/scrittura
Data	Lettura/scrittura
Orario	Lettura/scrittura
Indirizzo	Lettura/scrittura
Tipo di Luogo	Lettura/scrittura
Numero posti	Lettura/scrittura
Profilo (Partecipanti)	Lettura
Prenotazione	Lettura/scrittura

2.4.4. Partecipante: Tabella Ruolo-Informazioni

Informazione	Tipo di Accesso
Prenotazione	Lettura/scrittura
Messaggio	Lettura/scrittura
Testo (Commento)	Scrittura
Valutazione	Scrittura
Profilo (Altri partecipanti)	Lettura

2.5. Scomposizione del problema

2.5.1. Tabella scomposizione delle funzionalità

Funzionalità	Scomposizione
Gestione Profilo	Registrazione, Modifica Profilo, Visualizzazione Commenti
Gestione Evento	Creazione Evento, Modifica Evento, Cancellazione Evento, Rimozione Partecipante

2.5.2. Gestione Profilo: Tabella Sotto-Funzionalità

Sotto-funzionalità	Sotto-funzionalità	Legame	Informazioni
Modifica Profilo	Registrazione	Modifica Profilo dipende da Registrazione (non si può modificare se non ci si è registrati)	Profilo
Visualizzazione Commenti	Registrazione	Dipende da Registrazione	Profilo, Commento

2.5.3. Gestione Evento: Tabella Sotto-Funzionalità

Sotto-funzionalità	Sotto-funzionalità	Legame	Informazioni
Modifica Evento	Creazione Evento	Modifica Evento dipende da Creazione Evento	Evento
Cancellazione Evento	Creazione Evento	Cancellazione Evento dipende da Creazione Evento	Evento
Rimozione Partecipante	Creazione Evento	Non si può rimuovere un partecipante se l'Evento non esiste	Evento, Profilo

2.5.4. Utilizzo Chat: Tabella Sotto-Funzionalità

Sotto-funzionalità	Sotto-funzionalità	Legame	Informazioni
Invio Messaggio	Visualizzazione Chat	Non si può inviare un messaggio se non si è aperta la chat dell'Evento	Messaggio, Chat

2.6. Modello del dominio

Nei diagrammi UML delle classi abbiamo scelto la notazione delle cardinalità che si legge nel seguente modo: “Un’istanza della classe [vicina alla molteplicità] è in relazione con [molteplicità] istanze della classe [lontana dalla molteplicità] e viceversa”. Per esempio la relazione tra Utente e Prenotazione si legge nel seguente modo: “Un Utente effettua da 0 a N prenotazioni e una Prenotazione è associata a esattamente 1 utente”.

I metodi get e set sono stati omessi per brevità, ma sono presenti in tutti i modelli.

2.6.1. Modello del dominio: Utente

Il modello riguardante il profilo è presentato nel seguente diagramma.

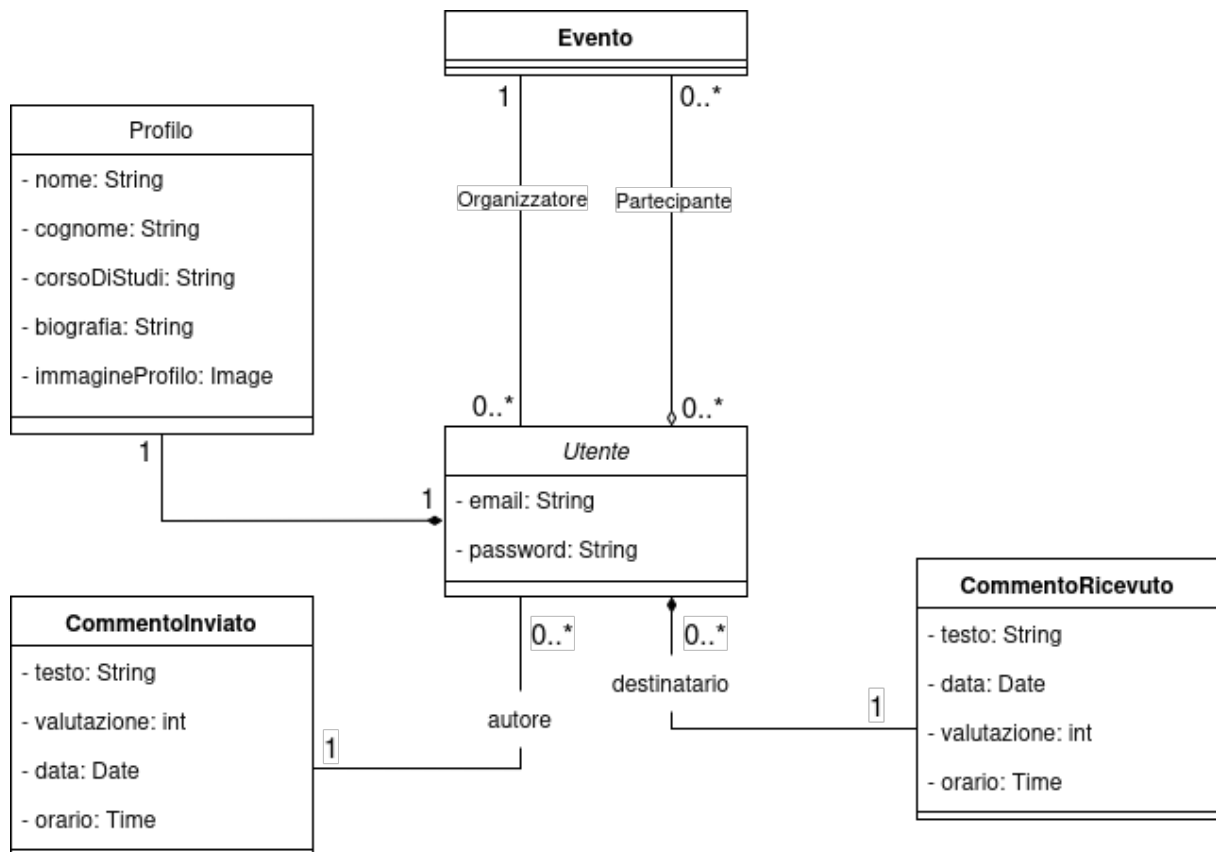


Figura 2.1: Modello del dominio: Utente

2.6.2. Modello del dominio: Evento

Il modello riguardante l’evento è presentato nel seguente diagramma.

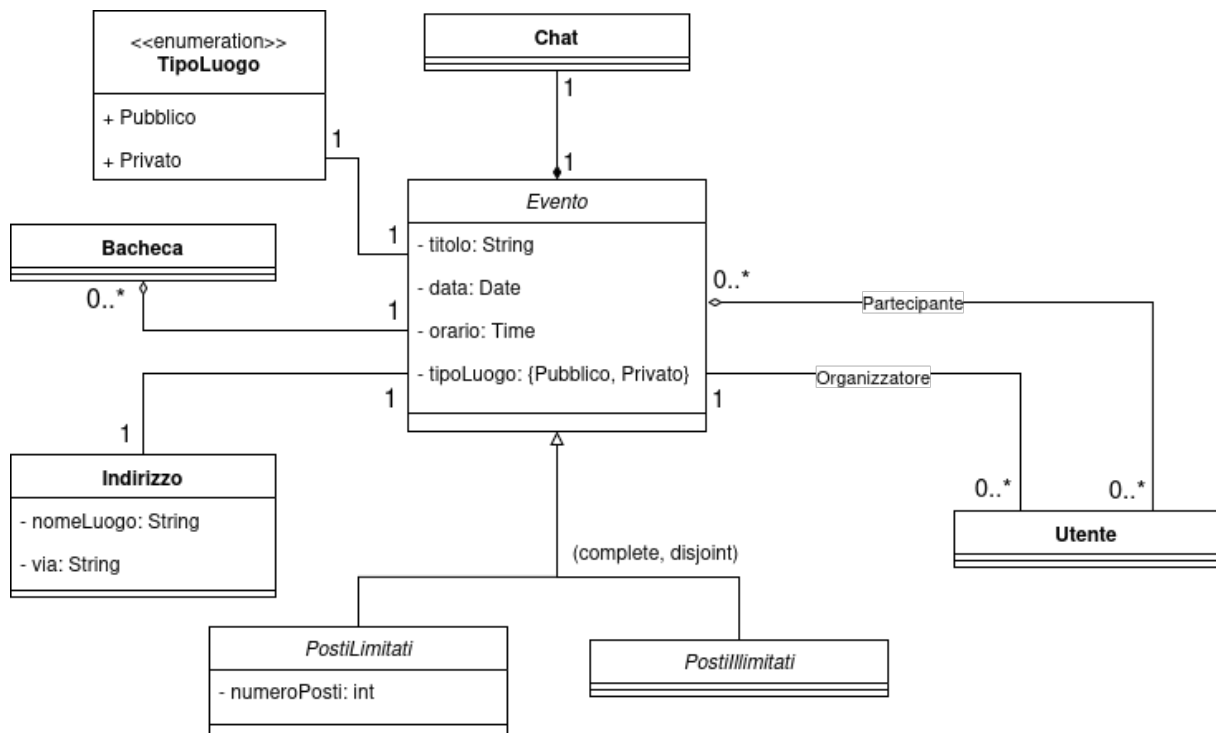


Figura 2.2: Modello del dominio: Evento

2.6.3. Modello del dominio: Filtro

Il modello riguardante il filtro è presentato nel seguente diagramma.

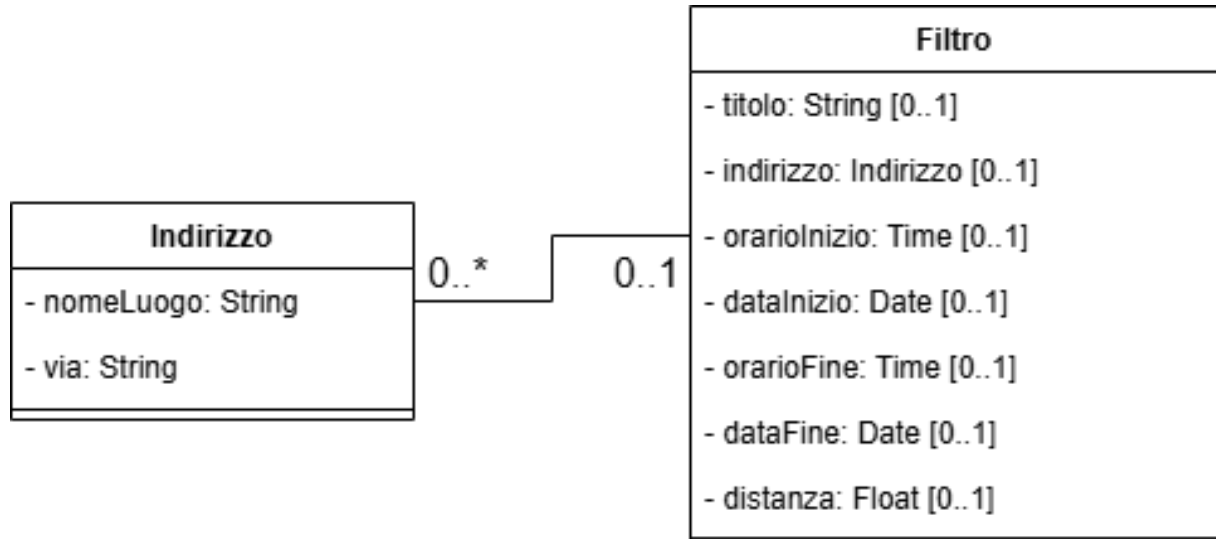


Figura 2.3: Modello del dominio: Filtro

2.6.4. Modello del dominio: Chat

Il modello riguardante la chat è presentato nel seguente diagramma.

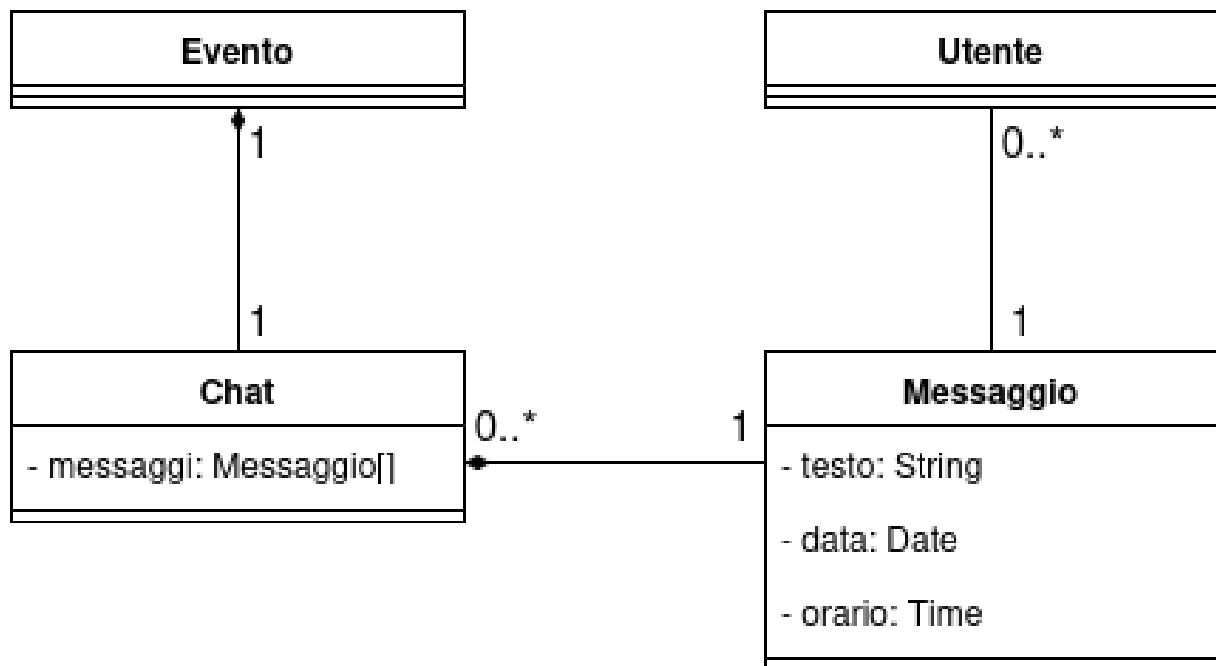


Figura 2.4: Modello del dominio: Chat

2.6.5. Modello del dominio: Log

Il modello riguardante il log è presentato nel seguente diagramma.

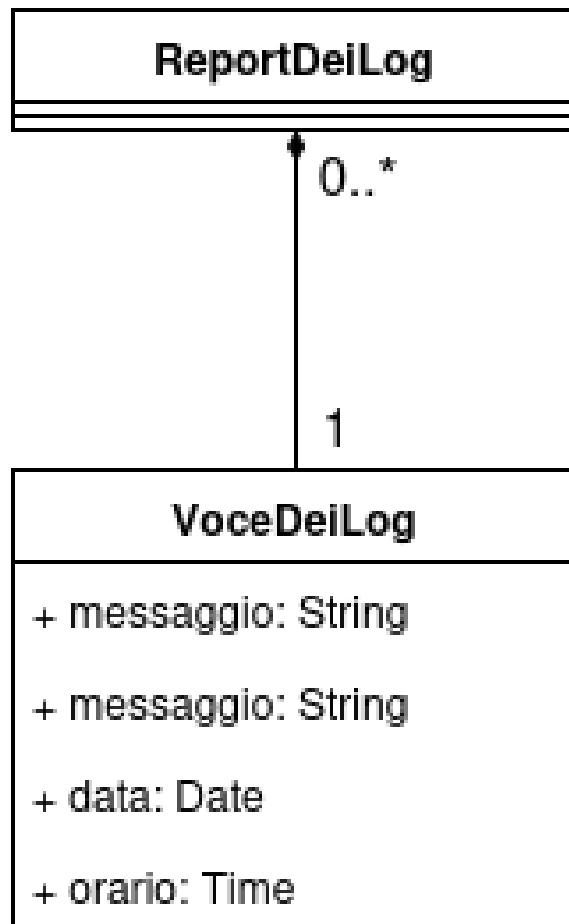


Figura 2.5: Modello del dominio: Log

2.7. Architettura logica

2.7.1. Struttura

2.7.1.1. Diagramma dei package

Di seguito è presentato il diagramma dei package e le diverse dipendenze che intercorrono tra di loro.

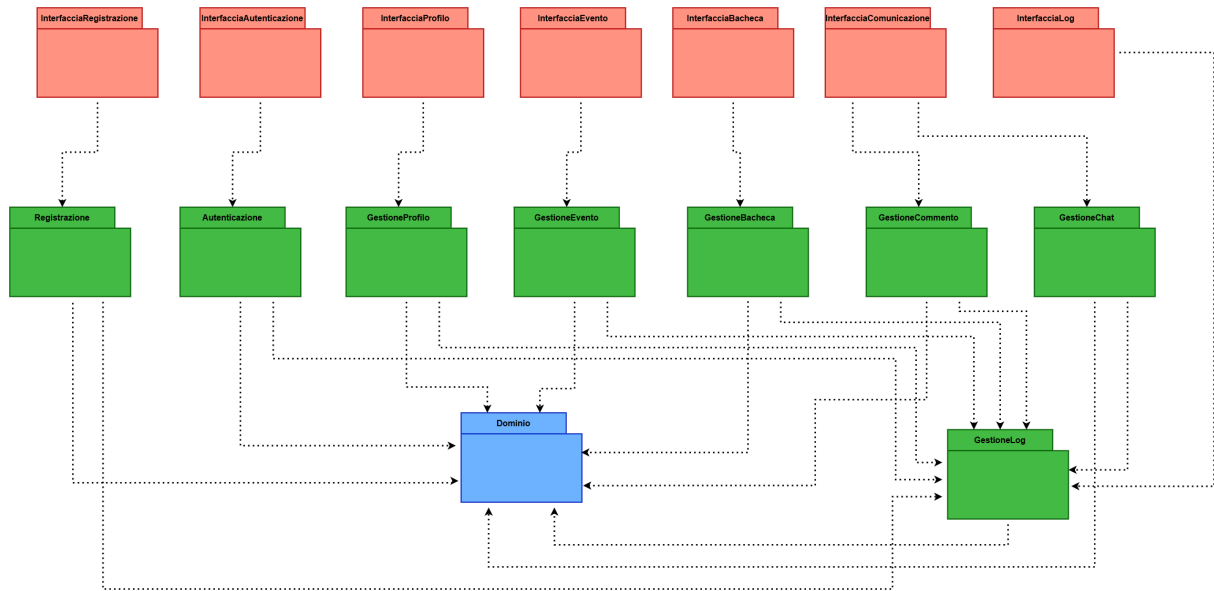


Figura 2.6: Diagramma dei package

2.7.1.2. Diagramma delle classi: Registrazione e Autenticazione

Di seguito è presentato il diagramma delle classi di view e controller per la registrazione e autenticazione degli utenti. Una volta autenticato l'utente è portato alla HomeUtente che è omessa perché connessa semplicemente a tutte le altre home, senza ulteriori funzionalità.

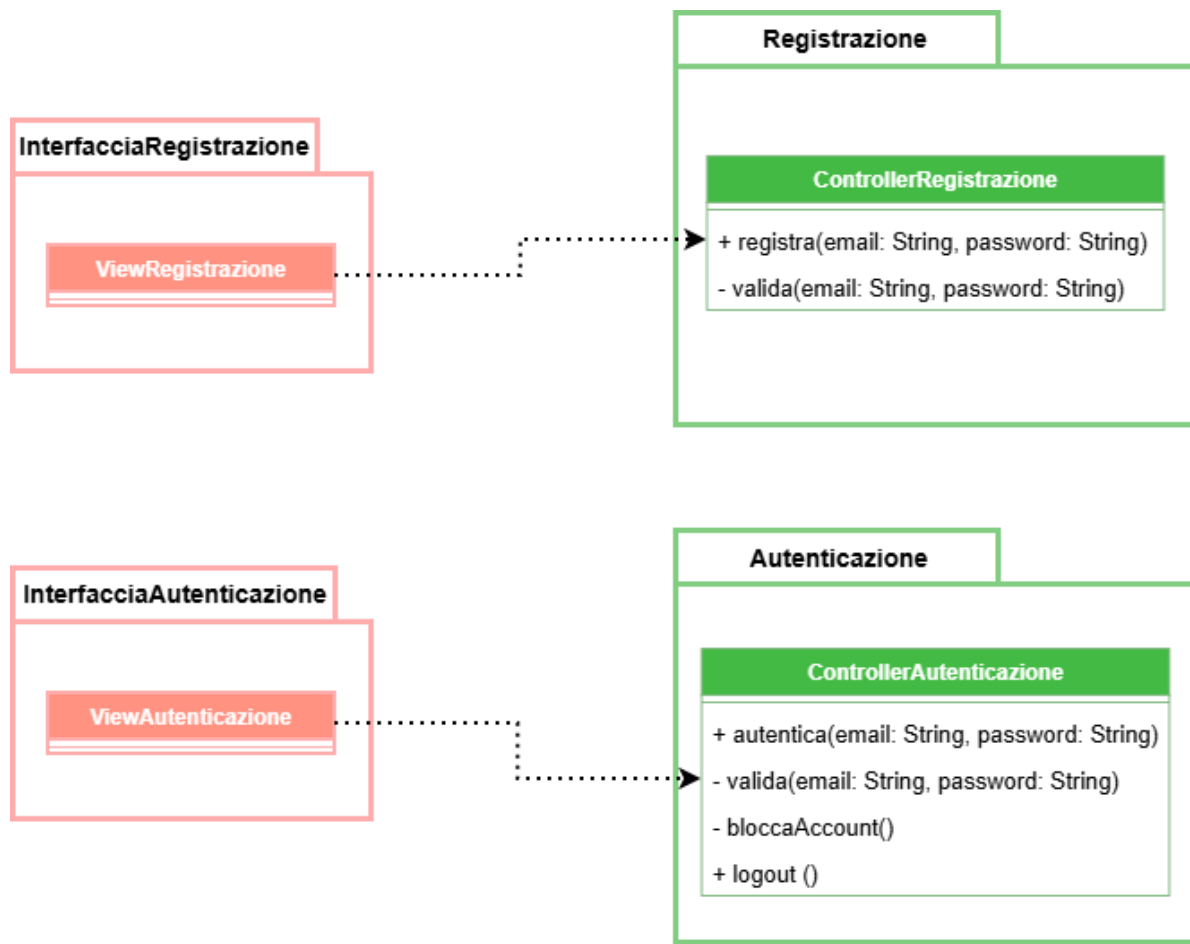


Figura 2.7: Diagramma delle classi: Registrazione e Autenticazione

2.7.1.3. Diagramma delle classi: Profilo

Di seguito è presentato il diagramma delle classi di view e controller per la gestione del profilo.

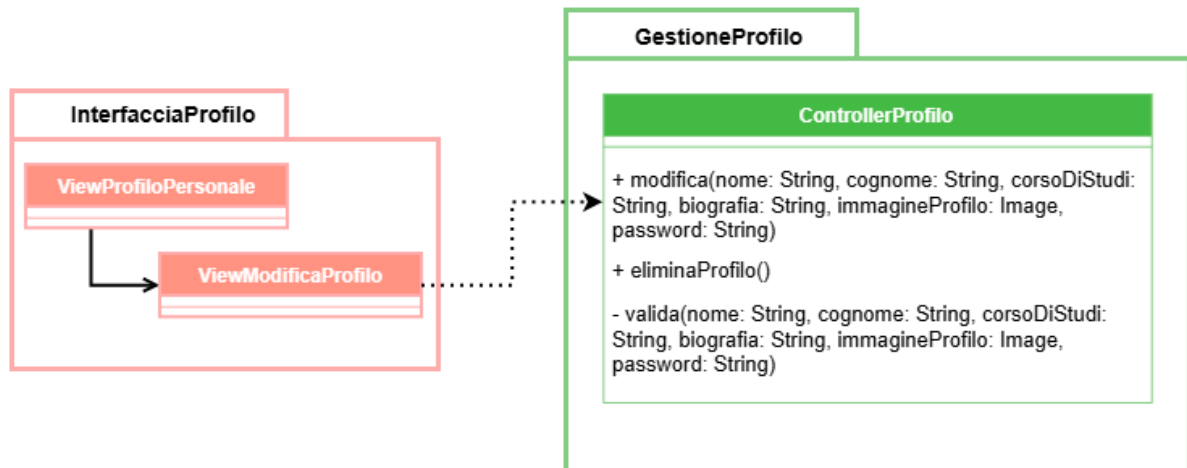


Figura 2.8: Diagramma delle classi: Profilo

2.7.1.4. Diagramma delle classi: Evento

Di seguito è presentato il diagramma delle classi di view e controller per la gestione dell'evento.

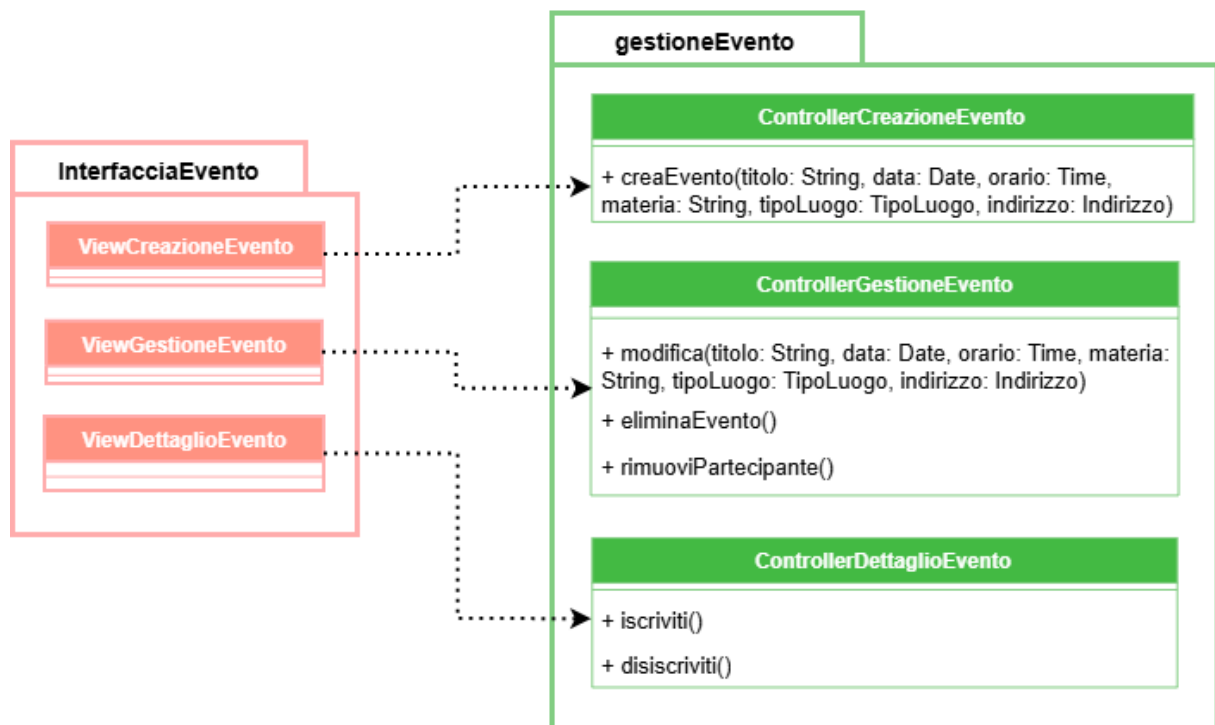


Figura 2.9: Diagramma delle classi: Evento

2.7.1.5. Diagramma delle classi: Bacheca

Di seguito è presentato il diagramma delle classi di view e controller per la gestione della bacheca.

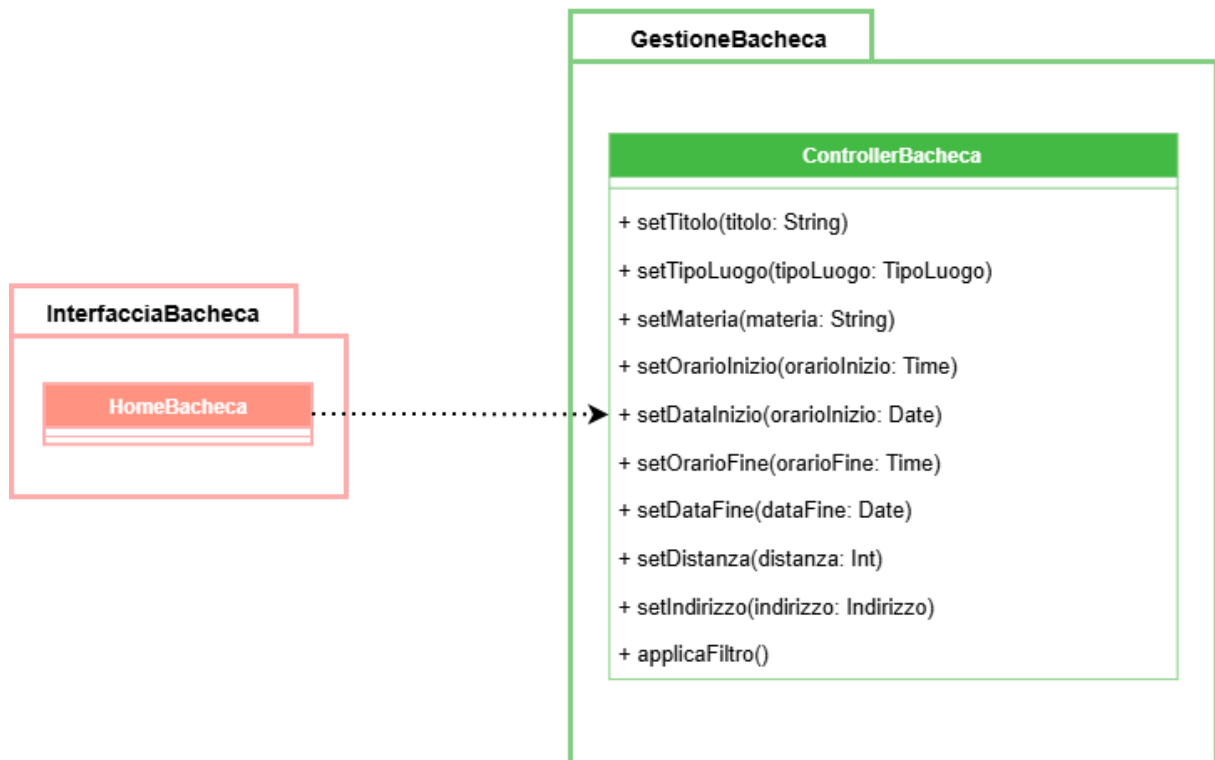


Figura 2.10: Diagramma delle classi: Bacheca

2.7.1.6. Diagramma delle classi: Comunicazione

Di seguito è presentato il diagramma delle classi di view e controller per la gestione della comunicazione.

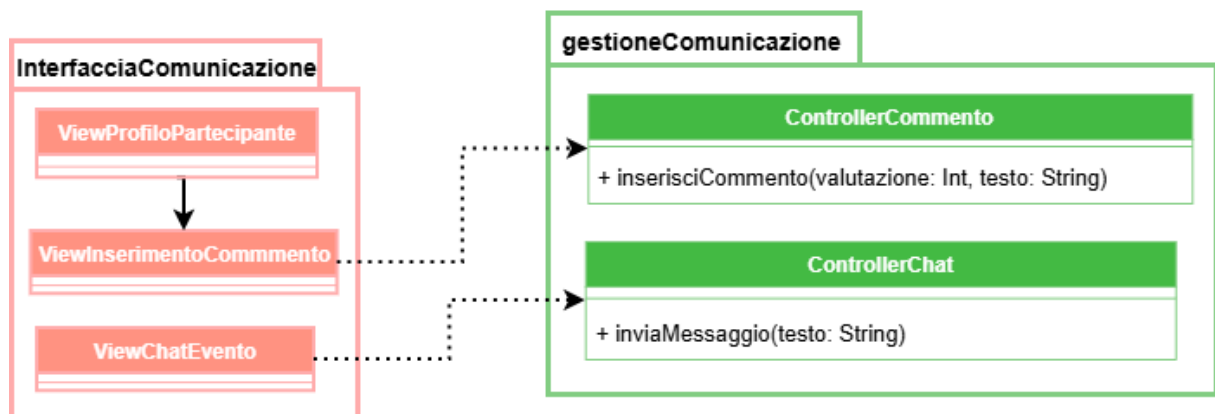


Figura 2.11: Diagramma delle classi: Comunicazione

2.7.2. Interazioni

Di seguito sono presentate le interazioni tra gli attori e il sistema.

2.7.2.1. Interazione: Registrazione

In figura è presentato un diagramma di sequenza che mostra l'interazione tra i componenti per effettuare la registrazione di un nuovo Utente.

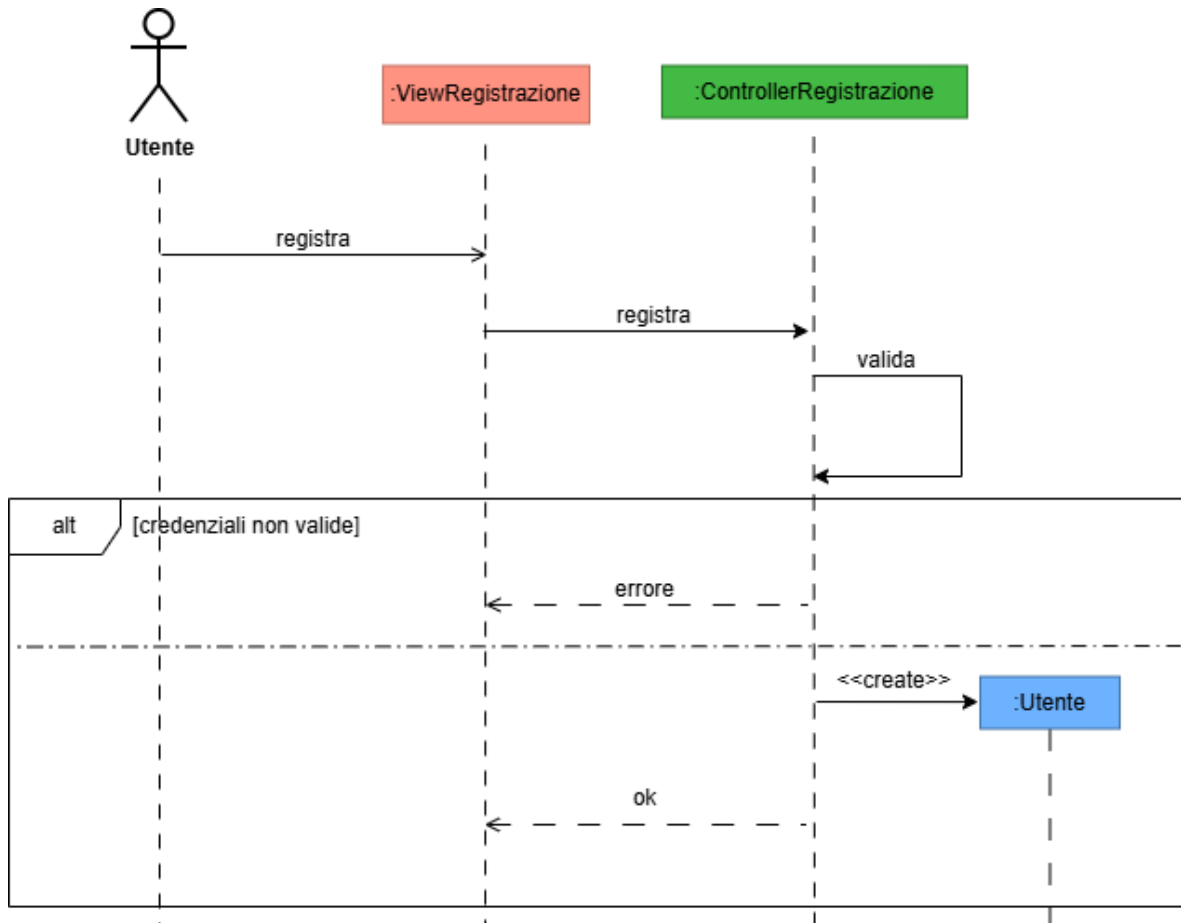


Figura 2.12: Interazione: Registrazione

2.7.2.2. Interazione: Autenticazione

In figura è presentato un diagramma di sequenza che mostra l'interazione tra i componenti per effettuare l'autenticazione di un Utente esistente.

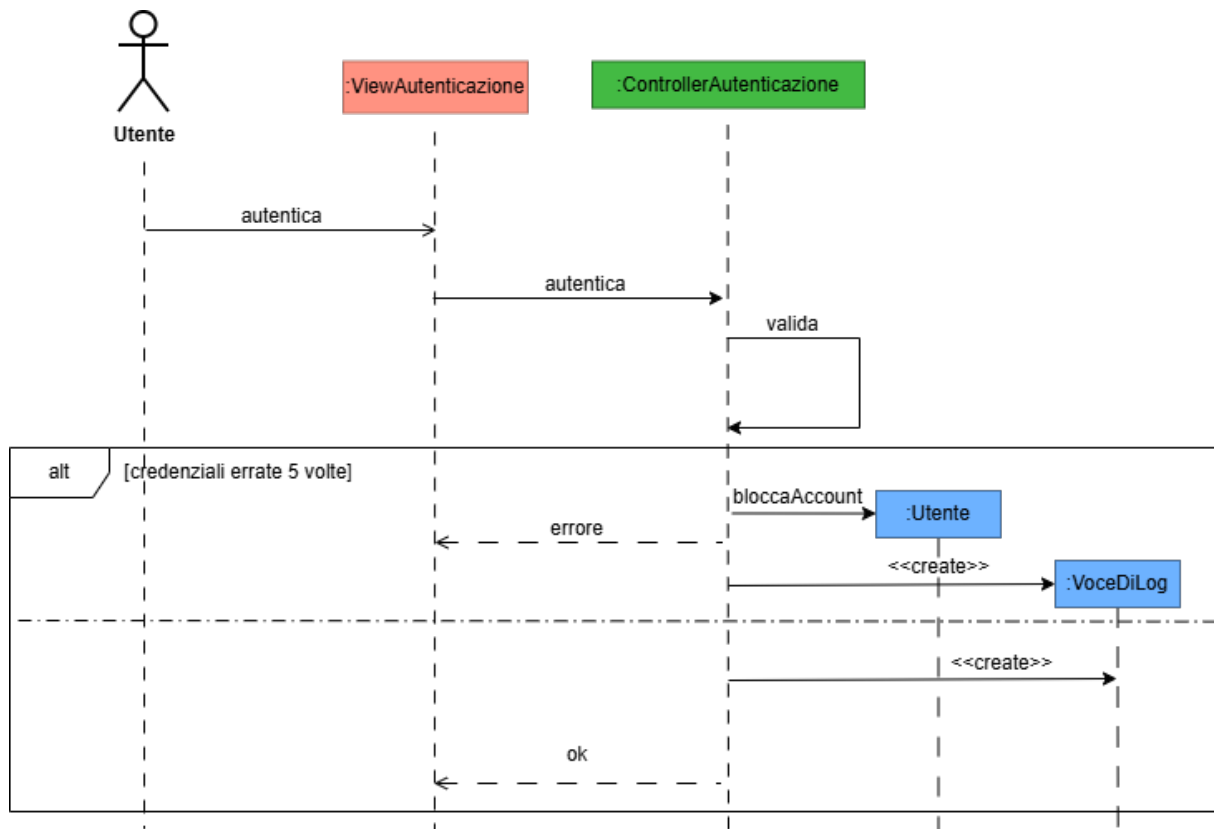


Figura 2.13: Interazione: Autenticazione

2.7.2.3. Interazione: Elimina Utente

In figura è presentato un diagramma di sequenza che mostra l'interazione tra i componenti per effettuare l'eliminazione di un Utente esistente.

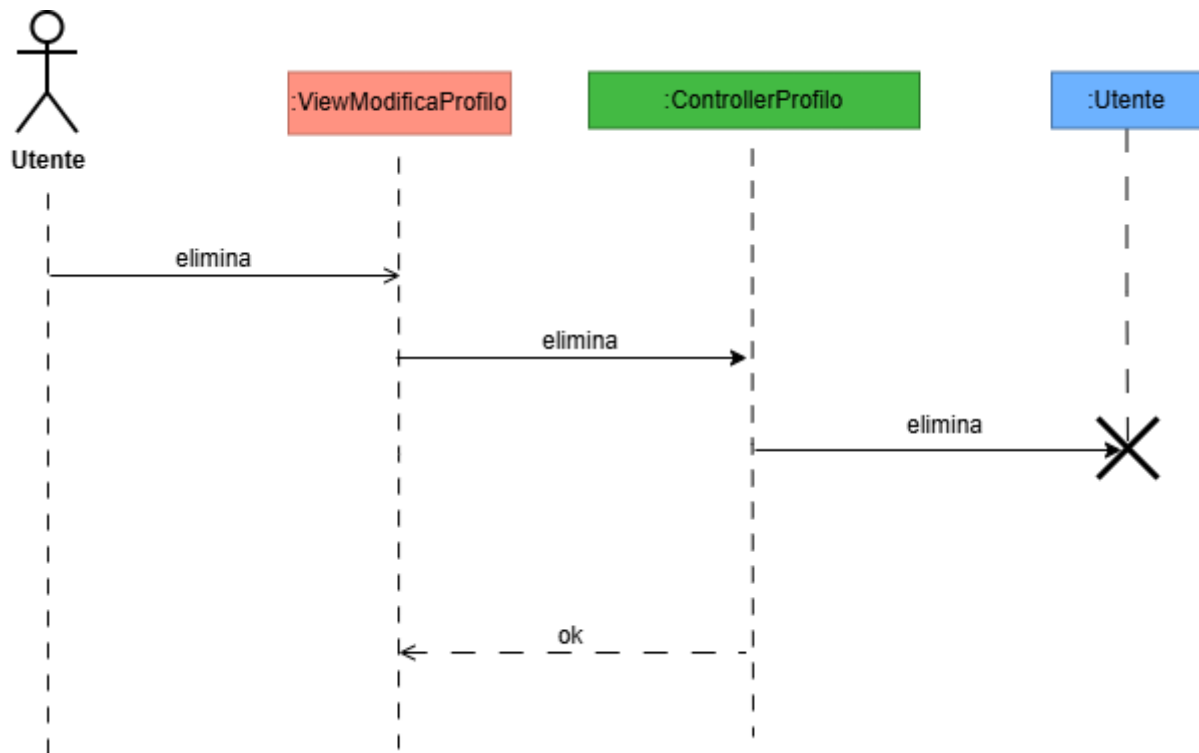


Figura 2.14: Interazione: Elimina Utente

2.7.2.4. Interazione: Modifica Profilo

In figura è presentato un diagramma di sequenza che mostra l'interazione tra i componenti per effettuare la modifica del profilo di un Utente esistente.

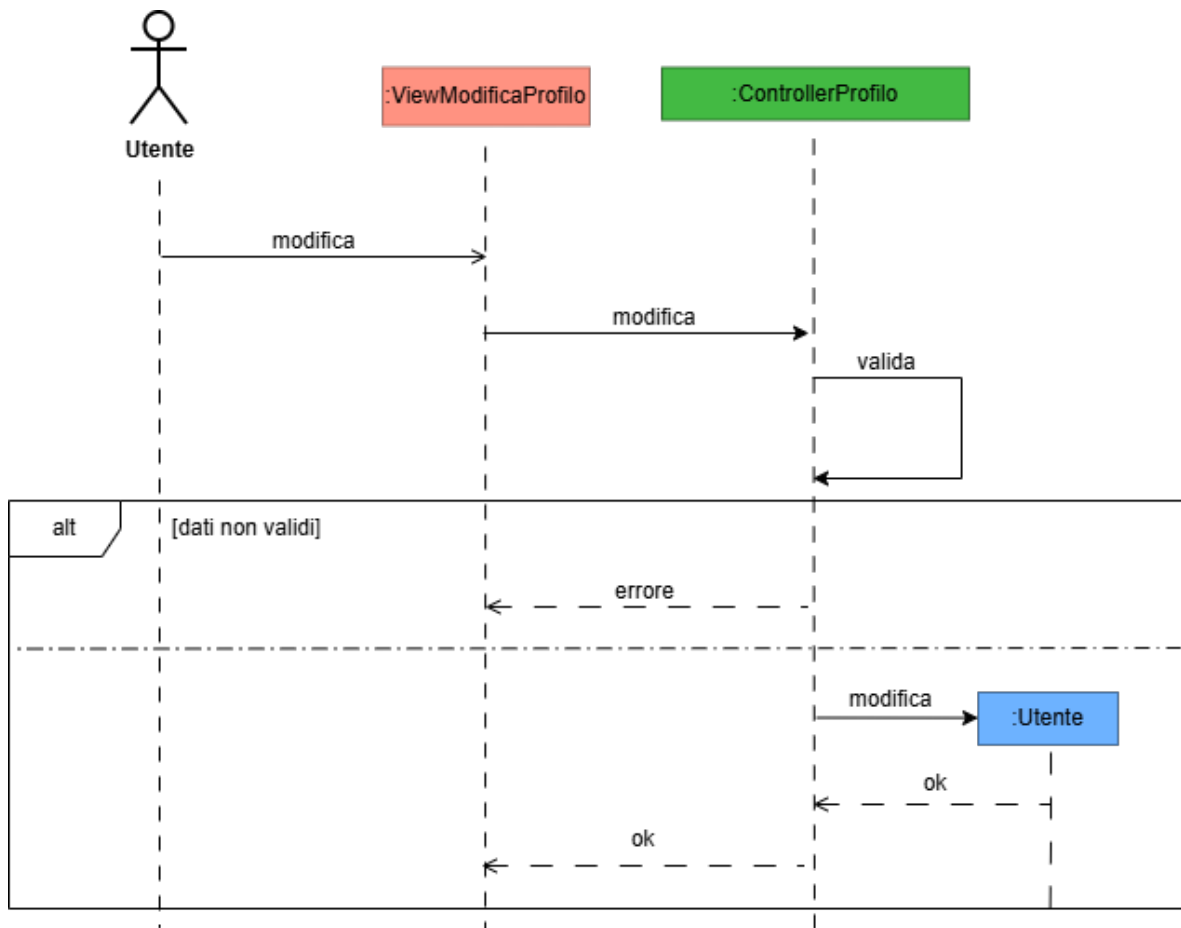


Figura 2.15: Interazione: Modifica Profilo

2.7.2.5. Interazione: Creazione Evento

In figura è presentato un diagramma di sequenza che mostra l'interazione tra i componenti per effettuare la creazione di un nuovo Evento.

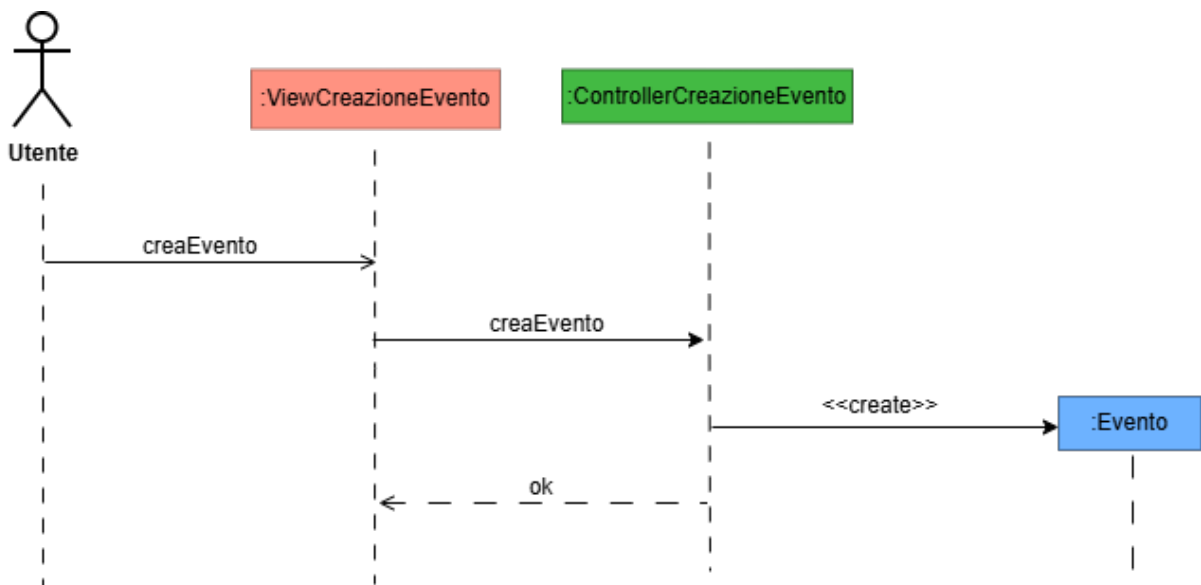


Figura 2.16: Interazione: Creazione Evento

2.7.2.6. Interazione: Gestione Evento

In figura è presentato un diagramma di sequenza che mostra l'interazione tra i componenti per effettuare la gestione di un Evento esistente.

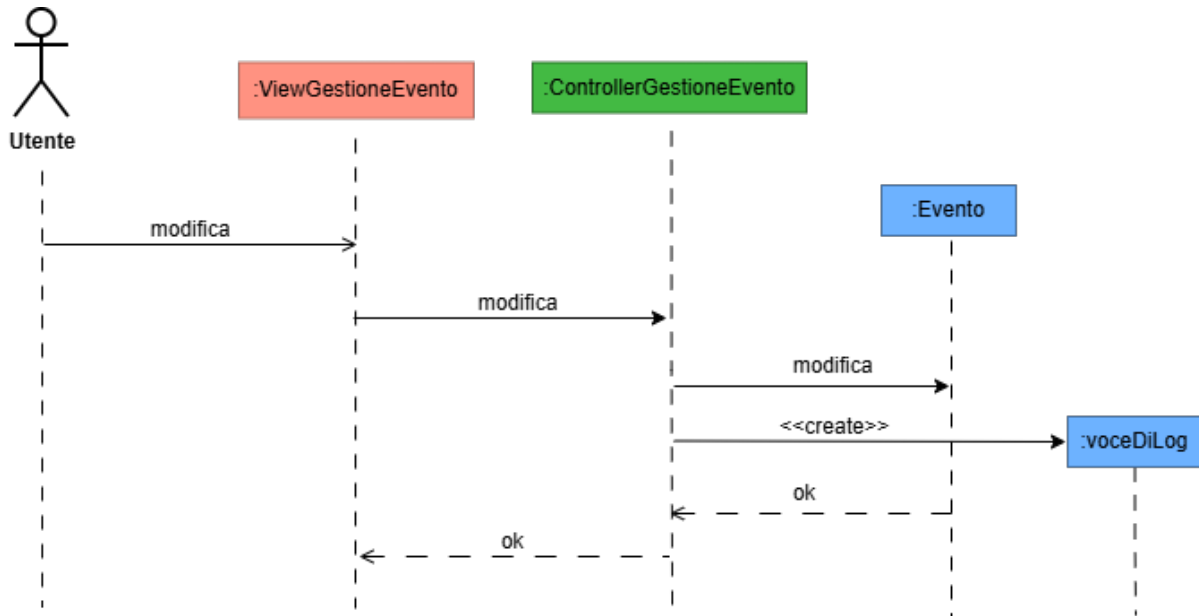


Figura 2.17: Interazione: Gestione Evento

2.7.2.7. Interazione: Chat

In figura è presentato un diagramma di sequenza che mostra l'interazione tra i componenti per effettuare l'utilizzo della Chat.

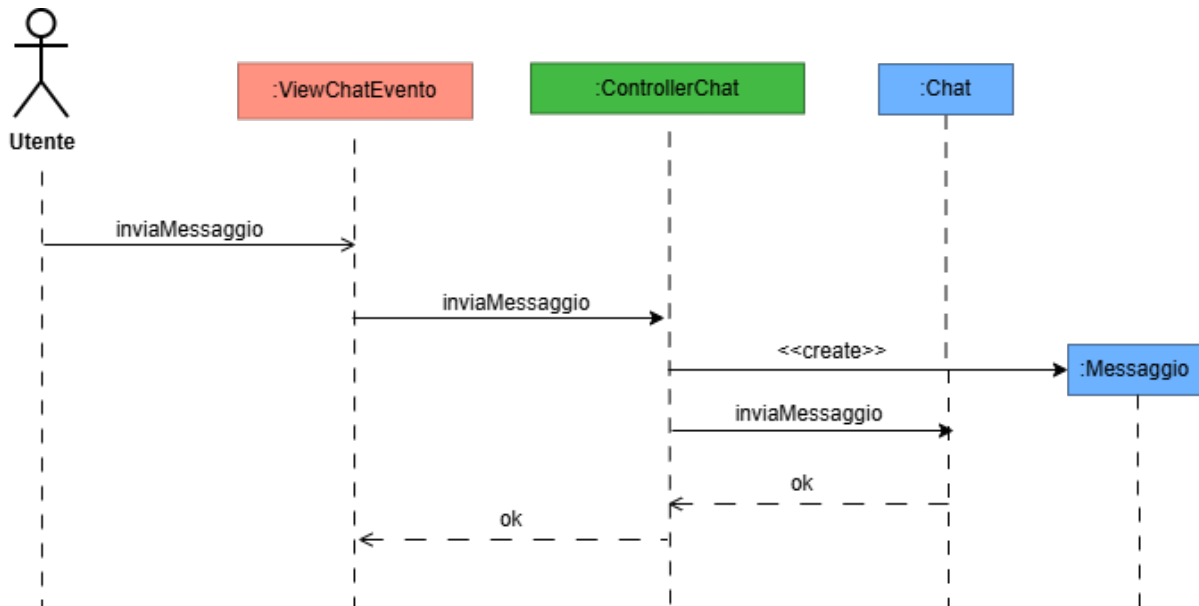


Figura 2.18: Interazione: Chat

2.7.2.8. Interazione: Commento

In figura è presentato un diagramma di sequenza che mostra l'interazione tra i componenti per effettuare la creazione di un Commento.

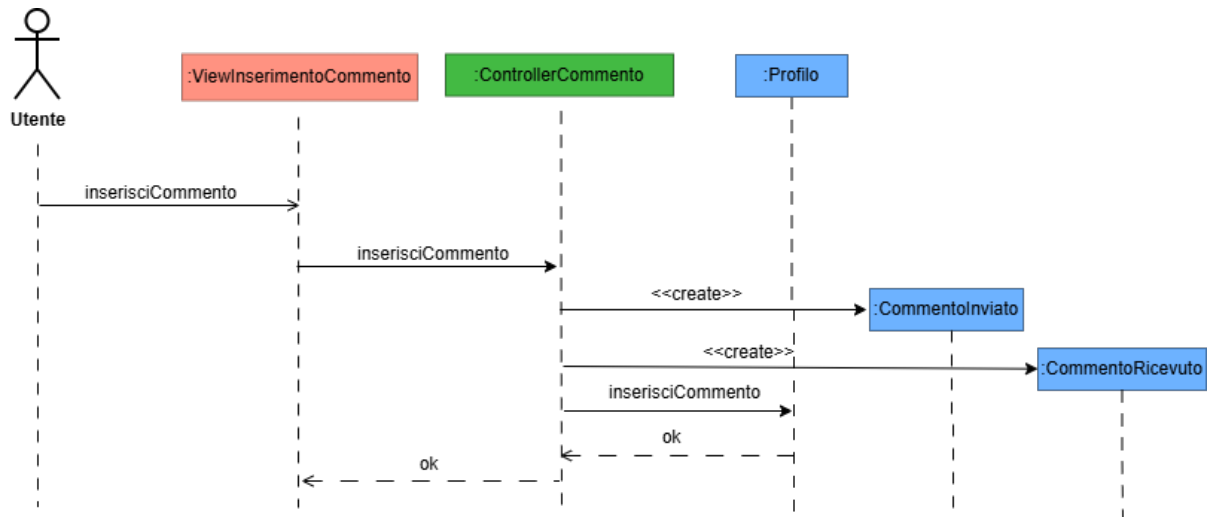


Figura 2.19: Interazione: Commento

2.7.3. Comportamento

Dato che il modello non presenta comportamenti particolarmente elaborati, non vengono mostrati i diagrammi di stato.

2.8. Piano di lavoro

Nome team	Componenti
Team Progettazione	Gattobigio, Lucchi, Porpora
Team Back-end	Gattobigio, Porpora
Team Database	Gattobigio, Lucchi, Porpora
Team Sicurezza	Gattobigio, Lucchi, Porpora
Team Front-end	Lucchi

Package	Progetto	Sviluppo
InterfacciaRegistrazione	Team Progettazione	Team Front-end
InterfacciaAutenticazione	Team Progettazione	Team Front-end
InterfacciaProfilo	Team Progettazione	Team Front-end
InterfacciaEvento	Team Progettazione	Team Front-end
InterfacciaBacheca	Team Progettazione	Team Front-end
InterfacciaComunicazione	Team Progettazione	Team Front-end
InterfacciaLog	Team Progettazione	Team Front-end
Registrazione	Team Progettazione	Team Database
Autenticazione	Team Progettazione	Team Sicurezza
GestioneProfilo	Team Progettazione	Team Back-end
GestioneEvento	Team Progettazione	Team Back-end
GestioneBacheca	Team Progettazione	Team Back-end
GestioneCommento	Team Progettazione	Team Back-end
GestioneChat	Team Progettazione	Team Back-end
Dominio	Team Progettazione	Team Back-end, Team Database, Team Sicurezza

Periodo	Fase
Giugno 2026	Creazione del primo prototipo.
Ottobre 2026	Completamento della prima versione con tutte le funzionalità.
Dicembre 2026	Rilascio ad un numero di utenti limitato.
Febbraio 2027	Rilascio al pubblico.

2.8.1. Caratteristiche primo prototipo (Giugno 2026)

Il primo prototipo del programma non implementerà tutte le funzionalità previste. Le funzionalità che saranno tralasciate sono in particolare quelle relative al fatto che il programma non sarà rilasciato al pubblico:

- La gestione dei log per la sicurezza
- La creazione dei termini e le condizioni di utilizzo per adeguarlo al GDPR
- Il filtro in base alla distanza dalla posizione dell'Evento, la ricerca è solo possibile tramite corrispondenza delle stringhe dell'indirizzo

2.8.2. Sviluppi futuri

Il committente ha richiesto di aggiungere la possibilità di penalizzare gli Utenti a seguito di disiscrizioni a ridosso di Eventi a cui si è iscritti o che si sono organizzati, oppure per comportamenti scorretti. Inoltre, è stata evidenziata la necessità di introdurre un sistema automatizzato di segnalazione e moderazione dei messaggi e dei commenti, capace di sanzionare o rimuovere autonomamente i contenuti inappropriati. Si richiede al Team Progettazione di tenere conto di questi sviluppi futuri.

2.9. Piano del collaudo

Di seguito, al fine di non rendere eccessivamente prolissa la trattazione, sono presentati solo alcuni test unitari del sistema. In particolare, sono stati scelti i test unitari delle classi che si ritengono centrali per il funzionamento del sistema

2.9.1. Collaudo: Evento

```
1 using NUnit.Framework;
2
3 [TestFixture]
4 public class TestEvento
5 {
6     private Evento eventoLimitato, eventoIllimitato;
7     private Profilo organizzatore;
8     private Indirizzo indirizzo;
9
10    [SetUp]
11    public void Setup()
12    {
13        organizzatore = new Profilo(
14            "Mario", "Rossi",
15            "Informatica", "Studente appassionato",
16            null
17        );
18
19        indirizzo = new Indirizzo("Via Zamboni 33", "Biblioteca
20            Universitaria");
21
22        eventoLimitato = new Evento(
23            "Studio Analisi II",
24            "Analisi Matematica II",
25            new DateTime(2026, 7, 15),
26            new Orario("14:00", "17:00"),
27            indirizzo,
28            TipoDiLuogo.Pubblico,
29            5,
30            organizzatore
31        );
32
33        eventoIllimitato = new Evento(
34            "Ripasso Algebra",
35            "Algebra e Geometria",
36            new DateTime(2026, 7, 20),
37            new Orario("10:00", "13:00"),
38            indirizzo,
39            TipoDiLuogo.Privato,
40            0,
41            organizzatore
42        );
43    }
```

```

44 [Test]
45 public void TestCreazioneEvento()
46 {
47     Assert.AreEqual("Studio Analisi II",
48         eventoLimitato.getTitolo());
49     Assert.AreEqual("Analisi Matematica II",
50         eventoLimitato.getMateria());
51     Assert.AreEqual(new DateTime(2026, 7, 15),
52         eventoLimitato.getData());
53     Assert.AreEqual(new Orario("14:00", "17:00"),
54         eventoLimitato.getOrario());
55     Assert.AreEqual(indirizzo,
56         eventoLimitato.getIndirizzo());
57     Assert.AreEqual(TipoDiLuogo.Pubblico,
58         eventoLimitato.getTipoDiLuogo());
59     Assert.AreEqual(5,
60         eventoLimitato.getNumeroPosti());
61     Assert.AreEqual(organizzatore,
62         eventoLimitato.getOrganizzatore());
63 }
64
65 [Test]
66 public void TestEventoIllimitato()
67 {
68     Assert.AreEqual("Ripasso Algebra",
69         eventoIllimitato.getTitolo());
70     Assert.AreEqual(TipoDiLuogo.Privato,
71         eventoIllimitato.getTipoDiLuogo());
72     Assert.AreEqual(0,
73         eventoIllimitato.getNumeroPosti());
74 }
75
76 [Test]
77 public void TestPartecipanti()
78 {
79     Profilo partecipante = new Profilo(
80         "Luca", "Bianchi",
81         "Informatica", "Studio volentieri in gruppo",
82         null
83     );
84     eventoLimitato.aggiungiPartecipante(partecipante);
85
86     CollectionAssert.Contains(
87         eventoLimitato.getPartecipanti(),
88         partecipante
89     );
90     Assert.AreEqual(1,
91         eventoLimitato.getPartecipanti().Count);
92 }
93
94 [Test]

```

```

95     public void TestRimozionePartecipante()
96     {
97         Profilo partecipante = new Profilo(
98             "Luca", "Bianchi",
99             "Informatica", "Studio volentieri in gruppo",
100             null
101         );
102         eventoLimitato.aggiungiPartecipante(partecipante);
103         eventoLimitato.rimuoviPartecipante(partecipante);
104
105         CollectionAssert.DoesNotContain(
106             eventoLimitato.getPartecipanti(),
107             partecipante
108         );
109         Assert.AreEqual(0,
110             eventoLimitato.getPartecipanti().Count);
111     }
112
113     [Test]
114     public void TestPostiDisponibili()
115     {
116         Assert.AreEqual(5,
117             eventoLimitato.getPostiDisponibili());
118
119         Profilo partecipante = new Profilo(
120             "Luca", "Bianchi",
121             "Informatica", "Studio volentieri in gruppo",
122             null
123         );
124         eventoLimitato.aggiungiPartecipante(partecipante);
125
126         Assert.AreEqual(4,
127             eventoLimitato.getPostiDisponibili());
128     }
129
130     [Test]
131     public void TestChatGenerata()
132     {
133         Assert.IsNotNull(eventoLimitato.getChat());
134     }
135 }

```

2.9.2. Collaudo: Profilo

```

1 using NUnit.Framework;
2
3 [TestFixture]
4 public class TestProfilo
5 {
6     private Profilo profiloCompleto, profiloMinimo;
7

```

```

8      [SetUp]
9      public void Setup()
10     {
11         profiloCompleto = new Profilo(
12             "Mario", "Rossi",
13             "Ingegneria Informatica",
14             "Studente del terzo anno",
15             "immagine.png"
16         );
17
18         profiloMinimo = new Profilo(
19             "Luca", "Bianchi",
20             "Matematica",
21             "",
22             null
23         );
24     }
25
26     [Test]
27     public void TestCreazioneProfilo()
28     {
29         Assert.AreEqual("Mario",
30             profiloCompleto.getNome());
31         Assert.AreEqual("Rossi",
32             profiloCompleto.getCognome());
33         Assert.AreEqual("Ingegneria Informatica",
34             profiloCompleto.getCorsoDiStudi());
35         Assert.AreEqual("Studente del terzo anno",
36             profiloCompleto.getBiografia());
37         Assert.AreEqual("immagine.png",
38             profiloCompleto.getImmagine());
39     }
40
41     [Test]
42     public void TestProfiloMinimo()
43     {
44         Assert.AreEqual("Luca",
45             profiloMinimo.getNome());
46         Assert.AreEqual("Bianchi",
47             profiloMinimo.getCognome());
48         Assert.AreEqual("Matematica",
49             profiloMinimo.getCorsoDiStudi());
50         Assert.AreEqual("",
51             profiloMinimo.getBiografia());
52         Assert.IsNull(profiloMinimo.getImmagine());
53     }
54
55     [Test]
56     public void TestSetterProfilo()
57     {
58         profiloCompleto.setNome("Giuseppe");

```

```

59     Assert.AreEqual("Giuseppe",
60         profiloCompleto.getNome());
61
62     profiloCompleto.setCognome("Verdi");
63     Assert.AreEqual("Verdi",
64         profiloCompleto.getCognome());
65
66     profiloCompleto.setCorsoDiStudi("Fisica");
67     Assert.AreEqual("Fisica",
68         profiloCompleto.getCorsoDiStudi());
69
70     profiloCompleto.setBiografia("Nuova biografia");
71     Assert.AreEqual("Nuova biografia",
72         profiloCompleto.getBiografia());
73
74     profiloCompleto.setImmagine("nuova_immagine.png");
75     Assert.AreEqual("nuova_immagine.png",
76         profiloCompleto.getImmagine());
77 }
78
79 [Test]
80 public void TestCommenti()
81 {
82     Commento commento = new Commento(
83         profiloMinimo,
84         "Ottimo compagno di studio",
85         4,
86         new DateTime(2026, 7, 16),
87         new Orario("18:00")
88     );
89     profiloCompleto.aggiungiCommento(commento);
90     CollectionAssert.Contains(
91         profiloCompleto.getCommenti(),
92         commento
93     );
94     Assert.AreEqual(1,
95         profiloCompleto.getCommenti().Count);
96 }
97 }

```

3. Progettazione

3.1. Progettazione architetturale

3.1.1. Requisiti non funzionali

Dall'Analisi del Problema (Tabella Vincoli) sono emersi diversi requisiti non funzionali che impongono dei vincoli al sistema, in particolare l'usabilità (accessibilità e reattività) e la sicurezza (protezione dei dati personali).

Questi due aspetti si trovano parzialmente in contrasto tra loro, poiché misure di sicurezza molto stringenti possono peggiorare l'esperienza d'uso e rallentare i tempi di risposta. Per gestire questo problema e bilanciare i requisiti, sono state prese le seguenti decisioni:

- **Sicurezza:** si è scelto di utilizzare protocolli crittografici standard per la protezione dei canali di comunicazione pubblici e meccanismi robusti per il controllo degli accessi e la gestione delle sessioni (tramite token).
- **Usabilità e Portabilità:** per favorire l'uso dell'applicazione da molteplici dispositivi, si è deciso di salvare i dati in modo centralizzato su un database remoto e non in locale sul dispositivo dell'utente, esponendo i servizi tramite API fruibili via web.

3.1.2. Scelta dell'architettura

Dal punto di vista architetturale, per rispettare i requisiti evidenziati, l'architettura più idonea per il sistema è un'architettura Client/Server Web a 3 livelli:

- **L1 - Client:** Il client si occupa della presentazione e dell'interazione con l'utente. Per garantire un'elevata reattività dell'interfaccia, specialmente per funzionalità che richiedono aggiornamenti dinamici come la Chat di gruppo, si opta per un approccio basato su *fat client* (Single Page Application). In questo modo il client delega le elaborazioni complesse al server e gestisce lo stato dell'interfaccia.
- **L2 - Server:** Il server si occupa di accentrare la logica di business, il controllo degli accessi e la validazione dei dati. Il server metterà a disposizione delle API REST (*Representational State Transfer*) *stateless* per la gestione standard dell'applicativo. Inoltre, per gestire le funzionalità in tempo reale come la Chat di gruppo, il server gestirà connessioni persistenti e bidirezionali tramite protocollo WebSocket. L'utilizzo di API *stateless* unito a WebSocket permette di coniugare scalabilità e reattività.
- **L3 - Persistenza:** Per la gestione della persistenza si avrà un server dedicato nel quale sarà installato un opportuno DBMS che gestirà in modo completo la base dati dell'applicazione. Per risolvere il conflitto di impedenza tra il modello a oggetti del server e il modello relazionale del database, verrà utilizzato il paradigma Object-Relational Mapping (ORM).

3.1.3. Scelte tecnologiche

Al fine di favorire la separazione delle responsabilità e la manutenibilità del codice, si è scelto di adottare il pattern architetturale Model-View-Controller (MVC) sul lato server e Model-View-ViewModel (MVVM) sul lato frontend.

Per la comunicazione tra L1 e L2 verrà utilizzato il protocollo HTTPS, in modo che il livello TLS garantisca l'integrità e la riservatezza dei dati scambiati, tutelando la privacy

degli utenti. Le richieste alle API saranno autenticate tramite l'uso di token di sessione sicuri (es. JSON Web Token).

In aggiunta alle API REST, data l'importanza del requisito della Chat di gruppo per gli Eventi di studio, il server adotterà tecnologie capaci di mantenere una comunicazione bidirezionale in tempo reale (come il protocollo WebSocket), permettendo ai client di ricevere e inviare i nuovi messaggi con latenza minima.

Per il Client (L1), verrà impiegato un framework web frontend reattivo moderno (React), utilizzando lo strumento di build *Vite* per velocizzare l'ambiente di sviluppo locale e ottimizzare la compilazione finale dei file statici. Per l'implementazione del Server (L2), si utilizzerà un framework web lato server robusto, orientato agli oggetti e ad alte prestazioni (es. ASP.NET con C#). Per la Persistenza (L3), la scelta ricade su un DBMS relazionale (PostgreSQL), adatto alla natura strutturata dei dati del dominio, interfacciato tramite la libreria ORM nativa.

3.1.4. Diagramma dei package

Di seguito è riportato il diagramma dei package dell'applicazione. La presenza di un layer di persistenza (viola) tra i controller e il modello permette una certa trasparenza rispetto alle operazioni di salvataggio e lettura dei dati nel livello dei controller. Si sceglie di utilizzare la tecnica ORM (come spiegato in L3 - Persistenza) per mappare le entità alle tabelle del database.

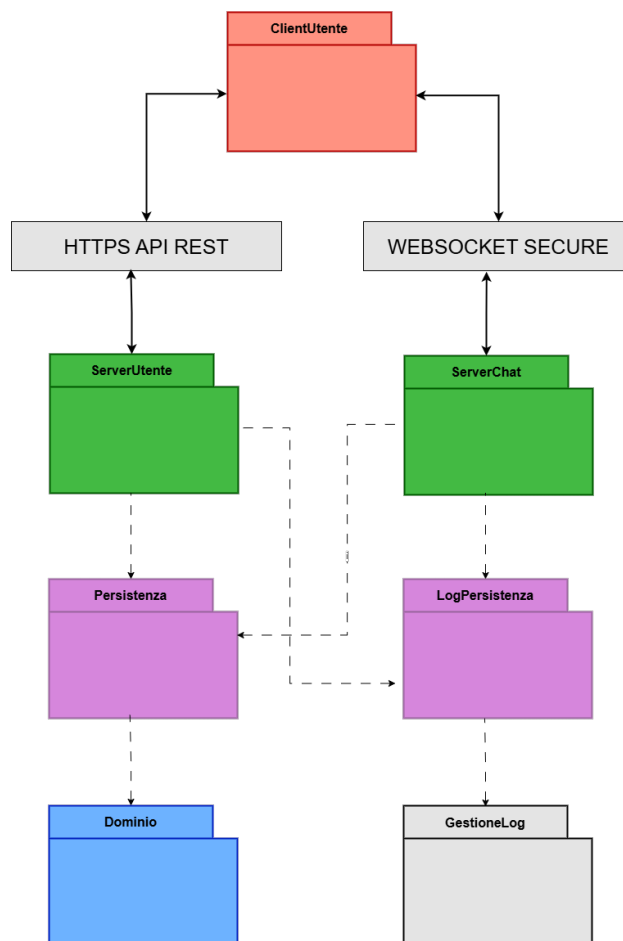


Figura 3.1: Diagramma dei package: Progettazione

3.1.5. Diagramma dei componenti

Nella figura sottostante è riportata l'architettura del Sistema, organizzata attraverso un diagramma del componenti.

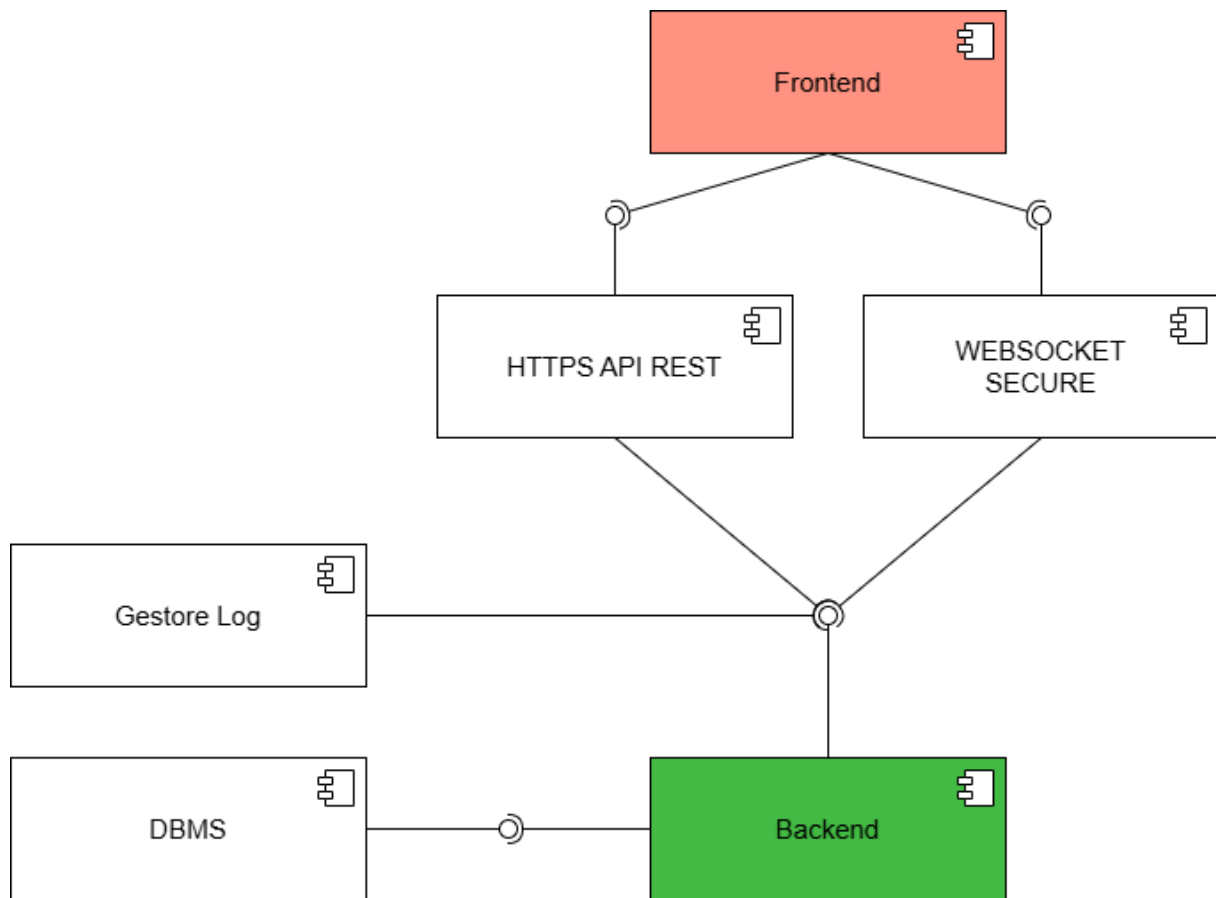


Figura 3.2: Diagramma dei componenti

3.2. Progettazione di dettaglio

3.2.1. Struttura

3.2.1.1. Diagrammi di dettaglio del modello

3.2.1.1.1. Diagramma di dettaglio: Account

Di seguito è presentato il diagramma delle classi del modello del dominio riguardante l'Account.

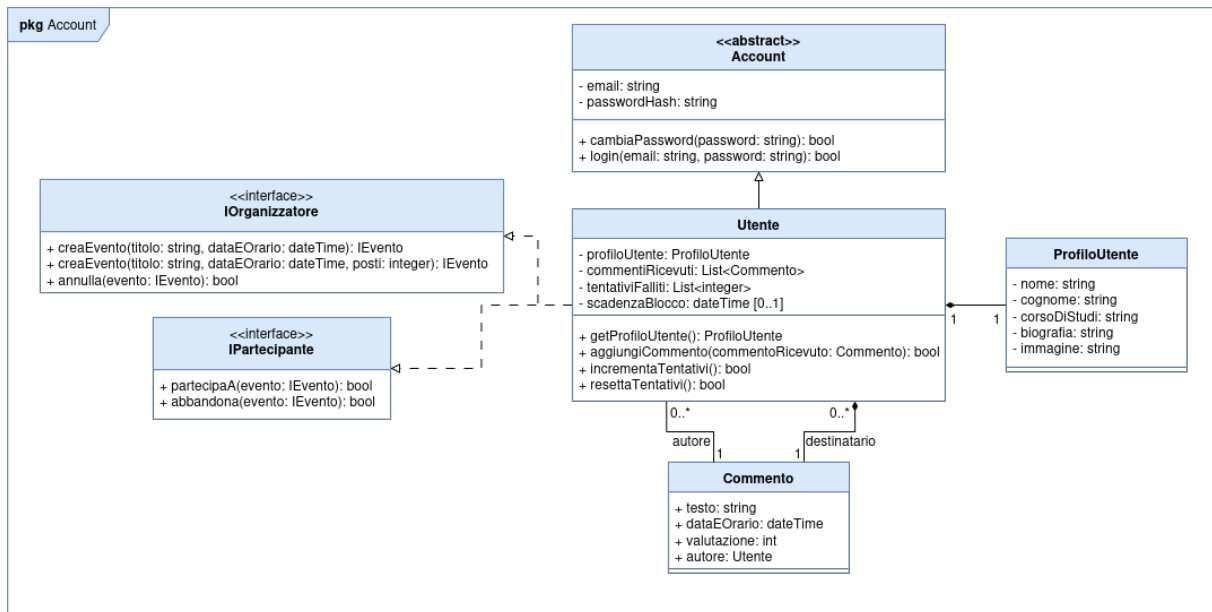


Figura 3.3: Diagramma di dettaglio: Account

In questo diagramma è da notare che:

- Il concetto generale di fruitore del sistema è stato rinominato da **Utente** ad **Account** per favorire l'espandibilità dell'applicazione.
- È stata introdotta la classe astratta **Account**, la quale incapsula esclusivamente i dati fondamentali di base (come l'email), mentre le informazioni sensibili di autenticazione (come l'hash della password) sono gestite direttamente dal database tramite il contesto.
- La sottoclasse **Utente** modella lo studente standard che utilizza l'app, raggruppando i dettagli del **ProfiloUtente** e la lista dei **commentiRicevuti**. Questa separazione permette l'introduzione futura di nuovi account istituzionali (come ad esempio un account gestito direttamente da UniBo per promuovere sessioni di studio) che non necessitano di campi incompatibili, come il corso di studi o il sistema di recensioni tramite commenti.
- Per favorire il disaccoppiamento e rispettare il principio di segregazione delle interfacce (ISP), sono state create le interfacce **IOrganizzatore** e **IPartecipante**. In questo modo, eventuali enti organizzatori futuri potranno implementare solo la logica di creazione degli eventi, senza essere forzati ad implementare la logica di partecipazione, tipica invece delle persone fisiche.
- Rispetto all'analisi, in cui erano presenti due entità separate (**CommentoInviato** e **CommentoRicevuto**), si è scelto di unificare la struttura in una singola classe **Commento**. Questa contiene gli attributi comuni (testo, valutazione, data e orario) ed esprime il mittente e il destinatario attraverso due relazioni distinte con l'entità **Utente**. Tale approccio riduce la duplicazione del codice, semplifica la logica di business e agevola

la persistenza dei dati sul database, mantenendo al contempo una chiara tracciabilità bidirezionale.

- A livello di modello logico, per l'Account sono stati previsti attributi come **tentativiFalliti** e **scadenzaBlocco** per gestire la sicurezza dell'accesso: dopo 5 errori di autenticazione consecutivi, l'account viene sospeso temporaneamente.

3.2.1.1.2. Diagramma di dettaglio: Evento

Di seguito è presentato il diagramma delle classi del modello del dominio riguardante gli Eventi.

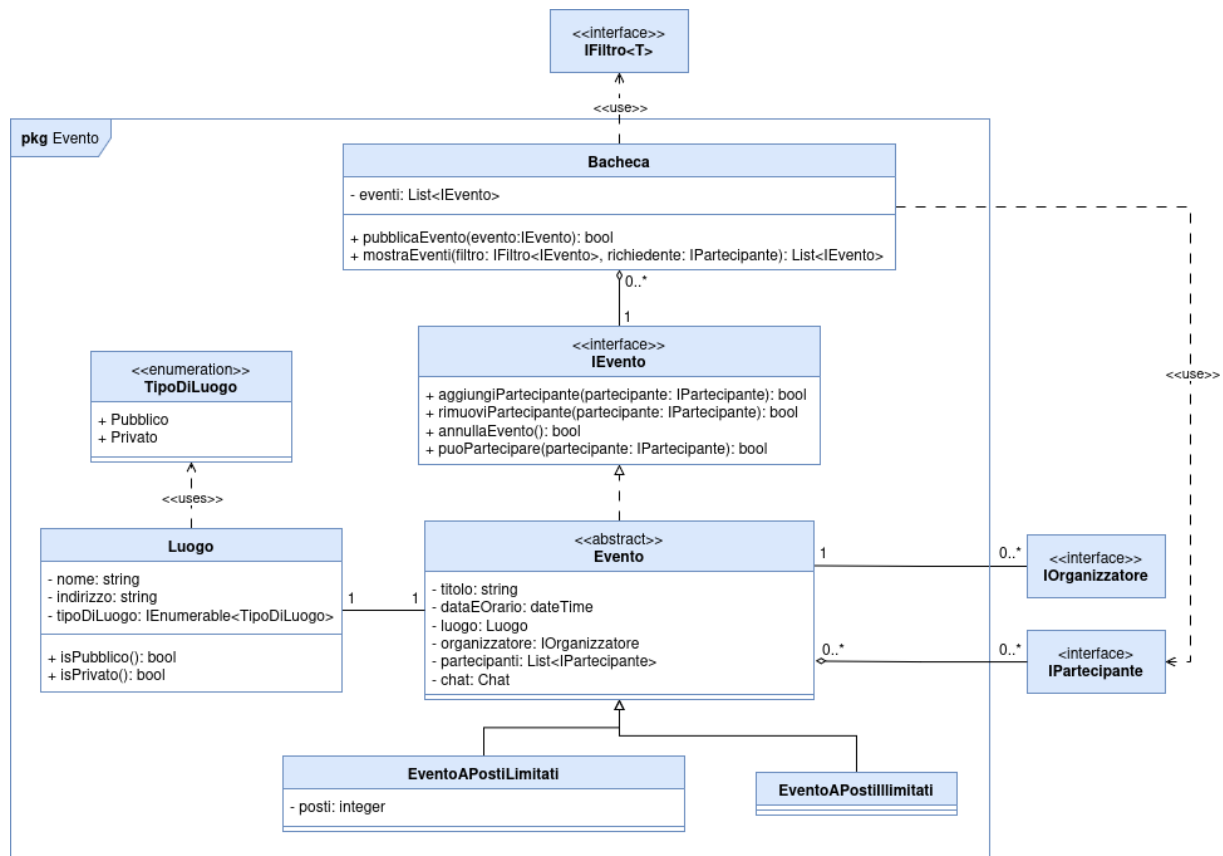


Figura 3.4: Diagramma di dettaglio: Evento

In questo diagramma è da notare che:

- È stata definita l'interfaccia `IEvento` che impone i metodi contrattuali di base (`aggiungiPartecipante`, `rimuoviPartecipante`, `annullaEvento`, `puoPartecipare`). Questo garantisce un alto grado di espandibilità (Open/Closed Principle): se in futuro si volessero modellare nuove tipologie di attività, come ad esempio sessioni di studio telematiche (che necessitano di un link anziché di un Indirizzo fisico) o eventi come esami o conferenze, queste potranno implementare l'interfaccia senza alterare la logica preesistente del sistema.
- Rispetto alla fase di analisi in cui la capienza era un semplice parametro, la classe astratta `Evento` si specializza nelle classi concrete `EventoAPostiLimitati` (che introduce l'attributo privato `posti`) ed `EventoAPostiIllimitati`. Questa scelta garantisce scalabilità ed estensibilità: le logiche di validazione delle iscrizioni e il calcolo dei posti disponibili sono delegate, evitando l'uso di complessi blocchi condizionali nel codice client e facilitando l'eventuale introduzione futura di eventi con logiche di accesso differenti.
- È stata introdotta la classe `Bacheca` per centralizzare la gestione e l'accesso agli eventi. Essa funge da punto di raccolta unico e fornisce i metodi utili alla pubblicazione e al filtraggio degli eventi per l'interfaccia utente.
- L'introduzione dell'enumerazione `TipoDiLuogo` permette di limitare e tipizzare in modo sicuro la definizione di un luogo, specificando chiaramente se esso è pubblico o privato.

3.2.1.1.3. Diagramma di dettaglio: Chat

Di seguito è presentato il diagramma delle classi del modello del dominio riguardante la Chat.

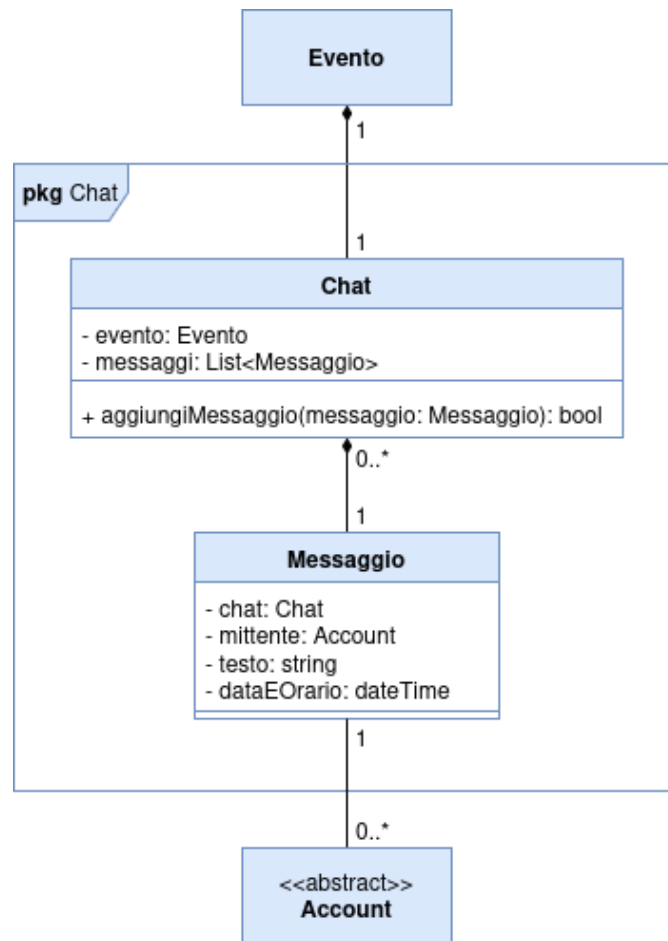


Figura 3.5: Diagramma di dettaglio: Chat

In questo diagramma è da notare che:

- La struttura è stata progettata per essere estremamente lineare e focalizzata. La classe **Chat** modella l'insieme della conversazione fungendo da aggregatore per la lista di messaggi.
- La cardinalità definisce in modo stringente che una **Chat** appartiene ad uno e un solo **Evento**.
- La classe **Messaggio** incapsula tutte le informazioni necessarie per la tracciabilità delle comunicazioni, mantenendo al suo interno i riferimenti alla **Chat** di appartenenza, all'**Account** del mittente, il contenuto testuale e il timestamp temporale (**dataEOrario**).

3.2.1.1.4. Diagramma di dettaglio: Filtro

Di seguito è presentato il diagramma delle classi del modello del dominio riguardante i Filtri.

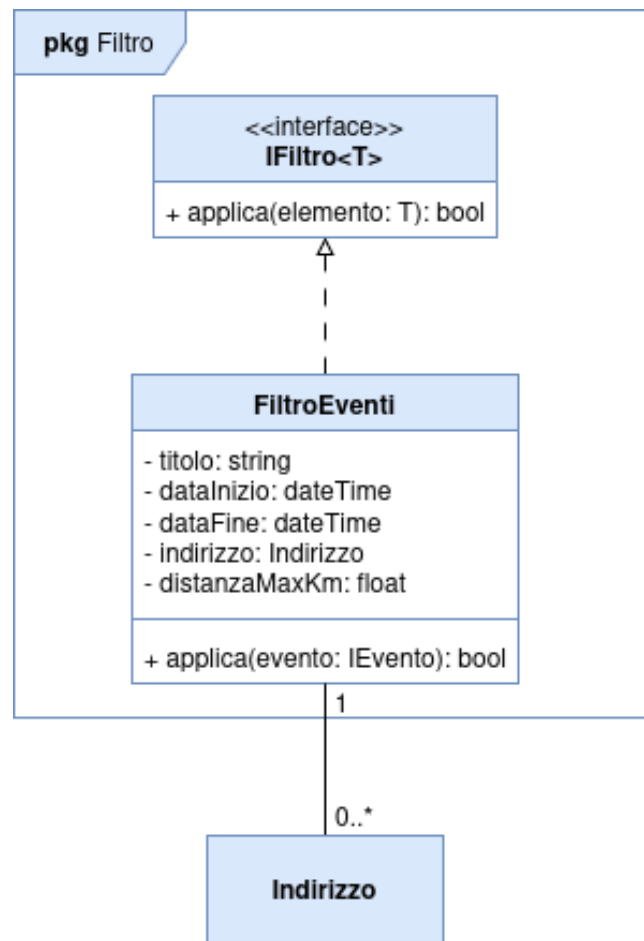


Figura 3.6: Diagramma di dettaglio: Filtro

In questo diagramma è da notare che:

- È stata creata l'interfaccia generica `IFiltro<T>` dotata del metodo `applica()`. Questa scelta architetturale applica il pattern Strategy e garantisce il pieno rispetto del principio Open/Closed (OCP), permettendo di aggiungere agilmente in futuro nuovi criteri di filtraggio senza dover modificare il codice delle classi client (come la Bacheca).
- La classe `FiltroEventi` raggruppa logicamente tutti i molteplici parametri di ricerca (come titolo, finestre temporali, indirizzo e distanzaMaxKm), evitando così lunghe liste di parametri nei metodi e migliorando la coesione dei dati di ricerca.
- Il filtro basato su `distanzaMaxKm` calcola la distanza fisica (in chilometri) tra due punti geografici distinti: l'indirizzo inserito in input dall'utente (che fa da centro) e l'indirizzo dell'evento. Entrambi gli indirizzi stradali vengono geocodificati in coordinate di latitudine e longitudine; dopodiché, la distanza tra i due punti viene calcolata geometricamente sulla superficie terrestre, includendo nella bacheca del client solo gli eventi che ricadono nel raggio impostato.

3.2.1.1.5. Diagramma di dettaglio: Log

Di seguito è presentato il diagramma delle classi del modello del dominio riguardante i Log.

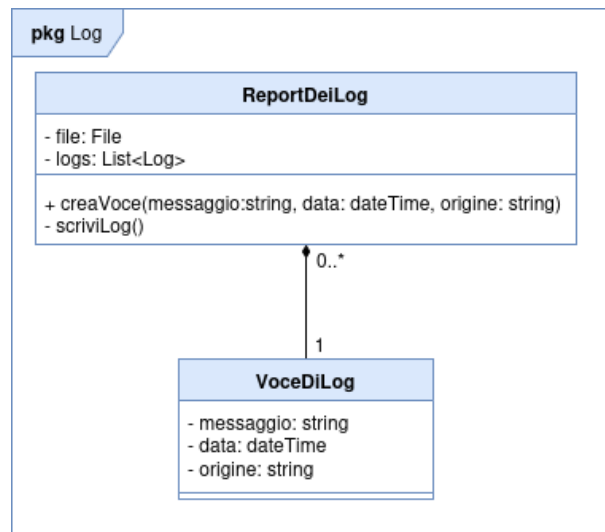


Figura 3.7: Diagramma di dettaglio: Log

In questo diagramma è da notare che:

- Nel rispetto del principio di singola responsabilità (SRP), l'intera gestione del tracciamento è stata isolata all'interno della classe **ReportDeiLog**, che si occupa di aggregare le singole entità **VoceDiLog** e gestire il file di output.
- Il metodo `scriviLog()` all'interno di **ReportDeiLog** è stato reso privato: esso è progettato per essere invocato periodicamente e in background da un **Thread** separato. Questo approccio permette di riversare sul file di testo le **VoceDiLog** generate nell'ultimo periodo senza bloccare o rallentare il flusso di esecuzione principale dell'applicazione.

3.2.1.2. Diagramma di dettaglio: Client

3.2.1.2.1. Client: Interfaccia Profilo

Di seguito è presentato il diagramma delle classi dell'interfaccia del profilo.

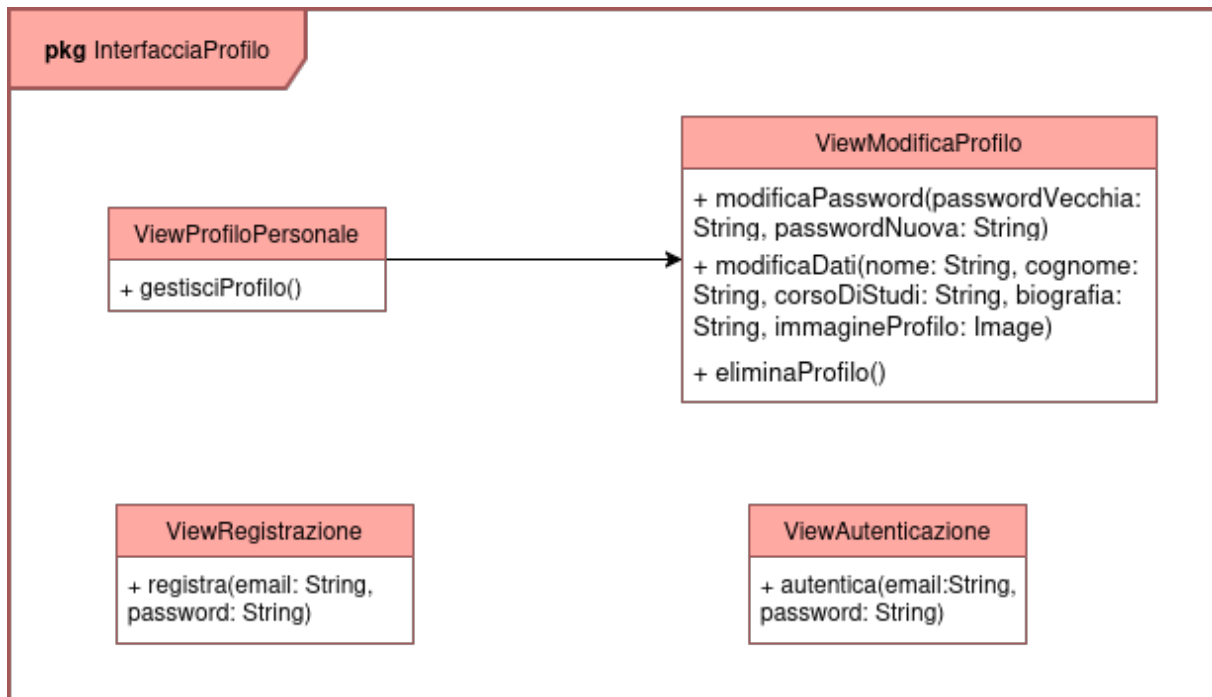


Figura 3.8: Diagramma di dettaglio: Interfaccia Profilo

É da notare che rispetto all'analisi si è scelto di separare il metodo `modificaProfilo()` nei metodi `modificaDati()` e `modificaPassword()`.

3.2.1.2.2. Client: Interfaccia Evento

Di seguito è presentato il diagramma delle classi dell'interfaccia dell'evento.

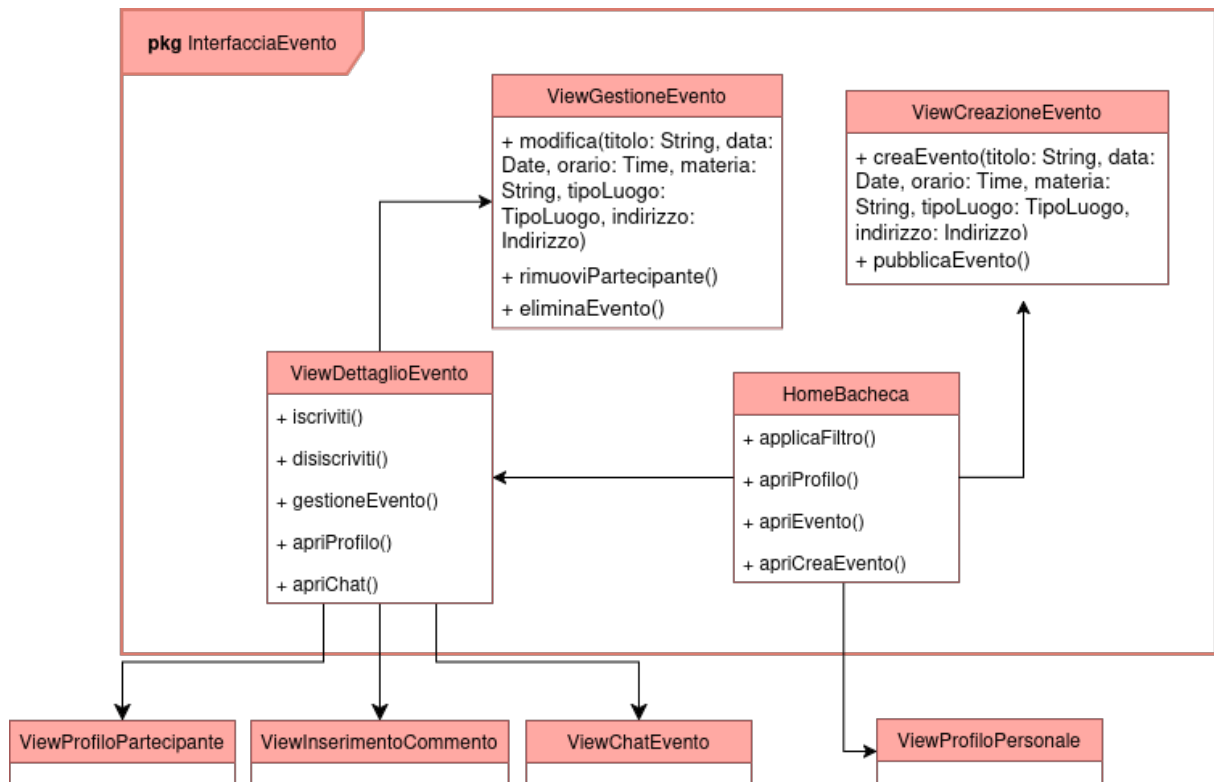


Figura 3.9: Diagramma di dettaglio: Interfaccia Evento

3.2.1.2.3. Client: Interfaccia Comunicazione

Di seguito è presentato il diagramma delle classi dell'interfaccia della comunicazione.

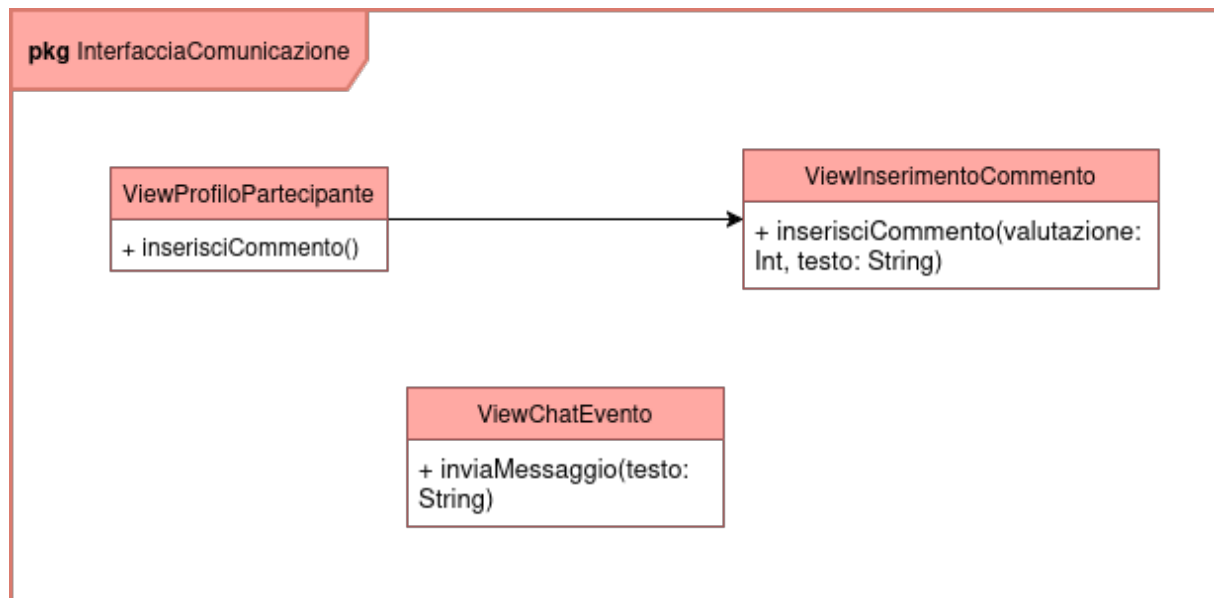


Figura 3.10: Diagramma di dettaglio: Interfaccia Comunicazione

3.2.1.3. Diagramma di dettaglio: Controller

Di seguito sono presentati i diagrammi di dettaglio relativi ai Controller dell'applicazione. Come spiegato nella scelta dell'architettura e nelle scelte tecnologiche, è stato deciso di mettere a disposizione delle API REST per permettere l'interazione tra client e server. A questo fine, è da considerare che:

- Per migliorare l'interoperabilità e renderlo più manutenibile, si è scelto di utilizzare i DTO (Data Transfer Object) in modo da non esporre direttamente il dominio. Dato che generalmente rispettano la struttura delle classi già mostrate, in questo documento si ignoreranno.
- Per rendere più efficiente e sostenibile la comunicazione, si è scelto di utilizzare la paginazione quando si richiedono liste di oggetti, in modo da non sovraccaricare il server e il client con dati inutili. Tuttavia, per semplificare la lettura dei diagrammi, non si è rappresentata la paginazione.
- Per gestire l'autorizzazione e l'autenticazione, si è scelto di utilizzare i token JWT (JSON Web Token) che permettono di identificare l'utente e le sue autorizzazioni. Per semplificare la lettura dei diagrammi, la loro gestione non è stata rappresentata.

3.2.1.3.1. Server: Interfacce

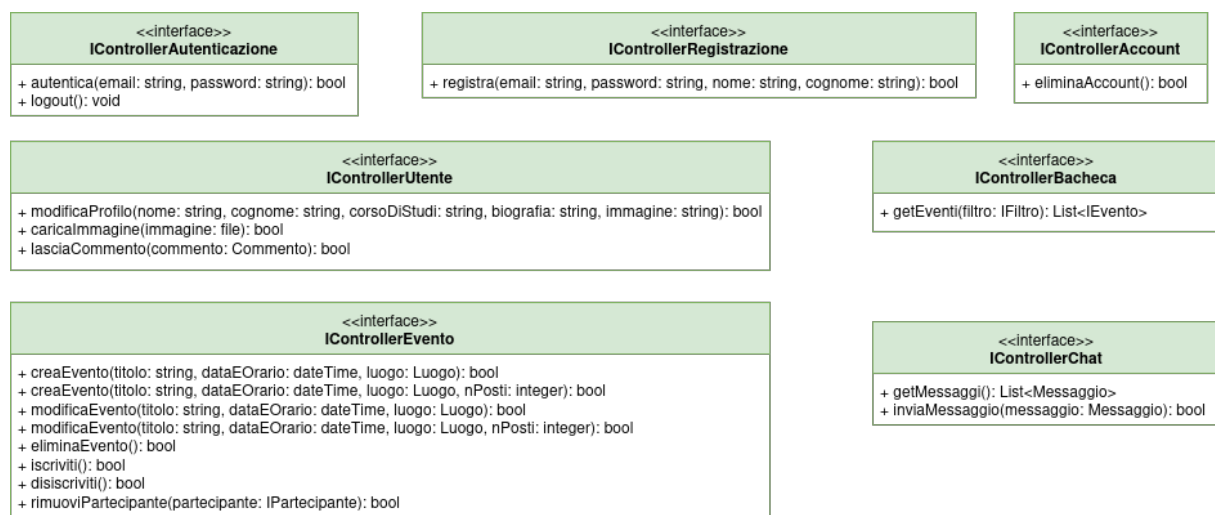


Figura 3.11: Diagramma di dettaglio: Interfacce Controller

3.2.1.3.2. Server: Controller

Di seguito è presentato il diagramma di dettaglio delle implementazioni dei controller.

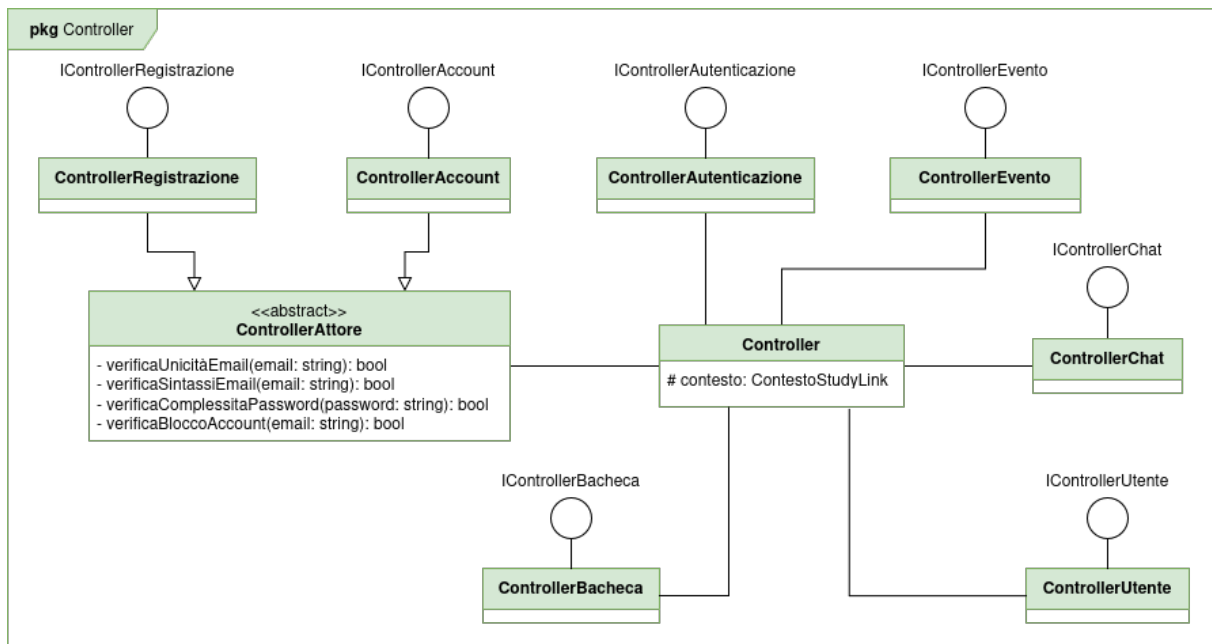


Figura 3.12: Diagramma di dettaglio: Implementazione Controller

È da notare che:

- Tutti i controller condividono una classe base comune **Controller**, la quale detiene il riferimento protetto al contesto del database (`# contesto: ContestoStudyLink`). In questo modo, l'accesso alle tabelle tramite ORM è centralizzato ed ereditato in modo uniforme.
- È stata introdotta la classe astratta **ControllerAttore** (che estende la classe base **Controller**) come superclasse per **ControllerRegistrazione** e **ControllerAccount**, centralizzando le logiche di validazione legate all'anagrafica degli attori.
- All'interno di **ControllerAttore** sono definiti esclusivamente come privati (-) i metodi di validazione sensibili: `verificaUnicitàEmail()`, `verificaSintassiEmail()` e `verificaComplessitàPassword()`. Questa soluzione assicura che tali metodi critici non siano accessibili o invocabili dall'esterno dell'architettura dei controller, garantendo una maggiore sicurezza ed evitando la duplicazione di codice.
- Ogni controller concreto implementa la sua rispettiva interfaccia (come ad esempio **IControllerRegistrazione** e **IControllerAccount**), garantendo il disaccoppiamento tra la definizione delle API del server e la loro effettiva implementazione.
- All'interno del controller preposto all'autenticazione è implementata la logica per il blocco temporaneo dell'account in caso di molteplici errori. Durante il login, il controller verifica i `tentativiFalliti`: se raggiungono o superano quota 5, il sistema rifiuta l'accesso a prescindere dalla validità delle credenziali inserite e imposta la `scadenzaBlocco` per inibire ulteriori tentativi per 15 minuti.

3.2.1.4. Diagramma di dettaglio: Persistenza

3.2.1.4.1. Persistenza: ContestoStudyLink

La classe ContestoStudyLink è utilizzata dall'ORM per interfacciarsi con il DBMS.

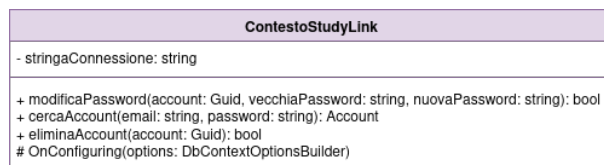


Figura 3.13: Diagramma di dettaglio: Persistenza

3.2.2. Interazioni

Di seguito sono presentate le figure che mostrano le interazioni tra i componenti per le principali funzionalità del sistema.

3.2.2.1. Interazione: Autenticazione

Di seguito è presentata la figura che mostra le interazioni tra i componenti per l'autenticazione dell'utente.

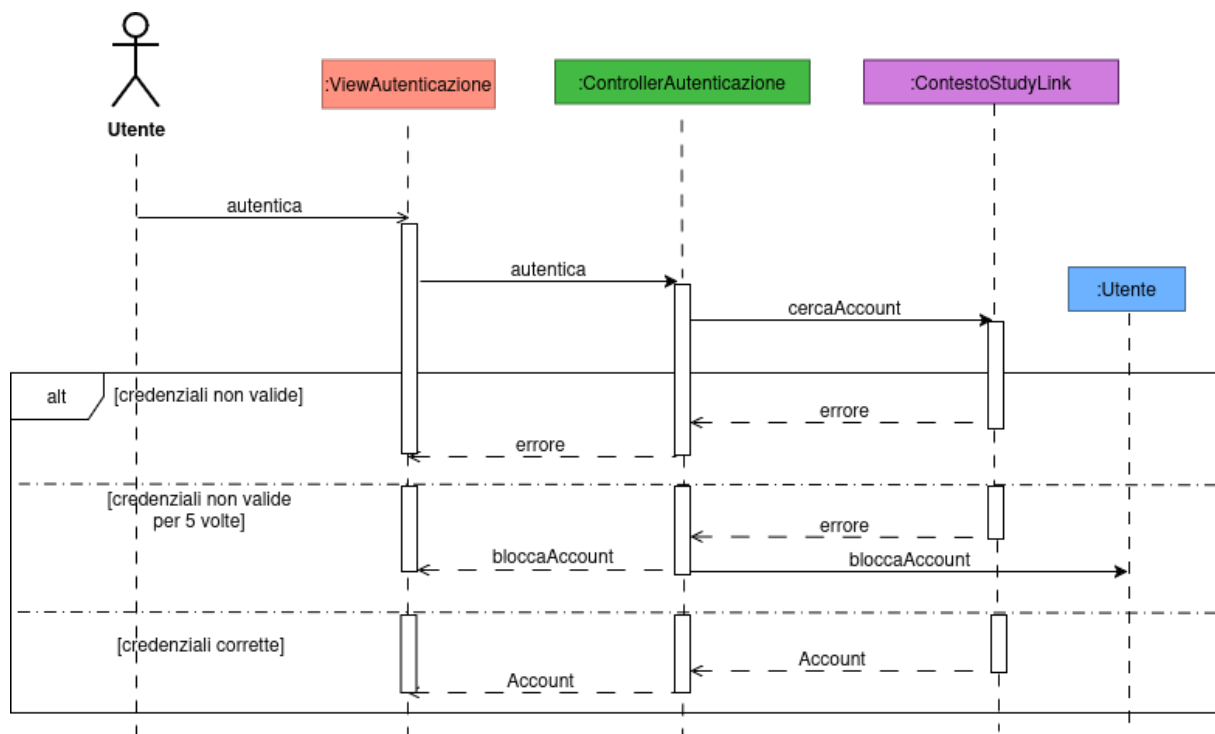


Figura 3.14: Diagramma di sequenza: Autenticazione

3.2.2.2. Interazione: Gestione profilo

Di seguito sono presentate le figure che mostrano le interazioni tra i componenti per la gestione del profilo dell'utente.

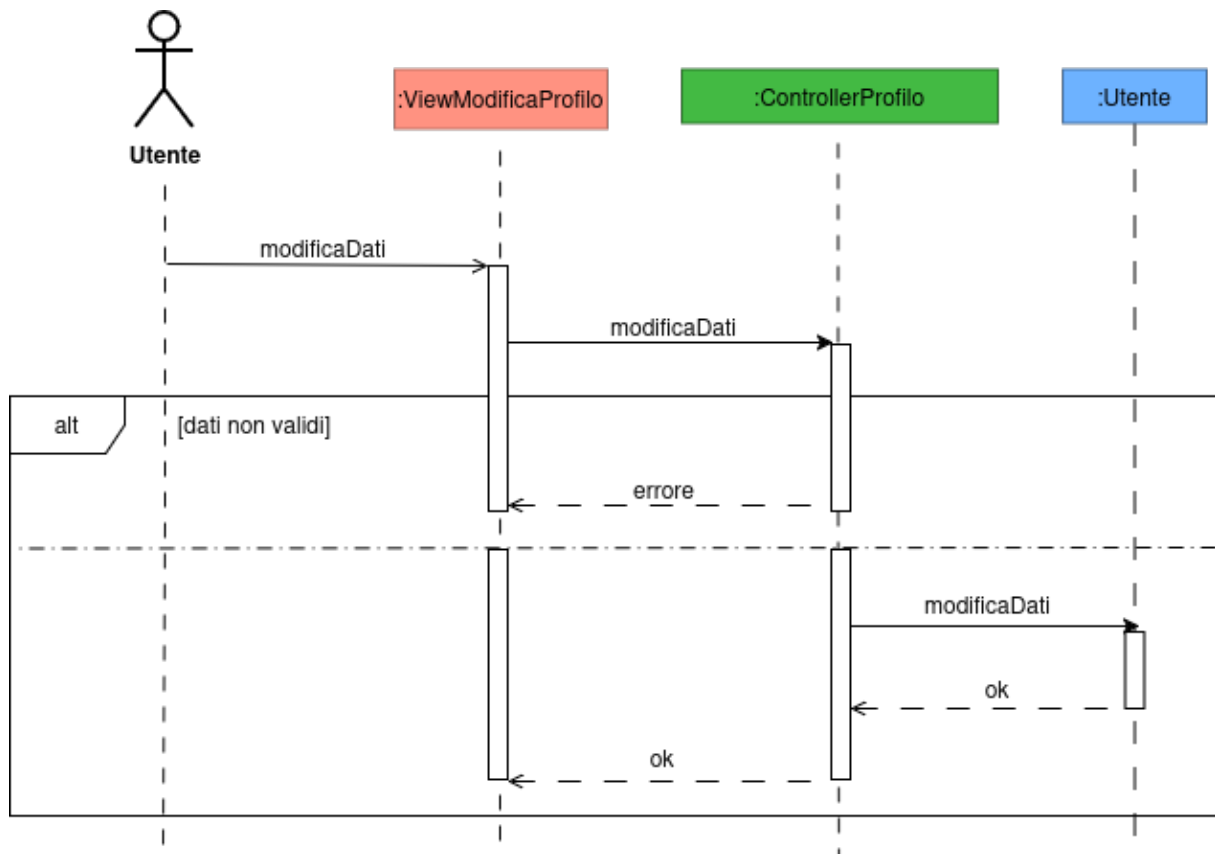


Figura 3.15: Diagramma di sequenza: Gestione profilo - Modifica dati

Il metodo che in analisi è `modificaProfilo()` è stato diviso in `modificaDati()` e `modificaPassword()` per poter applicare controlli diversi a seconda dell'importanza dei dati. Anche se non è indicato nel diagramma, per motivi di spazio, ci sarà la memorizzazione dei dati sul database e la scrittura dei log.

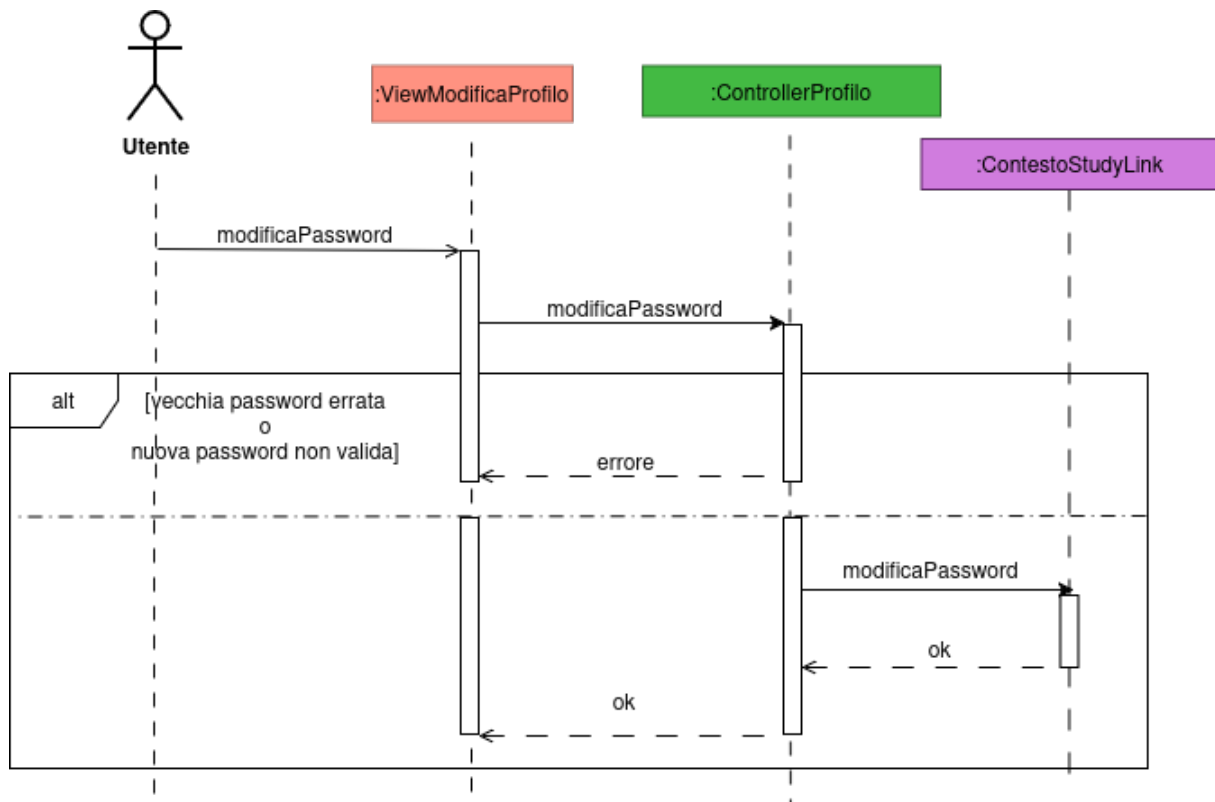


Figura 3.16: Diagramma di sequenza: Gestione profilo - Modifica password

3.2.2.3. Interazione: Elimina account

Di seguito è presentata la figura che mostra le interazioni tra i componenti per l'eliminazione dell'account.

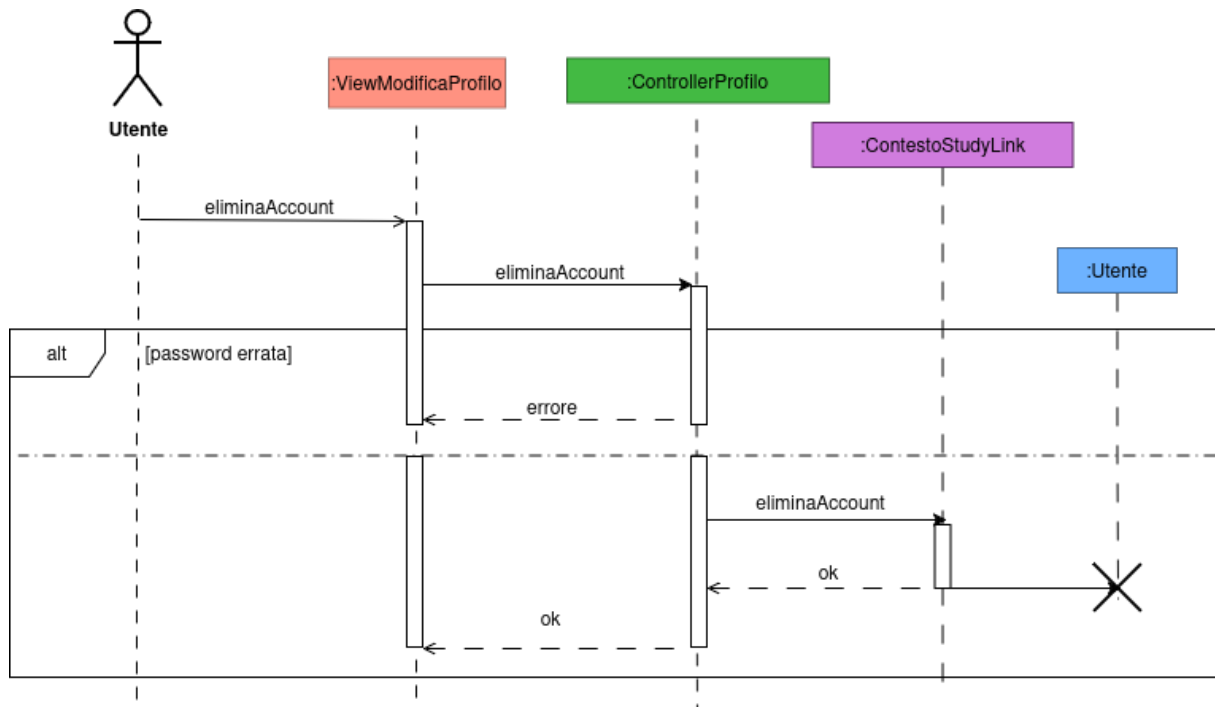


Figura 3.17: Diagramma di sequenza: Elimina account

3.2.2.4. Interazione: Gestione evento

Di seguito sono presentate le figure che mostrano le interazioni tra i componenti per la gestione degli eventi. Per brevità e per mostrare le differenze con quelli mostrati in analisi, si è riportato solo il caso della creazione e della gestione dell'evento. Anche se non è indicato nel diagramma, per motivi di spazio, ci sarà la memorizzazione dei dati sul database.

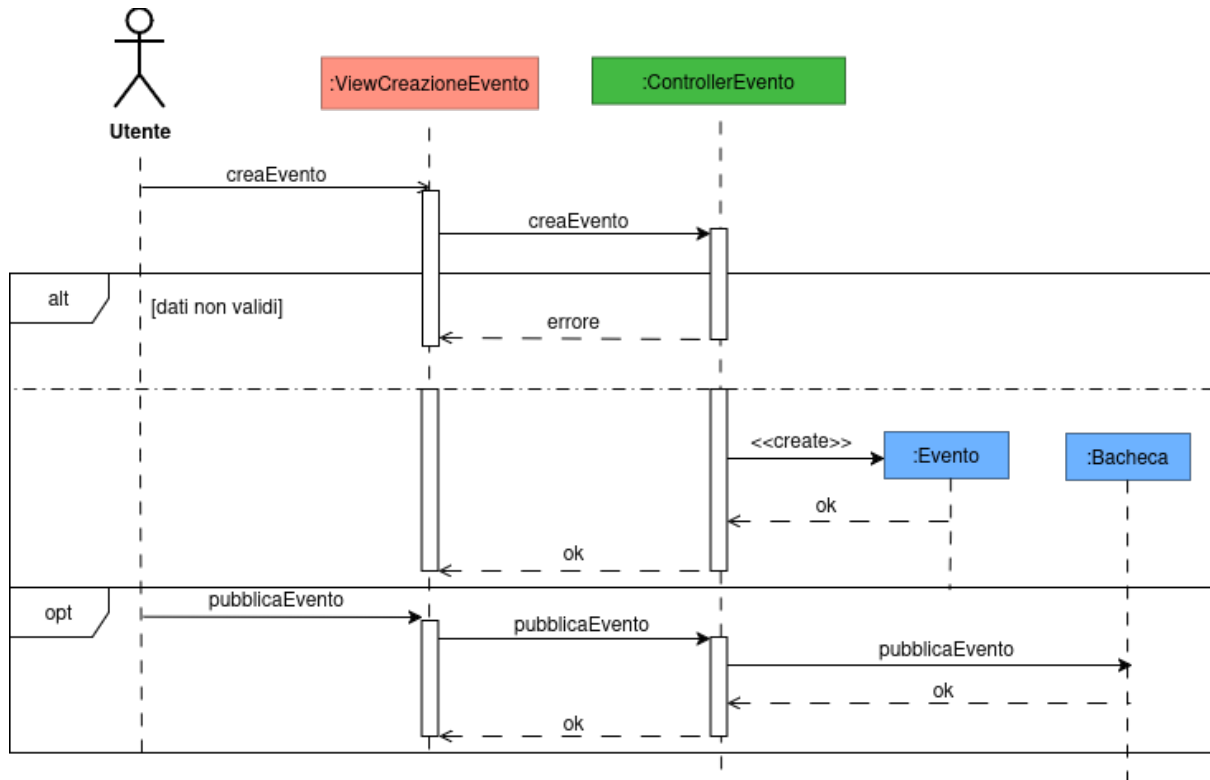


Figura 3.18: Diagramma di sequenza: Gestione evento - Creazione

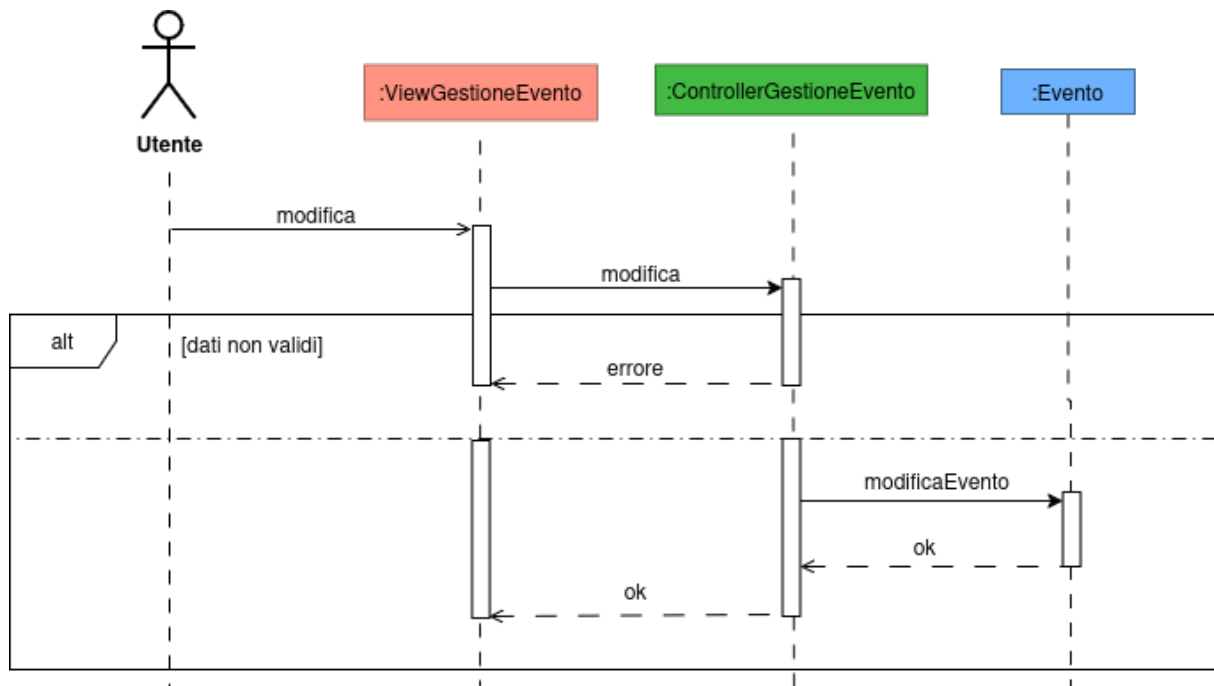


Figura 3.19: Diagramma di sequenza: Gestione evento - Gestione

3.2.2.5. Interazione: Elimina evento

Si omette il diagramma relativo all'eliminazione dell'evento in quanto il suo flusso di interazione risulta essere praticamente identico a quello della modifica dell'evento, differendo unicamente nell'operazione finale di eliminazione dell'entità anziché di aggiornamento.

3.2.2.6. Interazione: Bacheca

Di seguito è presentata la figura che mostra le interazioni tra i componenti per il filtraggio degli eventi in bacheca.

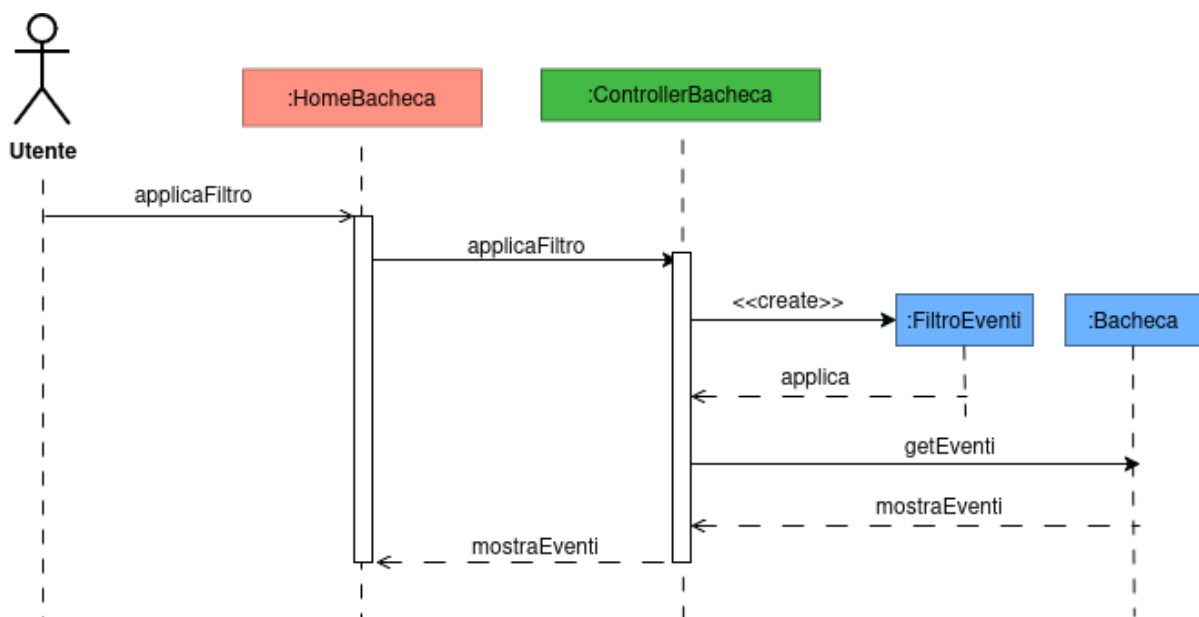


Figura 3.20: Diagramma di sequenza: Bacheca - Filtro eventi

3.2.2.7. Interazione: Comunicazione

Di seguito sono presentate le figure che mostrano le interazioni tra i componenti per la comunicazione tra gli utenti tramite chat e commenti.

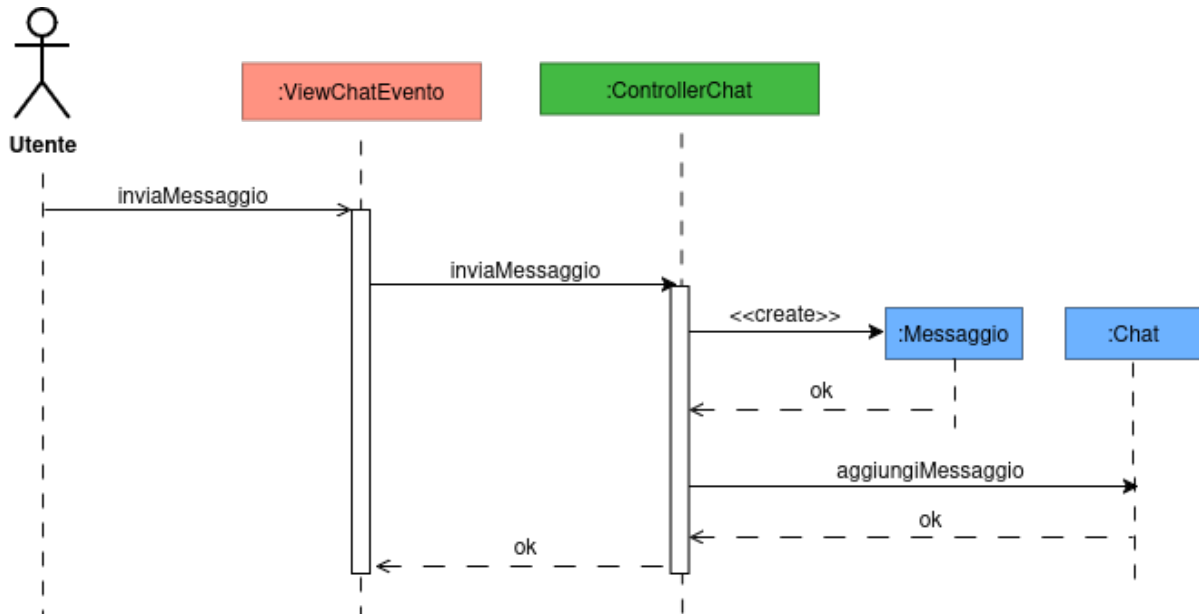


Figura 3.21: Diagramma di sequenza: Comunicazione - Chat

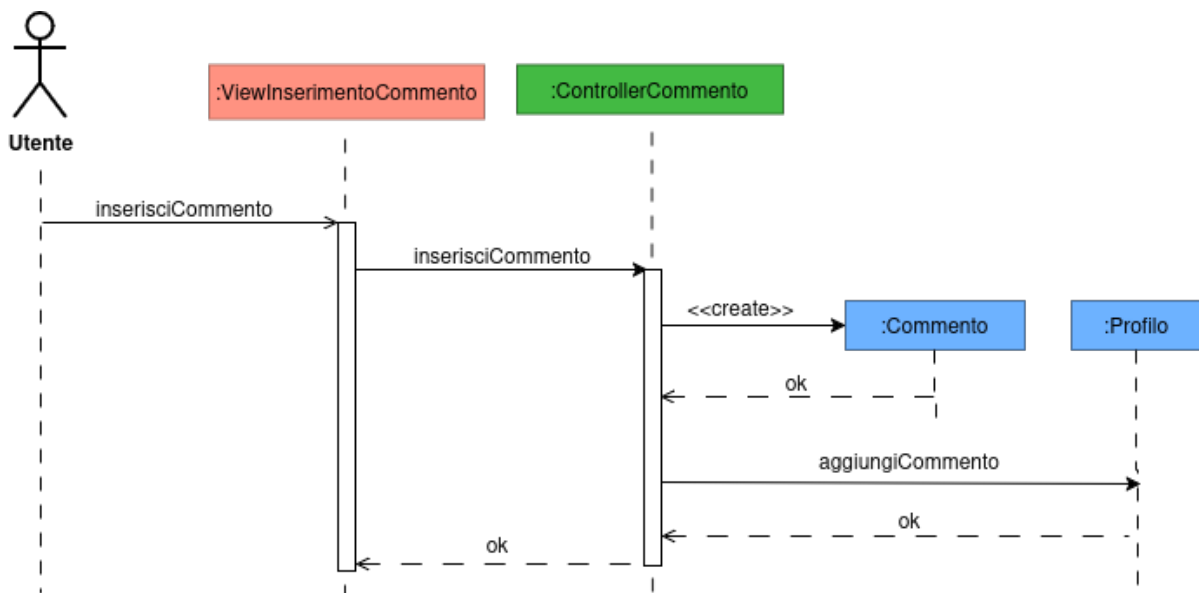


Figura 3.22: Diagramma di sequenza: Comunicazione - Commento

3.2.3. Interfacce utente

Nelle prossime sezioni mostreremo le interfacce principali del sistema. Visto lo scopo dell'applicazione, prevediamo che gli utenti la useranno quasi sempre dal telefono. Per questo motivo, in questo documento abbiamo deciso di inserire solo le schermate della versione per smartphone.

Non abbiamo messo la versione per computer perché è molto facile da immaginare: si tratta semplicemente della stessa grafica allargata e riadattata per uno schermo più grande.

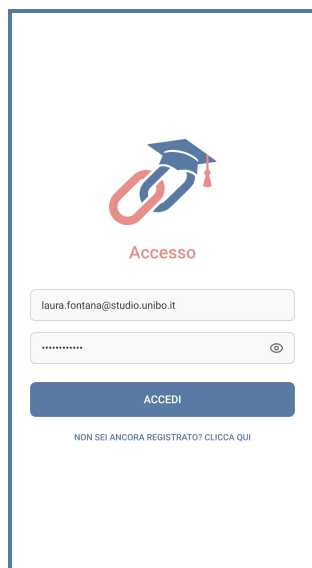


Figura 3.23: Schermata di Accesso

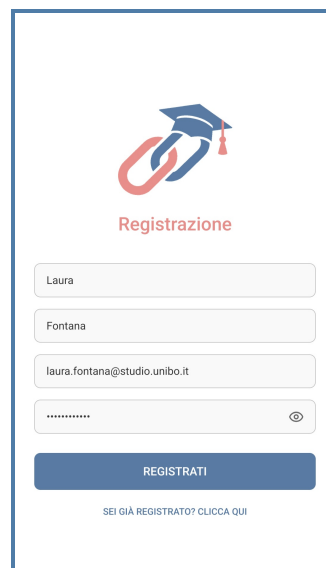


Figura 3.24: Schermata di Registrazione



Figura 3.25: Bacheca - Esplora

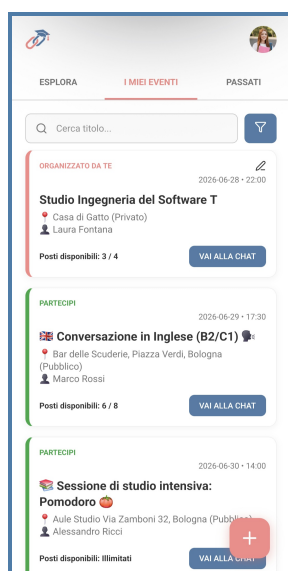


Figura 3.26: Bacheca - I Miei Eventi

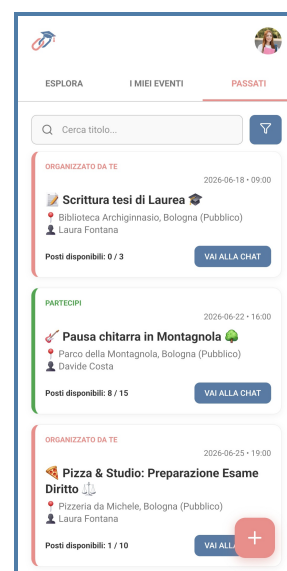
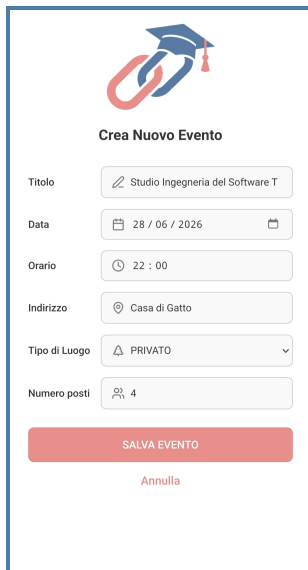


Figura 3.27: Bacheca - Eventi Passati



Crea Nuovo Evento

Titolo: Studio Ingegneria del Software T

Data: 28 / 06 / 2026

Orario: 22 : 00

Indirizzo: Casa di Gatto

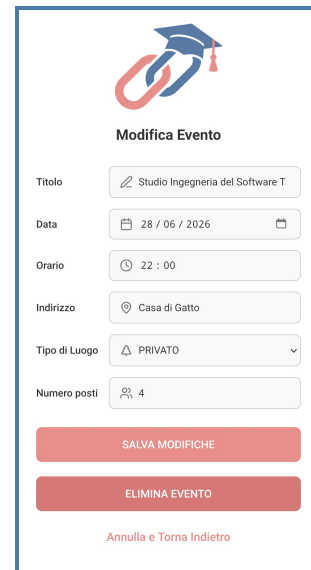
Tipo di Luogo: PRIVATO

Numero posti: 4

SALVA EVENTO

Annulla

Figura 3.28: Creazione Evento



Modifica Evento

Titolo: Studio Ingegneria del Software T

Data: 28 / 06 / 2026

Orario: 22 : 00

Indirizzo: Casa di Gatto

Tipo di Luogo: PRIVATO

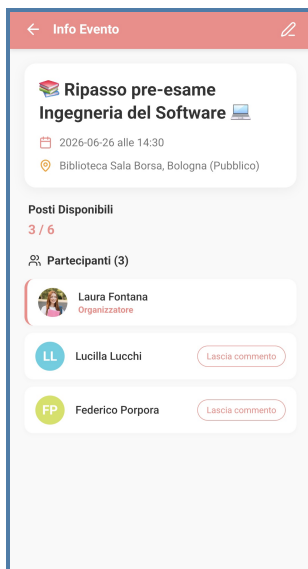
Numero posti: 4

SALVA MODIFICHE

ELIMINA EVENTO

Annulla e Torna Indietro

Figura 3.29: Modifica Evento



Info Evento

Ripasso pre-esame Ingegneria del Software

2026-06-26 alle 14:30

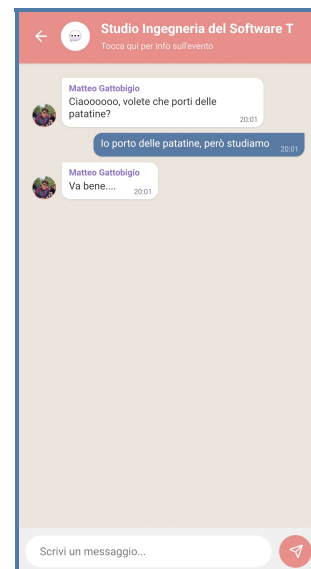
Biblioteca Sala Borsa, Bologna (Pubblico)

Posti Disponibili
3 / 6

Partecipanti (3)

- Laura Fontana (Organizzatore)
- Lucilla Lucchi (Lascia commento)
- Federico Porpora (Lascia commento)

Figura 3.30: Dettagli Evento Passato



Studio Ingegneria del Software T
Tocca qui per info sull'evento

Matteo Gattobigio: Ciao ciao, volete che porti delle patatine? 20:01

Io porto delle patatine, però studiamo 20:01

Matteo Gattobigio: Va bene.... 20:01

Scrivi un messaggio...

Figura 3.31: Chat di Gruppo

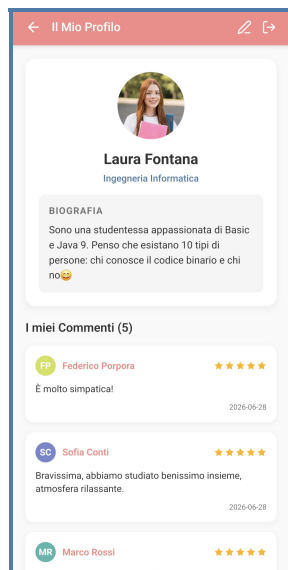


Figura 3.32: Profilo Personale

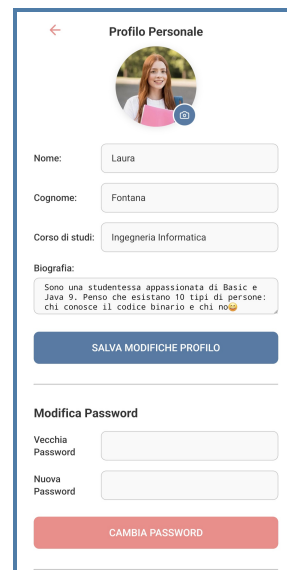


Figura 3.33: Modifica Profilo



Figura 3.34: Profilo Pubblico Utente

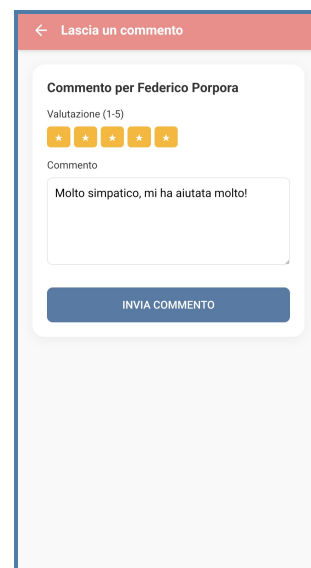


Figura 3.35: Aggiunta Commento

3.3. Progettazione della persistenza

Di seguito è riportato il diagramma ER della persistenza del sistema.

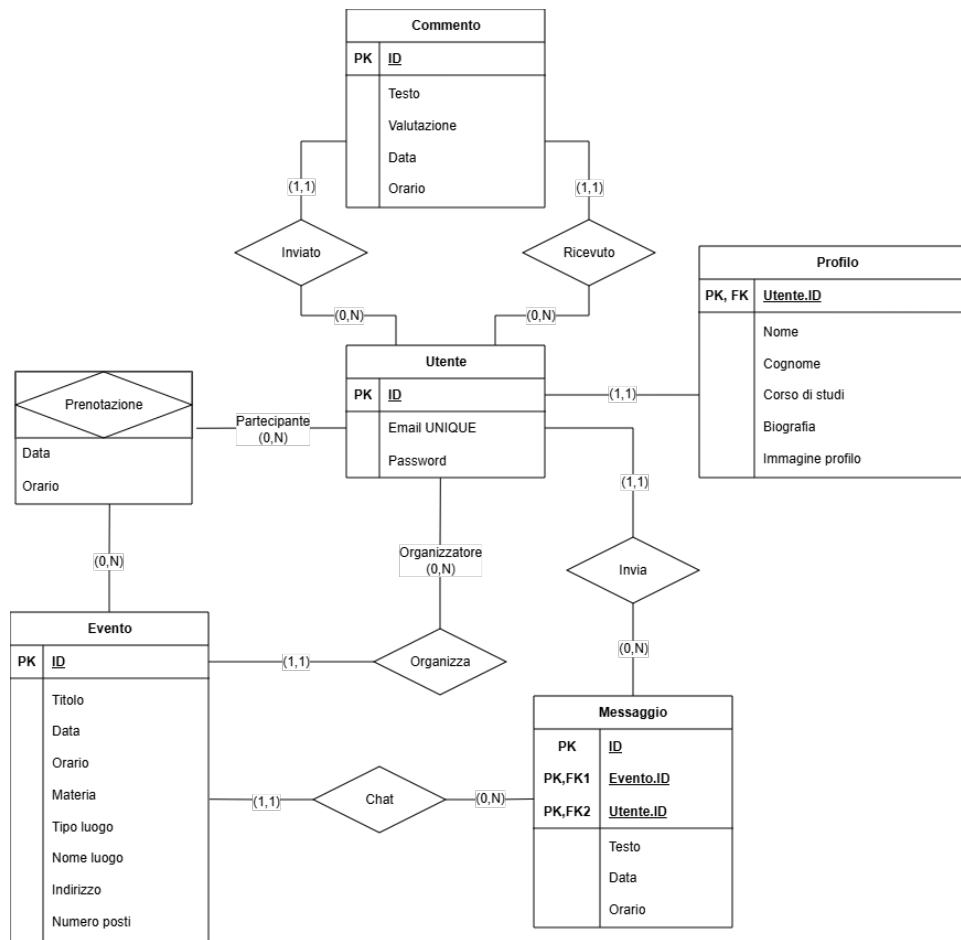


Figura 3.36: Diagramma Entity-Relationship

È da notare che:

- **Separazione tra autenticazione e profilo (1:1)**: L'entità **Profilo** è separata da **Utente** tramite una relazione uno-a-uno, in cui la chiave primaria di **Profilo** funge contemporaneamente da chiave esterna riferita a **Utente.ID**. Questo evita di sovraccaricare la tabella delle credenziali con dati personali e pesanti (come l'immagine di profilo), ottimizzando le operazioni di autenticazione.
- **Doppia relazione Utente-Commento**: L'entità **Commento** è legata a **Utente** attraverso due relazioni distinte: **Inviato** (lo studente autore del commento) e **Ricevuto** (lo studente che riceve la recensione). Questa struttura consente di implementare un sistema di feedback bidirezionale chiaro, tracciando univocamente mittente e destinatario.
- **Prenotazione come entità associativa**: La partecipazione degli utenti agli eventi è gestita tramite la relazione molti-a-molti **Prenotazione**, arricchita dagli attributi **Data** e **Orario**. Questo consente di memorizzare non solo l'associazione tra utente ed evento, ma anche il momento esatto in cui è avvenuta la prenotazione.
- **Messaggistica e vincolo di contesto**: L'entità **Messaggio** ha una chiave primaria composta che include le chiavi esterne riferite sia a **Evento** (**Evento.ID**) sia a **Utente** (**Utente.ID**).

Utente (**Utente.ID**). Ciò assicura l'integrità referenziale, legando indissolubilmente ogni messaggio alla specifica chat dell'evento e all'utente che lo ha inviato.

3.4. Progettazione del collaudo

Il collaudo è effettuato attraverso sistemi di testing automatici eseguiti sulle modifiche del codice (*continuous integration*). I test sono divisi in tre categorie a seconda di ciò che vanno a verificare:

- **Test di unità:** controllano il comportamento dei singoli componenti;
- **Test di integrazione:** controllano le interazioni tra i vari componenti;
- **Test punto a punto:** controllano il funzionamento del sistema nel suo complesso.

Coerentemente con il Piano del collaudo (Sezione 2.9), sono presentati solo i test di unità delle classi ritenute centrali per il funzionamento del sistema, riallineati alla struttura emersa dalla Progettazione di dettaglio (Sezione 3.2).

In particolare, rispetto all'analisi, l'entità **Profilo** è stata scomposta nelle classi **Account**, **Utente** e **ProfiloUtente**, le entità **CommentoInviato** e **CommentoRicevuto** sono state unificate nella singola classe **Commento**, mentre l'**Evento** si specializza in **EventoAPostiLimitati** ed **EventoAPostiIllimitati**. Vengono presentati i test di unità di **Evento** e **Profilo**.

3.4.1. Test di unità: Evento

```
1 using NUnit.Framework;
2
3 [TestFixture]
4 public class TestEvento
5 {
6     private EventoAPostiLimitati eventoLimitato;
7     private EventoAPostiIllimitati eventoIllimitato;
8     private Utente organizzatore;
9     private Luogo luogoPubblico, luogoPrivato;
10
11     [SetUp]
12     public void Setup()
13     {
14         organizzatore = new Utente(
15             "matteo.gattobigio@studio.unibo.it",
16             "Password1!",
17             new ProfiloUtente(
18                 "Matteo", "Gattobigio",
19                 "Ingegneria Informatica", "Studente appassionato
20                 e vivace",
21                 null
22             )
23         );
24
25         luogoPubblico = new Luogo(
26             "Biblioteca Universitaria",
27             "Via Zamboni 33",
28             TipoDiLuogo.Pubblico
29         );
30
31         luogoPrivato = new Luogo(
32             "Appartamento studio",
```

```

32         "Via Irnerio 12",
33         TipoDiLuogo.Privato
34     );
35
36     eventoLimitato = new EventoAPostiLimitati(
37         "Studio Analisi II",
38         new DateTime(2026, 7, 15, 14, 0, 0),
39         luogoPubblico,
40         organizzatore,
41         2
42     );
43
44     eventoIllimitato = new EventoAPostiIllimitati(
45         "Ripasso Reti di Calcolatori",
46         new DateTime(2026, 7, 20, 10, 0, 0),
47         luogoPrivato,
48         organizzatore
49     );
50 }
51
52 [Test]
53 public void TestCreazioneEvento()
54 {
55     Assert.AreEqual("Studio Analisi II",
56         eventoLimitato.getTitolo());
57     Assert.AreEqual(new DateTime(2026, 7, 15, 14, 0, 0),
58         eventoLimitato.getDataEOrrario());
59     Assert.AreEqual(luogoPubblico,
60         eventoLimitato.getLuogo());
61     Assert.AreEqual(organizzatore,
62         eventoLimitato.getOrganizzatore());
63     Assert.AreEqual(2,
64         eventoLimitato.getPosti());
65     Assert.IsTrue(eventoLimitato.getLuogo().isPubblico());
66 }
67
68 [Test]
69 public void TestEventoIllimitato()
70 {
71     Assert.AreEqual("Ripasso Reti di Calcolatori",
72         eventoIllimitato.getTitolo());
73     Assert.AreEqual(luogoPrivato,
74         eventoIllimitato.getLuogo());
75     Assert.IsTrue(eventoIllimitato.getLuogo().isPrivato());
76 }
77
78 [Test]
79 public void TestAggiungiPartecipante()
80 {
81     Utente partecipante = new Utente(
82         "federico.porpora@studio.unibo.it",

```

```

83         "Password1!",
84         new ProfiloUtente(
85             "Federico", "Porpora",
86             "Ingegneria Informatica", "Studio volentieri in
            gruppo",
87             null
88         )
89     );
90
91     eventoLimitato.aggiungiPartecipante(partecipante);
92
93     CollectionAssert.Contains(
94         eventoLimitato.getPartecipanti(),
95         partecipante
96     );
97     Assert.AreEqual(1,
98         eventoLimitato.getPartecipanti().Count);
99 }
100
101 [Test]
102 public void TestRimozionePartecipante()
103 {
104     Utente partecipante = new Utente(
105         "federico.porpora@studio.unibo.it",
106         "Password1!",
107         new ProfiloUtente(
108             "Federico", "Porpora",
109             "Ingegneria Informatica", "Studio volentieri in
            gruppo",
110             null
111         )
112     );
113
114     eventoLimitato.aggiungiPartecipante(partecipante);
115     eventoLimitato.rimuoviPartecipante(partecipante);
116
117     CollectionAssert.DoesNotContain(
118         eventoLimitato.getPartecipanti(),
119         partecipante
120     );
121     Assert.AreEqual(0,
122         eventoLimitato.getPartecipanti().Count);
123 }
124
125 [Test]
126 public void TestPuoPartecipare()
127 {
128     Utente lucilla = new Utente(
129         "lucilla.lucchi@studio.unibo.it", "Password1!",
130         new ProfiloUtente(
131             "Lucilla", "Lucchi",

```



```

132         "Ingegneria Informatica", "Studio bene quando
133         faccio tante pause",
134         null
135     );
136     Utente p2 = new Utente(
137         "anna.verdi@studio.unibo.it", "Password1!",
138         new ProfiloUtente("Anna", "Verdi", "Matematica", "",
139             null)
140     );
141     Utente p3 = new Utente(
142         "sara.neri@studio.unibo.it", "Password1!",
143         new ProfiloUtente("Sara", "Neri", "Fisica", "", null)
144     );
145     // Evento a posti limitati (2 posti): si riempie
146     Assert.IsTrue(eventoLimitato.puoPartecipare(lucilla));
147     eventoLimitato.aggiungiPartecipante(lucilla);
148     eventoLimitato.aggiungiPartecipante(p2);
149     Assert.IsFalse(eventoLimitato.puoPartecipare(p3));
150
151     // Evento a posti illimitati: accetta sempre
152     eventoIllimitato.aggiungiPartecipante(lucilla);
153     eventoIllimitato.aggiungiPartecipante(p2);
154     eventoIllimitato.aggiungiPartecipante(p3);
155     Assert.IsTrue(eventoIllimitato.puoPartecipare(
156         new Utente(
157             "extra@studio.unibo.it", "Password1!",
158             new ProfiloUtente("Extra", "Utente", "Chimica", "
159                 ", null)
160         ));
161 }
162
163 [Test]
164 public void TestChatGenerata()
165 {
166     Assert.IsNotNull(eventoLimitato.getChat());
167 }
168 }

```

3.4.2. Test di unità: Profilo

```

1 using NUnit.Framework;
2
3 [TestFixture]
4 public class TestProfilo
5 {
6     private ProfiloUtente profiloCompleto, profiloMinimo;
7     private Utente utente;
8

```

```

9      [SetUp]
10     public void Setup()
11     {
12         profiloCompleto = new ProfiloUtente(
13             "Federico", "Porpora",
14             "Ingegneria Informatica",
15             "Studio volentieri in gruppo",
16             "immagine.png"
17         );
18
19         profiloMinimo = new ProfiloUtente(
20             "Luca", "Bianchi",
21             "Matematica",
22             "",
23             null
24         );
25
26         utente = new Utente(
27             "federico.porpora@studio.unibo.it",
28             "Password1!",
29             profiloCompleto
30         );
31     }
32
33     [Test]
34     public void TestCreazioneProfilo()
35     {
36         Assert.AreEqual("Federico",
37             profiloCompleto.getNome());
38         Assert.AreEqual("Porpora",
39             profiloCompleto.getCognome());
40         Assert.AreEqual("Ingegneria Informatica",
41             profiloCompleto.getCorsoDiStudi());
42         Assert.AreEqual("Studio volentieri in gruppo",
43             profiloCompleto.getBiografia());
44         Assert.AreEqual("immagine.png",
45             profiloCompleto.getImmagine());
46     }
47
48     [Test]
49     public void TestProfiloMinimo()
50     {
51         Assert.AreEqual("Luca",
52             profiloMinimo.getNome());
53         Assert.AreEqual("Bianchi",
54             profiloMinimo.getCognome());
55         Assert.AreEqual("Matematica",
56             profiloMinimo.getCorsoDiStudi());
57         Assert.AreEqual("",
58             profiloMinimo.getBiografia());
59         Assert.IsNull(profiloMinimo.getImmagine());

```

```

60     }
61
62     [Test]
63     public void TestSetterProfilo()
64     {
65         profiloCompleto.setNome("Giuseppe");
66         Assert.AreEqual("Giuseppe",
67             profiloCompleto.getNome());
68
69         profiloCompleto.setCognome("Verdi");
70         Assert.AreEqual("Verdi",
71             profiloCompleto.getCognome());
72
73         profiloCompleto.setCorsoDiStudi("Fisica");
74         Assert.AreEqual("Fisica",
75             profiloCompleto.getCorsoDiStudi());
76
77         profiloCompleto.setBiografia("Nuova biografia");
78         Assert.AreEqual("Nuova biografia",
79             profiloCompleto.getBiografia());
80
81         profiloCompleto.setImmagine("nuova_immagine.png");
82         Assert.AreEqual("nuova_immagine.png",
83             profiloCompleto.getImmagine());
84     }
85
86     [Test]
87     public void TestCreazioneUtente()
88     {
89         Assert.AreEqual("federico.porpora@studio.unibo.it",
90             utente.getEmail());
91         Assert.AreEqual(profiloCompleto,
92             utente.getProfiloUtente());
93         Assert.AreEqual(0,
94             utente.getCommentiRicevuti().Count);
95     }
96
97     [Test]
98     public void TestCommenti()
99     {
100         Utente autore = new Utente(
101             "luca.bianchi@studio.unibo.it",
102             "Password1!",
103             profiloMinimo
104         );
105
106         Commento commento = new Commento(
107             autore,
108             "Ottimo compagno di studio",
109             4,
110             new DateTime(2026, 7, 16, 18, 0, 0)

```

```
111         );
112
113         utente.aggiungiCommento(commento);
114
115         CollectionAssert.Contains(
116             utente.getCommentiRicevuti(),
117             commento
118         );
119         Assert.AreEqual(1,
120             utente.getCommentiRicevuti().Count);
121         Assert.AreEqual(4,
122             commento.getValutazione());
123         Assert.AreEqual(autore,
124             commento.getAutore());
125     }
126 }
```

3.5. Progettazione del deployment

Il deployment dell'applicazione sarà così gestito:

- La parte server del programma sarà installata su un server fisico. Per favorire l'eventuale espansione orizzontale del sistema, il processo sarà automatizzato tramite Docker. Il server sarà composto da:
 - Il server web che si occupa della gestione delle richieste REST in arrivo, sviluppato in ambiente .NET;
 - Un DBMS (PostgreSQL) con cui il server web interagirà per la gestione della persistenza.
- La parte client sarà fornita da un server web che servirà l'applicazione frontend sviluppata tramite Vite. Non richiede particolari azioni da parte dell'utente finale, che potrà accedere all'interfaccia tramite un comune browser.

3.5.1. Artefatti

L'applicazione è divisa in due artefatti principali:

- Il software che compone il **backend**, che implementa la logica di business, il livello di traduzione REST e la gestione della persistenza;
- Il software che compone il **frontend**, che implementa le view e il livello di traduzione tra l'interfaccia dell'Utente e le chiamate HTTP dirette al backend.

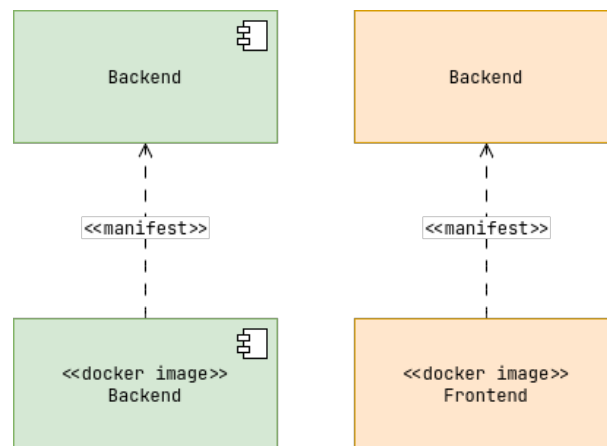


Figura 3.37: Diagramma degli artefatti

Questi artefatti racchiudono tutti i package definiti in fase di analisi e progettazione, fornendo all'Utente le funzionalità per agire sia come Organizzatore che come Partecipante.

3.5.2. Target di deploy

In linea con le scelte architetture, l'applicazione sarà *web-based*, di conseguenza il deploy è diviso in due sezioni:

- Il deploy del **backend** verrà effettuato inizialmente su un singolo nodo, ma sarà eseguito con strumenti di automazione come Docker per favorire una futura espansione orizzontale, con l'eventuale necessità di load balancer.
- Il deploy del **frontend** verrà eseguito analogamente al backend, servito su più nodi indipendenti o piattaforme specializzate il cui compito è distribuire il software ai browser web dei clienti.

I programmi sul server saranno eseguiti da utenti del sistema con permessi minimi, in modo da garantire la sicurezza del nodo contro eventuali attacchi. Le credenziali per il database (PostgreSQL) e le chiavi segrete per l'autenticazione JWT verranno iniettate a livello di deployment tramite variabili d'ambiente in modo da essere facilmente modificabili. Di seguito sono presentati i due possibili scenari di deploy:

- Uno schema base con un singolo nodo che gestisce i container del frontend, del backend e del database (più semplice per lo sviluppo e il testing);

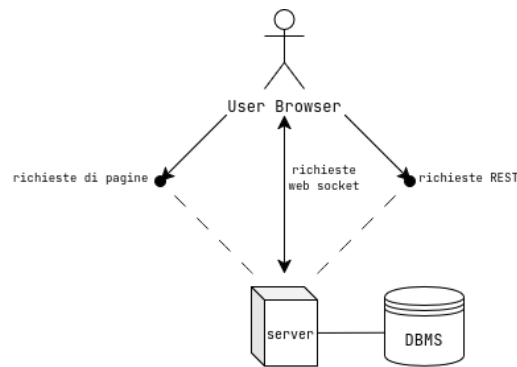


Figura 3.38: Deploy: Schema base

- Uno schema scalabile orizzontalmente, dove i target sono configurati in automatico tramite orchestratori (es. Docker) e un reverse proxy gestisce il bilanciamento del carico tra i vari nodi REST.

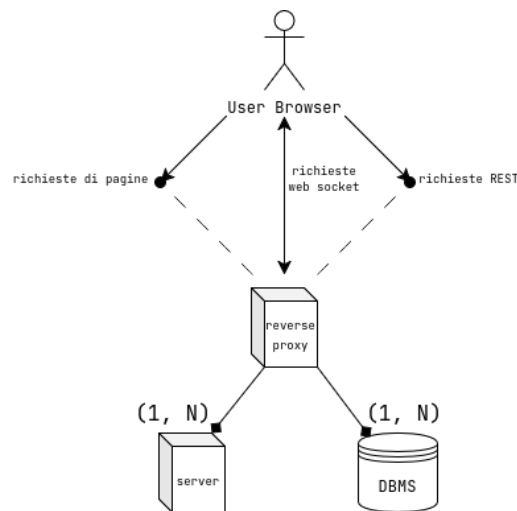


Figura 3.39: Deploy: Schema scalabile